



Persistence-based topological optimization: a survey

Mathieu Carriere, Yuichi Ike, Théo Lacombe, Naoki Nishikawa

► To cite this version:

Mathieu Carriere, Yuichi Ike, Théo Lacombe, Naoki Nishikawa. Persistence-based topological optimization: a survey. 2026. <hal-05559932>

HAL Id: hal-05559932

<https://hal.science/hal-05559932v1>

Preprint submitted on 23 Mar 2026

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY-SA 4.0 - Attribution - ShareAlike - International License

Persistence-based topological optimization: a survey

Mathieu Carrière¹, Yuichi Ike², Théo Lacombe³, and Naoki Nishikawa⁴

¹`mathieu.carriere@inria.fr`, DataShape, Centre Inria d’Université Côte d’Azur, Sophia-Antipolis, France

²`ike@ms.u-tokyo.ac.jp`, Graduate School of Mathematical Sciences, The University of Tokyo, 3-8-1 Komaba Meguro-ku Tokyo 153-8914, Japan

³`theo.lacombe@univ-eiffel.fr`, Laboratoire d’Informatique Gaspard Monge, Université Gustave Eiffel, CNRS, F-77454 Marne-la-Vallée, France

⁴`nishikawa-naoki259@g.ecc.u-tokyo.ac.jp`, Graduate School of Information Science and Technology, The University of Tokyo, 7-3-1 Hongo Bunkyo-ku Tokyo 113-0033, Japan; RIKEN AIP, Nihonbashi 1-chome Mitsui Building, 15th floor, 1-4-1 Nihonbashi, Chuo-ku, Tokyo 103-0027, Japan

Abstract

Computational topology provides a tool, *persistent homology*, to extract quantitative descriptors from structured objects (images, graphs, point clouds, etc). These descriptors can then be involved in optimization problems, typically as a way to incorporate topological priors or to regularize machine learning models. This is usually achieved by minimizing adequate, topologically-informed losses based on these descriptors, which, in turn, naturally raises theoretical and practical questions about the possibility of optimizing such loss functions using gradient-based algorithms. This has been an active research field in the topological data analysis community over the last decade, and various techniques have been developed to enable optimization of persistence-based loss functions with gradient descent schemes.

This survey presents the current state of this field, covering its theoretical foundations, the algorithmic aspects, and showcasing practical uses in several applications. It includes a detailed introduction to persistence theory and, as such, aims at being accessible to mathematicians and data scientists newcomers to the field. It is accompanied by an open-source library¹ which implements the different approaches covered in this survey, providing a convenient playground for researchers to get familiar with the field.

Keywords: Topological Data Analysis, Computational Topology, Persistent Homology, Optimization, Machine Learning.

Acknowledgments. YI and NN are supported by JSPS Grant-in-Aid for Transformative Research Areas (A) Grand Number JP22H05107 and JST, CREST Grant Number JPMJCR24Q1, Japan. TL is supported by l’Agence Nationale de la Recherche (ANR) under grant TheATRE ANR-24-CE23-7711. MC is supported by l’Agence Nationale de la Recherche (ANR) under grants TopModel ANR-23-CE23-0014 and 3IA ANR-23-IACL-0001.

¹https://github.com/git-westriver/benchmark_ph_optimization

Contents

1	Introduction	2
2	Background on Topological Data Analysis	5
2.1	Simplicial complexes, homology groups and filtrations	5
2.2	Persistent homology and persistence diagrams	9
2.3	Persistence computation	12
3	Differential framework for persistence diagrams	14
3.1	Formulation of differentiability	14
3.2	Examples of topological losses	16
3.3	Persistence diagrams of parametrized families of filtrations	19
3.4	Stratified filtrations and directional differentiability	21
3.5	Examples of stratified filtrations and their differentials	22
4	Optimizing persistence-based objective functions	24
4.1	Vanilla gradient descent	24
4.2	Stratified gradient descent	26
4.3	Big-step gradient descent	30
4.4	Gradient extensions	32
4.4.1	Smoothing gradient with downsampling	33
4.4.2	Extending gradient with diffeomorphic interpolation	34
4.5	Summary table	35
5	An overview of some applications	36
5.1	Filtration learning	36
5.1.1	Filtration learning for images	37
5.1.2	Filtration learning for graphs	37
5.1.3	Filtration learning for geometric complexes	37
5.2	Topological regularization	39
5.2.1	Topological penalization of model complexity	40
5.2.2	Penalization to favor topological priors	41
6	Numerical illustrations	43
6.1	Illustration of the different gradient descent methods	44
6.2	Topological Autoencoder	47
7	Conclusion	50

1 Introduction

Topological Data Analysis (TDA) is a field of data science that focuses on the characterization, inference, and encoding of topological features (such as connected components, branches, loops, voids, etc.) in structured objects (such as graphs, point clouds, time series, etc.). Such topological features have often been shown to carry complementary information to traditional data descriptors in downstream machine learning tasks [82, 109, 138, 3, 128], and to be able to significantly improve models in terms of predicting performance in a wide range of applications including computer graphics [99, 33, 121], analysis of machine learning models [110, 77, 14, 5], material and molecular science [63, 112, 95], or computational biology [23, 8, 42] for example.

However, the advent of deep learning has shifted the machine learning paradigm from manually crafting data features to the automatic learning of such features through the optimization of neural

network architectures with *gradient descent*. Hence, among the different directions driving current TDA research, the ability to *differentiate functions depending on topological features* has recently appeared as an important question for smooth integration of TDA into modern deep learning pipelines. This field of research, called *persistence-based topological optimization*, has recently seen several breakthroughs, both in terms of theoretical developments and numerical implementations and experiments. In essence, persistence-based topological optimization involves the theoretical derivation of *differentiability properties* of topological features built using an algebraic machinery called *persistent homology*, and practical implementations of corresponding, well-defined *gradients* associated to their computation. Note that this concept should not be confused with “topological optimization” as used in material science [115], which consider the optimization of topological features not based on persistent homology (though persistent homology is increasingly used in that field as well, see [120, 71] and Section 5 for instance). For the sake of concision, we will nonetheless use the terminology *topological optimization* to refer to persistence-based topological optimization in this survey. Figure 1.1 provides a general overview on the methodology we will present in this survey.

Such gradients can then be used subsequently in several different tasks based on gradient descent, such as filtration learning [29, 64, 68] or model regularization [38, 79]. While vanilla methods for computing these topological gradients have been identified since 2016 thanks to the seminal work of Gameiro, Hiraoka and Obayashi [59], it quickly appeared that such natural first approaches were limited both in terms of lack of theoretical guarantees and erratic behaviors in numerical experiments. The difficulty associated to addressing these limitations left these questions open in the TDA community for a few years until several answers were proposed recently, with new developments still appearing as of today.

In this survey, our aim is to present in a unified and consistent framework the different methods, with their theoretical results and algorithms, that are currently available in the literature for performing topological optimization, as well as to provide a all-in-one-place public library² where these different approaches are implemented.

Outline. Section 2 provides a general introduction to Topological Data Analysis, with a focus on the tools required later in this survey. We recall the construction of persistence diagrams (PDs) from filtrations on simplicial complexes, including their algorithmic computation as it will be core to define and compute gradients involving PDs.

Section 3 is dedicated to the construction of a differential structure for maps valued in and from the space of persistence diagrams, following the work of Leygonie, Oudot, and Tillman [80]. In essence, this construction allows the user to treat the byproduct of persistence homology, a topological descriptor called *persistence diagram* (PD)—when it comes to compute differentials—as vectors in an Euclidean space by identifying them with their *lift*. The main result is that even if several lifts are possible, the choice has no impact when computing the (usual) gradient of composite maps going through the non-linear space of persistence diagrams. This formal construction theoretically justifies practical implementations in which persistence diagrams are indeed manipulated as arrays (i.e., vectors).

Section 4 makes use of the now defined gradients to perform (gradient-based) topological optimization. We start by reviewing the so-called *vanilla* gradient descent (with fixed or decreasing step size) and its practical limitations, and then present several variations of it that have been recently introduced in the literature and that either strengthen the theoretical guarantees or the numerical efficiency of the vanilla approach.

Section 5 proposes an overview of practical applications involving topological optimization in a wide range of settings: filtration learning, dimensionality reduction, computer vision and regularization of objective functions used in machine learning.

Eventually, Section 6 showcases the different methods presented in Section 4 in practice, in the context of topological optimization for point clouds. All methods have been (re)implemented in an all-in-one Python library that may be of its own interest for future research in the field.

²https://github.com/git-westrivier/benchmark_ph_optimization

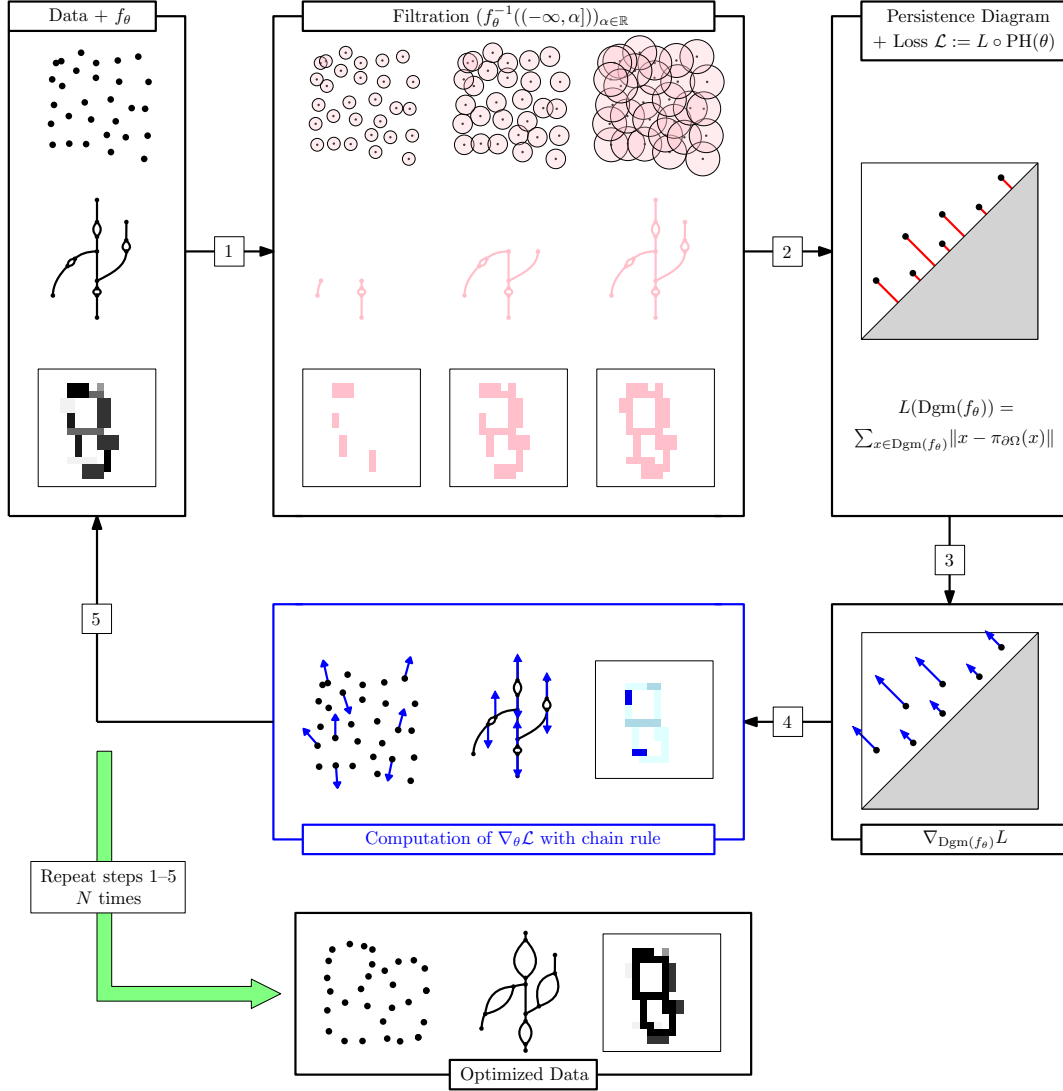


Figure 1.1: Standard topological optimization scheme typically used in deep learning pipelines, with examples on point cloud, graph and image data. Note that step 1 needs not be at the beginning of the pipeline, and can be incorporated at any stage (this happens when input data are computed as the outputs of some other models, such as, e.g., image filters automatically computed by CNNs). The blue square is the main theoretical question that we discuss in this survey.

Notations. We summarize below the core notation that will be used throughout the paper:

- $\mathbb{N} = \{0, 1, 2, \dots\}$ denotes the set of natural numbers including 0 and $\mathbb{R}$ denotes the set of real numbers.
- For a finite set A , the cardinality of A is denoted by $|A|$.
- $\mathcal{D}$ denotes the space of persistence diagrams (PDs), that are finite multisets³ supported on the closed extended half-plane $\Omega := \{(b, d) \in \overline{\mathbb{R}}^2 \mid b \leq d\}$ (Definition 2.19). The notation $\mathcal{D}^o$ denotes ordinary persistence diagrams, i.e., PDs with points supported on the open half-plane $\{(b, d) \in \mathbb{R}^2, b < d\}$. It is equipped with partial matching metrics denoted by FG_q , $q \in [1, +\infty]$ (Definition 2.20). The empty persistence diagram is denoted by $\emptyset$.
- If $x \in \Omega$ belongs to the support of a persistence diagram $\alpha \in \mathcal{D}$, $m(x)$ denotes its multiplicity.
- In equations, PH typically denotes a map from a manifold M (typically $M = \mathbb{R}^d$) valued in $\mathcal{D}$, L typically denotes a map from $\mathcal{D}$ to some manifold N (typically $N = \mathbb{R}^{d'}$ with $d' = 1$), and $\mathcal{L}$ denotes a composite map $\mathcal{L} := L \circ \text{PH}: M \rightarrow N$.
- Given a simplicial complex K (Definition 2.1), the set of filtrations on K is denoted by Filt_K (Definition 2.7). Given $f \in \text{Filt}_K$, $\text{Dgm}(f) \in \mathcal{D}$ denotes the persistence diagram induced by f on K (Definition 2.7), and $\text{Dgm}_p(f)$ denotes its persistence diagram restricted to homology dimension p .
- $\mathbb{F}_2$ denotes the field with two elements, i.e., $\mathbb{Z}/2\mathbb{Z}$.

2 Background on Topological Data Analysis

The main backbone of TDA is the so-called *persistent homology* (PH) theory, which can be used to define the main TDA descriptors called *persistence diagrams* (PDs). Hence, in this section, we briefly recall the basics associated to the construction of PDs from PH. We will then discuss their differentiability properties in the following sections.

2.1 Simplicial complexes, homology groups and filtrations

Generally speaking, the main difficulty for defining quantitative topological descriptors comes from the fact that topological information is usually only well-defined for continuous spaces, as opposed to discrete data (such as point clouds, graphs, meshes, etc.) that one has to deal with in data science. As a solution to this issue, the so-called *simplicial complexes* are the most basic bricks of TDA, as they are formal objects that can be handled by computers thanks to their combinatorial nature, yet for which topological information, in the form of *homology groups*, can still be defined.

Definition 2.1. Let V be a finite set. A *simplicial complex* whose vertex set is V is a collection K of subsets of V satisfying the following conditions:

- (i) $\emptyset \notin K$;
- (ii) for any $v \in V$, $\{v\} \in K$;
- (iii) if $\sigma \in K$ and $\emptyset \neq \tau \subseteq \sigma$, then $\tau \in K$.

The set V is called the *vertex set* of K , and is denoted by $V(K)$. Moreover, any $\sigma \in K$ with cardinality $|\sigma| = p + 1$ is called a *simplex of dimension p* , or p -simplex for short, and the set of all p -simplices, denoted by $\text{Sk}_p(K)$, is called the *p -skeleton* of K . The dimension of K is defined as the maximum of the dimension of $\sigma \in K$.

³i.e., sets where repetition of points is allowed.

Example 2.2. Graphs (represented as a set of vertices V and a set of edges $E \subset V \times V$) are particular cases of simplicial complexes, namely simplicial complexes of dimension 1.

We can now define the *homology groups* of simplicial complexes. In essence, these groups aim at encoding the numbers of *holes* of the simplicial complexes using linear algebra. Indeed, these holes, or *cycles*, can be entirely characterized as the elements of the kernel of a linear map called the *boundary operator*. In words, cycles are linear combinations of simplices, or *chains*, whose boundaries are null.

Definition 2.3. Let K be a simplicial complex. Let $\mathbb{F}_2$ be the field consisting of two elements. The vector space of p -chains of K , denoted by $C_p(K)$, is the $\mathbb{F}_2$ -vector space with basis $\text{Sk}_p(K)$. The *boundary operator* $\partial_p: C_p(K) \rightarrow C_{p-1}(K)$ is the linear operator defined on any p -simplex $\sigma = \{v_0, v_1, \dots, v_p\}$ as:

$$\partial_p(\sigma) = \sum_{i=0}^p \{v_0, \dots, v_{i-1}, v_{i+1}, \dots, v_p\}.$$

Finally, we call $\text{Ker } \partial_p$ the vector space of p -cycles of K , and $H_p(K) := \text{Ker } \partial_p / \text{Im } \partial_{p+1}$ the p -th homology group⁴ of K .

Considering the quotient of $\text{Ker } \partial_p$ by $\text{Im } \partial_{p+1}$ is motivated by the relation $\partial_p \circ \partial_{p+1} = 0$. Doing so, non-zero elements of $H_p(K)$ correspond to (equivalence class of) p -cycles that are not obtained as boundaries of $(p+1)$ -chains.

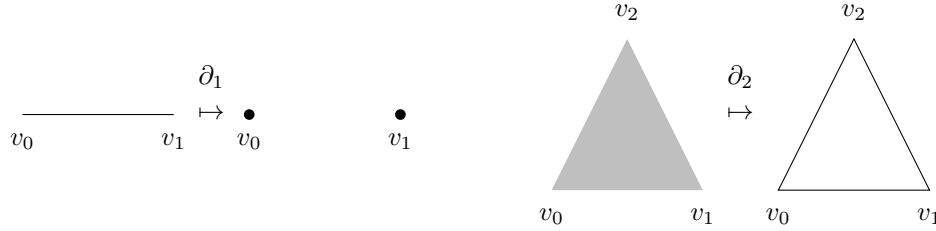


Figure 2.1: Boundary operator

Example 2.4. Let $\{v_0, v_1\}$ and $\{v_0, v_1, v_2\}$ be the 1-simplex and the 2-simplex displayed in Figure 2.1. We have

$$\partial_1\{v_0, v_1\} = \{v_1\} + \{v_0\}$$

and

$$\partial_2\{v_0, v_1, v_2\} = \{v_1, v_2\} + \{v_0, v_2\} + \{v_0, v_1\}.$$

These imply

$$\partial_1 \circ \partial_2(\{v_0, v_1, v_2\}) = (\{v_2\} + \{v_1\}) + (\{v_2\} + \{v_0\}) + (\{v_1\} + \{v_0\}) = 0,$$

since the coefficient field is $\mathbb{F}_2$.

Example 2.5. Figure 2.2 showcases a simplicial complex K embedded in $\mathbb{R}^3$, topologically equivalent to a torus. Here, the set of vertices V is $\{\{a\}, \{b\}, \dots, \{i\}\}$. Its 1-skeleton (edges) are depicted by blue lines, and its 2-skeleton (faces, e.g., $\{a, b, e\}$ and $\{a, e, f\}$) are depicted in shadow-gray. There is no higher dimensional simplex, i.e., K is of dimension 2. The 1-chains $\{a, b\} + \{b, c\} + \{c, a\}$ and $\{d, e\} + \{e, f\} + \{f, d\}$, displayed in red are both 1-cycles (i.e., belong to $\text{ker } \partial_1$), but happen to be equivalent in the homology group H_1 as they are both in the boundary of a same 2-chain (the “cylinder”

⁴Note that H_1 is actually a vector space, due to the fact that chains are built on top of the field $\mathbb{F}_2$. See Remark 2.6.

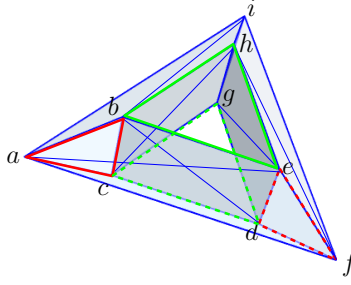


Figure 2.2: A simplicial complex (whose geometric realization is that of a torus embedded in $\mathbb{R}^3$).

obtained by restricting K to $\{a, b, c, d, e, f\}$). A similar comment holds for the 1-cycles $\{b, h, e\}$ and $\{c, d, g\}$ (displayed in green). Furthermore, red and green cycles are nonetheless independent from each others, and thus yield two different elements in H_1 , which are actually the two generators of that group, which is thus a vector space of dimension 2.

Remark 2.6. One can also define homology groups over a ring other than $\mathbb{F}_2$, e.g., $\mathbb{Z}$, for which we need to equip each simplex with an orientation and modify the boundary operator with $\pm$ -signs. This allows to identify *torsion subgroups* in the homology groups. This allows to distinguish between, e.g., a torus and a Klein bottle. As most applications of topological data analysis use $\mathbb{F}_2$, we specify our exposition to this choice in this survey and refer to [94] for a thorough discussion on that topic. Note that using a field (instead of a more general ring) as coefficients to build our chains implies that the homology groups are actually (finite dimensional) *vector spaces*. The list of dimensions of H_p for $p \in \mathbb{N}$ are called *Betti numbers* of K , denoted by $(\beta_p)_p$. Betti numbers are topological invariants, in that two objects that are homotopy equivalent (i.e., one can be obtain from the other by a smooth deformation, see [56, Ch. III.2] for a formal definition) have the same Betti numbers. In Example 2.5, one obtains $\beta_0 = 1$, $\beta_1 = 2$, $\beta_2 = 1$, and $\beta_p = 0$ for $p \geq 3$, accounting the for that that a torus has one connected component, two (independent) generating loops, and one cavity.

In general, datasets are not made of objects represented as simplicial complexes, preventing one from straightforwardly use this construction in practice. Furthermore, building a specific simplicial complex K on top of objects (say, point clouds) often relies on user-based arbitrary choices (e.g., connecting points that are closer than an arbitrary threshold ϵ). In order to avoid depending on such priors, the main idea of *persistent homology* (PH) is to look at the homology groups of a *growing sequence* of subcomplexes of a fixed (usually large) complex K , called a *filtration* of K .

Definition 2.7. A *filtration* of a simplicial complex K is an indexed family of complexes $(K_t)_{t \in \mathbb{R}}$ satisfying $K_t \subseteq K$ for all t , and $K_s \subseteq K_t$ whenever $s \leq t$. Whenever the intervals $I_\sigma := \{t \in \mathbb{R} \mid \sigma \in K_t\}$ attain their infimums, a filtration can be described by the function $f: K \rightarrow \mathbb{R}$ defined with $f(\sigma) := \min I_\sigma$. Reciprocally, any function $f: K \rightarrow \mathbb{R}$ satisfying $f(\sigma) \leq f(\sigma')$ for any pair $\sigma \subseteq \sigma'$ induces a filtration with $K_t := \{\sigma \in K \mid f(\sigma) \leq t\}$. The set of filtrations on K is denoted by Filt_K .

Remark 2.8. In practice, most of standard filtrations in TDA are provided through functions. Hence, we will use the terms filtration and function satisfying the condition in Definition 2.7 interchangeably hereafter, even though the former is more general than the latter. From now on, we will implicitly assume that all filtrations considered are actually induced by functions, i.e., the intervals I_σ defined above do attain their infimums.

Practically speaking, filtrations can be interpreted in different ways. For instance, in the case of Vietoris–Rips filtrations (see Example 2.11 below), these growing complexes can be seen as approxima-

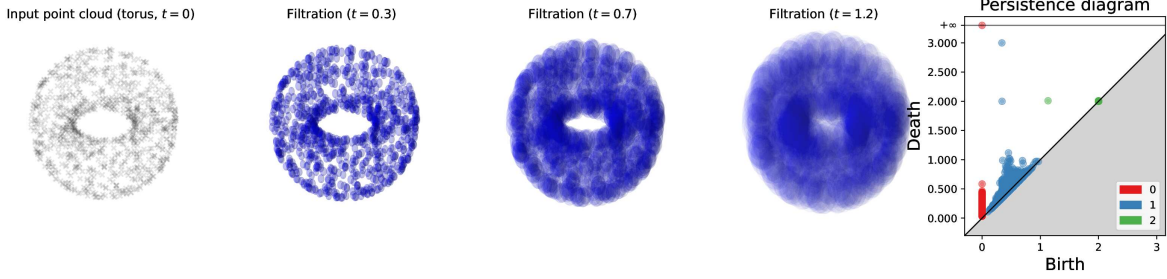


Figure 2.3: Illustration of the Čech filtration and resulting persistence diagram. From left to right, an input point cloud (sample of $n = 1000$ points on a torus), increasing sublevel sets of the distance function to that point cloud, i.e., $\bigcup_{x \in X} B(x, t)$, and eventually on the right the corresponding persistence diagram (see Definition 2.19). Colors in the persistence diagram correspond to different homology dimension; the prominent point in red (death = $+\infty$) accounts for the unique connected component of the (underlying) manifold. The two prominent points in blue correspond to the two generating loops in H_1 (recall Example 2.5)—their death coordinate indicating a “small” radius $r_1 = 2$ and a large radius of $r_2 = 5$, and the green one accounts for the cavity (H_2 , also filled in at $t = 2$). Points closer to the diagonal $\{b = d\}$ (in particular, red and blue ones) correspond to less persistent features accounting for the sampling (they would vanish, i.e., collapse on the diagonal, when $n \rightarrow +\infty$).

tions of a topological space at different scales; computing the homology groups for all these complexes is thus a way to avoid computing the homology groups at a single arbitrary scale. We provide below several examples of filtration that are routinely used in the TDA literature for various types of data (point clouds, images, etc.).

Example 2.9 (Čech filtration on point clouds). Let $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ be a finite point cloud and $t \in \mathbb{R}$. The Čech simplicial complex $C[X]_t$ is defined by

$$\{x_{i_1}, \dots, x_{i_p}\} \in C[X]_t \iff \bigcap_{j=1}^p B(x_{i_j}, t) \neq \emptyset. \quad (2.1)$$

The family $(C[X]_t)_{t \in \mathbb{R}}$ is a filtration, which is called the *Čech filtration* of X . An important result referred to as the *nerve theorem* states that $C[X]_t$ has the same homology as the set $\bigcup_{x \in X} B(x, t) =: C[X]_t \subset \mathbb{R}^d$ (see [56, §III.2] and references therein). When X is sampled almost-uniformly on a sufficiently regular submanifold $M \subset \mathbb{R}^d$ with n large enough, $C[X]_t$ has itself the same homology as M for a wide range of values of t [93], making this filtration a natural candidate to build estimators to infer topological properties of M . See [35, 16, 53, 47] and Figure 2.3.

Remark 2.10. An important drawback of the Čech filtration that limits its use in practical applications lies in its computational efficiency: asserting whether the intersection of balls in (2.1) is non-empty becomes quickly intractable when the ambient dimension d increases.

Example 2.11 (Vietoris–Rips (VR) filtration on point clouds). Let $X = \{x_0, \dots, x_n\} \subset \mathbb{R}^d$ be a finite point cloud and $t \in \mathbb{R}$. The Vietoris–Rips simplicial complex $R[X]_t$ at scale t is the simplicial complex with vertex set $V(R[X]_t) = X$ and whose p -skeleton is prescribed by:

$$\{x_{i_0}, \dots, x_{i_p}\} \in R[X]_t \iff \|x_{i_j} - x_{i_k}\| \leq 2t \text{ for any } j, k \in \{0, \dots, p\}.$$

The family $\{R[X]_t\}_{t \in \mathbb{R}}$ is a filtration, which is called the *Vietoris–Rips (VR) filtration* of X . It can be equivalently described with the function $f_{\text{VR}}(X)$ defined as $f_{\text{VR}}(X)(\{x_{i_0}, \dots, x_{i_p}\}) = \max\{\|x_{i_j} - x_{i_k}\|/2 \mid j, k \in \{0, \dots, p\}\}$.

Remark 2.12. An important property of the VR filtration is that—in contrast to the Čech filtration—it only depends on the pairwise distance matrix $(\|x_i - x_j\|)_{1 \leq i, j \leq n}$. This has two interesting consequences: (i) computing $R[X]_t$ becomes linear in the ambient dimension⁵ d (the cost of computing a distance), a sharp advantage over the Čech filtration, (ii) the Rips filtration can be faithfully adapted to general metric spaces (e.g., a Riemannian manifold equipped with its geodesic distance). This flexibility of the VR filtration enables the design of many variants: considering the geodesic distance on a triangulated mesh [97], e.g., weighting the radius by some local information based on density [4, 92], etc.

Example 2.13 (Height filtration). Let K be a simplicial complex embedded in $\mathbb{R}^d$ and let $x_1, \dots, x_n$ denote its vertices. Pick a parameter $t \in \mathbb{R}$ and a direction $\theta \in \mathbb{S}^{d-1}$ (the unit sphere in $\mathbb{R}^d$). Then, define

$$H[K, \theta]_t := \left\{ \{x_{i_1}, \dots, x_{i_p}\} \in K \mid \max_{1 \leq j \leq p} \langle x_{i_j}, \theta \rangle \leq t \right\}.$$

The family $(H[K, \theta]_t)_{t \in \mathbb{R}}$ defines the *height filtration* of K in the direction θ . Looking at all possible directions $\theta \in \mathbb{S}^{d-1}$ is analogous to considering a Radon transform of K , and it can be proved that this transform is injective when $d = 2, 3$, i.e., $(H[K, \theta]_t)_t = (H[K', \theta]_t)_t, \forall \theta \in \mathbb{S}^{d-1} \Leftrightarrow K = K'$ [125].

Example 2.14 (Filtrations on graphs). As graphs are particular instances of simplicial complexes, many specific filtrations have been designed to deal with them. Values on vertices and/or edges can typically be obtained as output of diffusion processes (heat equation, wave equation, etc.) or geodesic distances; see for instance [6, 81, 57, 30]. These filtrations also extend naturally to triangulations (as they can be seen as “higher-dimensional” graphs with triangles in addition to vertices and edges) in order to deal with, e.g., 3D meshes [33, 105, 117].

Example 2.15 (Filtrations on images). Images are structured objects and can be easily turned into simplicial complexes by simply turning every pixel into two right triangles glued along their diagonals, and attaching these pairs of triangles together using pixel connectivity. Additionally, an even simpler way is to use specific complexes, called *cubical complexes*, that are specifically tailored for array data. The pixel values (such as, e.g., gray scale, one of the RGB channels, or segmentation masks) are then natural functions that can be used to filter images, and that are particularly efficient in generative models [132, 60, 10]. This approach extends straightforwardly to higher-dimensional images, such as scans with voxels [116].

2.2 Persistent homology and persistence diagrams

Let $\mathcal{K} := (K_t)_{t \in \mathbb{R}}$ be a filtration of a finite simplicial complex K . The main idea of persistent homology is to track the appearance and disappearance of topological features in the filtrations, seen as elements in the corresponding homology groups. For $s \leq t$, we denote the inclusion between K_s and K_t by $\iota_{t,s}: K_s \hookrightarrow K_t$. The map $\iota_{t,s}$ induces a linear map $(\iota_{t,s})_*: H_p(K_s) \rightarrow H_p(K_t)$ between the corresponding homology groups (a property referred to as functoriality). Persistent homology (PH) is then defined as the family of these homology groups and their connecting linear maps [56, 96].⁶

Definition 2.16 (Persistent homology). Let $\mathcal{K} = (K_t)_{t \in \mathbb{R}}$ be a filtration of a simplicial complex K . The p -th persistent homology of $\mathcal{K}$ is defined as the pair $((H_p(K_t))_{t \in \mathbb{R}}, (\iota_{s,t})_{t \leq s})$ of the family of homology groups $(H_p(K_t))_{t \in \mathbb{R}}$ together with the family of linear maps $((\iota_{s,t})_*)_{t \leq s}$ induced by inclusions.

⁵Note however that the *statistical efficiency* of the Vietoris–Rips filtration—when it comes to infer the topology of an underlying submanifold $M \subset \mathbb{R}^d$ on which X may be sampled—decreases when the *intrinsic* dimension of M increases [9, 53].

⁶Note that the term *persistence module* is often encountered in the TDA literature as well to denote a family of vector spaces connected by linear maps (but not necessarily indexed over $\mathbb{R}$).

Intuitively, the linear map $(\iota_{s,t})_*$ encodes the correspondence between the bases of $H_p(K_s)$ and $H_p(K_t)$: if a non-trivial cycle in these bases is present in both $H_p(K_s)$ and $H_p(K_t)$ (at the i -th and j -th positions), then the entry at location (j, i) in the matrix representation of $(\iota_{s,t})_*$ is 1. Similarly, if the i -th non-trivial cycle in the basis of $H_p(K_s)$ has become a boundary in K_t , then the i -th column of $(\iota_{s,t})_*$ is zero, and if the j -th non trivial cycle in the basis of $H_p(K_t)$ has not yet appeared in K_s , then the j -th row of $(\iota_{s,t})_*$ is zero.

The information captured by PH can be summarized by finding appropriate bases whose elements correspond exactly to the appearing or disappearing cycles in the filtration. The following theorem is a consequence of the explicit construction we present in Subsection 2.3 and can be found for instance in [96, Ch. 2].

Theorem 2.17. Let $\mathcal{K} = (K_t)_{t \in \mathbb{R}}$ be a filtration of a finite simplicial complex K . For any $t \in \mathbb{R}$, we let $\partial_{p,t} := \partial_p|_{C_p(K_t)}$, i.e., the restriction of ∂_p to the vector subspace $C_p(K_t) \subseteq C_p(K)$. Moreover, for each $z \in \text{Ker } \partial_{p,t}$, let $[z]_t$ denote the corresponding homology class in $H_p(K_t)$. Then, there exists $m \in \mathbb{N}$, a sequence $\{z_i\}_{i=1}^m$ on $C_p(K)$ and a multiset $\{(b_i, d_i)\}_{i=1}^m$ of pairs on $\mathbb{R} \cup \{+\infty\}$ that satisfy the following three conditions, for any $t \in \mathbb{R}$:

- (i) $b_i \leq t \iff z_i \in \text{Ker } \partial_{p,t}$,
- (ii) $t \geq d_i \iff z_i \in \text{Im } \partial_{p+1,t}$,
- (iii) let $I_t := \{i \in \{1, \dots, m\} \mid t \in [b_i, d_i]\}$, then the set $\{[z_i]_t \mid i \in I_t\}$ is a basis of $H_p(K_t)$.

Additionally, the multiset $\{(b_i, d_i)\}_{i=1}^m$ that satisfies the conditions above is unique (up to permutations).

The multiset $\{(b_i, d_i)\}_{i=1}^m$ defined in Theorem 2.17 is called *persistence diagram* (PD), and is usually represented as a multiset of points in the extended plane by using the b_i 's and d_i 's as coordinates. These coordinates are usually called the *birth times* and *death times* of the corresponding cycles, or topological features. For any filtration $f: K \rightarrow \mathbb{R}$, we let $\text{Dgm}(f)$ denote the corresponding PD.

Remark 2.18. A point (b, d) in a persistence diagram is said to be *essential* if its second coordinate is $d = +\infty$. Intuitively, the essential points in a persistence diagram correspond to the topological features of the final simplicial complex K at the end of the filtration. For instance, when using the Vietoris–Rips filtration (Example 2.11, see also Figure 2.3), the final simplicial complex is the complete simplicial complex on n points (i.e., the simplicial complex such that any subset of the vertices is a simplex), which has one connected component and no other higher-dimensional cycle. Hence, there is always exactly one essential point whose coordinates are $(0, +\infty)$ in homology dimension $p = 0$: this point accounts for the single connected component created at birth time $b = 0$ and persisting “forever”.

In the context of topological optimization, these essential points are barely used (i.e., the loss functions that are typically used in that context do not depend on such points), and one rather focuses on *ordinary* points in PDs, i.e., points (b, d) with $b < d < +\infty$. On the other hand, taking the essential part into account is important in some statements regarding the construction of persistence diagrams, such as Proposition 3.21. We thus decide to make an explicit distinction between these two types of objects to stress whenever one should consider PDs in their greatest generality (i.e., including essential parts) or may simplify by restricting to the ordinary points in PDs.

This yields the following definition.

Definition 2.19. A persistence diagram (PD) is a finite multiset of points supported on the extended closed half-plane $\bar{\Omega} := \{(b, d) \in \mathbb{R}^2 \mid b \leq d\}$. The set of persistence diagram is denoted by $\mathcal{D}$. We let $\mathcal{D}$ denote the set of all persistence diagrams.

An *ordinary* persistence diagram is a persistence diagrams with no essential part, i.e., a finite multiset of points supported on the open half-plane $\Omega := \{(b, d) \in \mathbb{R}^2 \mid b < d\}$. We let $\mathcal{D}^o$ denote the

set of all ordinary persistence diagrams. The *multiplicity* of a point x belonging to the support of a persistence diagram $\text{spt}(\alpha)$, $\alpha \in \mathcal{D}$, is denoted by $m(x)$.

Eventually, we let $\mathcal{D}_m$ (resp. $\mathcal{D}_m^o$) denote the subset of $\mathcal{D}$ (resp. $\mathcal{D}^o$) made of persistence diagram with at most $m \in \mathbb{N}$ points.

Moreover, ordinary persistence diagrams can be compared using *partial matching* distances.

Definition 2.20. Let $\alpha, \beta \in \mathcal{D}^o$ be two ordinary persistence diagrams and $q \in [1, +\infty)$. The q -th *diagram distance* between α and β is defined as

$$\text{FG}_q(\alpha, \beta) = \inf_{\pi \in \Gamma(\alpha, \beta)} \left(\sum_{x \in \alpha \cup \partial\Omega} \|x - \pi(x)\|^q \right)^{\frac{1}{q}}, \quad (2.2)$$

where $\|\cdot\|$ denote the q' -norm in $\mathbb{R}^2$ (typically $q' = 2$ in applications), $\Gamma(\alpha, \beta)$ is the set of partial matching between α and β , i.e., bijections between $\alpha \cup \partial\Omega$ and $\beta \cup \partial\Omega$, and $\partial\Omega := \{(t, t), t \in \mathbb{R}\}$ is the boundary of Ω , typically referred to as “the diagonal”. When $q = +\infty$, the sum becomes a supremum and the distance FG_∞ is referred to as the *bottleneck distance* between persistence diagrams. See also Figure 2.4.

Remark 2.21. When $q < \infty$, the distance FG_q is often referred to as the *Wasserstein distance* between persistence diagrams in the TDA literature [56, Ch. VIII.2], due to its similarity with the Wasserstein distance between probability measures used in optimal transport (see for instance [113, Ch. 5]). Interestingly, the distance FG_q was actually introduced initially by Figalli and Gigli [58] in a context unrelated to TDA. The connection with the metrics used in topological data analysis has been made later in [54]. We use the somewhat unusual notation FG to stress the distinction between these distances and their counterpart in optimal transport literature.

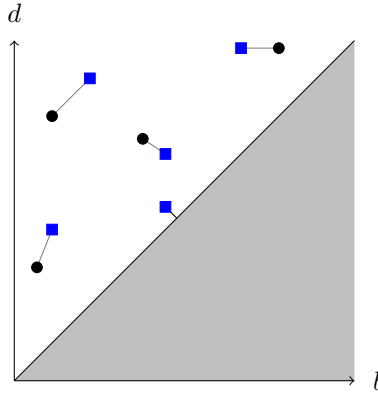


Figure 2.4: Partial matching distance between PDs.

Remark 2.22. The metric space $(\mathcal{D}_m^o, \text{FG}_2)$ enjoys several properties [86, 124, 41, 54]: it is a Polish space, non-negatively curved, with geodesics known in closed-form (provided that one can compute the optimal partial matching π between two diagrams in (2.2)). It implies that several statistical tools, such as probability distributions supported on $\mathcal{D}_m^o$ and their (Fréchet) means are well-defined [124, 123, 90, 25], enabling the use of some standard algorithms that only requires a metric structure, such as k -means clustering [85, 50, 76, 24].

However, this metric space does not exhibit a linear (Hilbert) structure and a fruitful line of works suggests that it cannot be embedded in such spaces (in the greatest generality) without distorting

arbitrarily the metric if the number of points in the persistence diagram goes to infinity or to 0 [22, 88, 27, 87, 106]. This means that most machine learning methods and models, which are designed for input data that belongs to a (often finite-dimensional) vector space, cannot be used faithfully when working with persistence diagrams.

Two natural workarounds consist of (i) either explicitly pushing persistence diagrams into a linear space nonetheless (hence necessarily loosing some geometric information as diagram distances are arbitrarily distorted) by manually designing a *vectorization* or *representation* of persistence diagrams, that is, a map $\Phi: \mathcal{D} \rightarrow \mathcal{H}$, where $\mathcal{H}$ is a Hilbert space (see, e.g., [20, 2, 37, 126, 65, 30]), or (ii) designing such a map implicitly using *kernel methods* (see, e.g., [31, 78, 73, 130]).

Moreover, the absence of a simple linear structure in $\mathcal{D}$ makes challenging to even properly define the derivative of maps taking value from (or valued in) $\mathcal{D}$, which is the problem at the core of topological optimization, that we focus on in this survey.

Remark 2.23. For the sake of simplicity, and because it matches standard practice, we restrict in this work to persistence diagrams with finitely many points. Most theoretical properties regarding $\mathcal{D}$ and $\mathcal{D}^\circ$ presented in this survey hold when defining persistence diagrams more generally as *locally* finite multisets α supported on Ω satisfying the integrability constraint $\text{Pers}_q(\alpha) := \text{FG}_q(\alpha, \emptyset) < \infty$. Here, $\emptyset$ denotes the empty persistence diagram. This extension can be understood as the completion of $\mathcal{D}^\circ$ for the metric FG_q when $q < +\infty$ (the case $q = +\infty$ is slightly more intricate and is discussed for instance in [15, 21, 101] and [54, §3.3]).

The main motivation to use the partial distance FG_q , $1 \leq q \leq +\infty$ lies in the following “stability theorems”, guaranteeing that close filtrations must induce close PDs.

Theorem 2.24 ([44, 36]). Let K be a finite simplicial complex. Then the map $\text{Filt}_K \rightarrow \mathcal{D}; f \mapsto \text{Dgm}(f)$ is 1-Lipschitz continuous in the following sense:

$$\text{FG}_\infty(\text{Dgm}(f), \text{Dgm}(g)) \leq \|f - g\|_\infty.$$

Similar (yet more intricate) stability results exist for $q < \infty$ [45, 118]. In particular, if X and X' are two point clouds with the same cardinality n , letting $\text{Dgm}_{\text{VR}}(\cdot)$ denote the persistence diagram built on top of a point cloud using the Vietoris–Rips filtration, one can prove that

$$\text{FG}_q(\text{Dgm}_{\text{VR}}(X), \text{Dgm}_{\text{VR}}(X')) \leq C(q, n) \inf_{\phi} \left(\sum_{x \in X} \|x - \phi(x)\|^q \right)^{\frac{1}{q}},$$

where ϕ is a bijection between X and X' , and $C(q, n)$ is a constant depending on q and n .

From an optimization perspective, these stability results are of importance: they often allow one to prove that composite maps of the form $\mathcal{L} = L \circ \text{PH}: \mathbb{R}^{d_1} \rightarrow \mathbb{R}^{d_2}$ (with $\text{PH}: \mathbb{R}^{d_1} \rightarrow \mathcal{D}$ and $L: \mathcal{D} \rightarrow \mathbb{R}^{d_2}$) are Lipschitz, and thus must be differentiable almost everywhere thanks to Rademacher’s theorem, without the need to define a global differential structure in $\mathcal{D}$. This observation will be at the core of Section 3. Before elaborating on this idea, we present how one can compute persistent homology in practice, as the corresponding algorithm is also crucial to understand how composite maps going through $\mathcal{D}$ can be differentiated.

2.3 Persistence computation

Now that we have seen how PDs are defined, we explain in this section how they can be computed practically. Given a filtration $f: K \rightarrow \mathbb{R}$ on a simplicial complex K , the computation of PDs splits into two parts: the first is purely combinatorial, and the second adapts the results to the provided filtration values. Moreover, they both rely on the (filtration-dependent) pre-order⁷ on the simplices of

⁷i.e., one can have $\sigma \neq \tau$ while $\sigma \preceq_f \tau \preceq_f \sigma$.

K defined by $\tau \preceq_f \sigma$ if and only if $f(\tau) \leq f(\sigma)$. This pre-order can be refined into a total order by breaking ties in some fairly arbitrary way, as long as it is consistent with face relations, i.e., as long as $\tau \subseteq \sigma \Rightarrow \tau \preceq_f \sigma$.

First part: combinatorial part (persistence pairing). The first part of PD computations, called the *persistence pairing algorithm*, is about finding the simplices σ_i 's that give rise to the appearance of the cycles z_i 's (which, in turn, create new topological features in the filtration) in the total order induced by f , as well as the simplices σ'_i 's that give rise to the boundaries z'_i 's killing these cycles (i.e., such that $z_i = \partial_p(z'_i)$) according to Theorem 2.17. The pairs (σ_i, σ'_i) are called *persistence pairs*.

Note that while different total orders (associated to the same pre-order) may yield different persistence pairs, they all translate to the same persistence diagram in the second part (see below). The basic algorithm to compute persistence pairs iterates over the ordered set of simplices $\sigma_1 \preceq_f \dots \preceq_f \sigma_{|K|}$ according to Algorithm 1 below—see [16, Section 11.5.2] for a detailed description of the algorithm. In a nutshell, this algorithm relies on the following observation: when inserting a simplex σ of dimension p in the filtration, this either creates a new generator in dimension p (increase the dimension of H_p), or kills one of dimension $p - 1$ (decrease the dimension of H_{p-1}).

Algorithm 1 PersistencePairs(f)

Input: Filtration $f \in \text{Filt}_K$
Order the simplices of K so that $\sigma_1 \preceq_f \dots \preceq_f \sigma_{|K|}$;
 $K_0 \leftarrow \emptyset$;
Pairs₀ = Pairs₁ = $\dots$ = Pairs _{$d-1$} = $\emptyset$;
for $j = 1$ to $|K|$ **do**
 $p \leftarrow \dim \sigma_j$;
 $K_j \leftarrow K_{j-1} \cup \{\sigma_j\}$;
 if σ_j does not create a new homology class in $H_p(K_j)$ **then**
 A homology class in $H_{p-1}(K_{l(j)})$ which was created by $\sigma_{l(j)}$ (for some $l(j) < j$) becomes homologous to 0 as the boundary of a chain created by σ_j ;
 Pairs _{$p-1$} $\leftarrow$ Pairs _{$p-1$} $\cup \{(\sigma_{l(j)}, \sigma_j)\}$;
 end if
end for
Output: Persistence pairs in each dimension Pairs₀, Pairs₁, $\dots$, Pairs _{$d-1$}

Note that for each dimension p , some p -dimensional simplices may remain unpaired at the end of the algorithm; they correspond to the pairs $(b_i, +\infty)$, i.e., correspond to *essential points* in the PD (as opposed to the *ordinary points* with finite coordinates, see Definition 2.19).

Second part: associated filtration values. The persistence diagram $\text{Dgm}(f)$ of the filtration f is then obtained by associating to each persistence pair $(\sigma_{l(j)}, \sigma_j)$ the point a_j in the extended plane whose coordinates are $a_j = (f(\sigma_{l(j)}), f(\sigma_j))$. Moreover, each unpaired simplex σ_j induces an essential point as per $a_j = (f(\sigma_j), +\infty)$. We let P^f and U^f denote the sets of persistence pairs and unpaired simplices respectively, so that the final output of the PD computation can be written as:

$$\text{Dgm}(f) = \{(f(\sigma), f(\sigma'))\}_{(\sigma, \sigma') \in P^f} \cup \{(f(\tau), \infty)\}_{\tau \in U^f}.$$

In practice, the above construction is usually done dimension by dimension (in order to get a single PD for every homological dimension) by restricting the algorithm to the simplices of K of dimension p and $p + 1$. The resulting p -th dimensional PD is denoted by $\text{Dgm}_p(f)$. The pairs $\{(P_p^f, U_p^f)\}_p$ are called barcode templates in [80, Def. 4.3].

Finally, it is also useful to characterize those filtrations that induce the same persistence pairs and unpaired simplices.

Definition 2.25 ([80, Def. 4.2]). Two filtrations $f, g: K \rightarrow \mathbb{R}$ are said to be *ordering equivalent*, written as $f \sim g$, if they induce the same pre-order on the simplices of K . Note that this relation is an equivalence relation on Filt_K .

Since the persistence pairs and the unpaired simplices only depend on the pre-order induced by a filtration (after arbitrarily breaking the ties), we have the following result.

Proposition 2.26. If f and g are ordering equivalent filtrations, then $P_p^f = P_p^g$ and $U_p^f = U_p^g$.

3 Differential framework for persistence diagrams

In applications, filtrations over a (finite) simplicial complex K are often *parametrized* by some θ belonging to a submanifold $M \subseteq \mathbb{R}^d$ through a map $M \ni \theta \mapsto f_\theta \in \text{Filt}_K$. For instance, the height filtration (Example 2.13) over a point cloud $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ is typically parametrized by the direction $\theta \in M = S^{d-1}$, the Čech and Vietoris–Rips filtrations (Examples 2.9 and 2.11) can be considered as being parametrized by the point cloud $X \in M = \mathbb{R}^{n \times d}$ itself, etc. It is natural to wonder whether some parameters are preferable for a given purpose, which is quantified by a task-dependent *loss function* $L: \mathcal{D} \rightarrow \mathbb{R}$ that evaluates the quality of a diagram $\text{Dgm}(f_\theta) \in \mathcal{D}$ for a given parameter $\theta \in M$. Therefore, one seeks to optimize (say, minimize) the composite map $\mathcal{L}: M \ni \theta \mapsto L(\text{Dgm}(f_\theta)) \in \mathbb{R}$, a task that we refer to as *(persistence-based) topological optimization*. All the methods we present in this survey rely on minimizing $\mathcal{L}$ using gradient-based methods. See also Figure 1.1.

Remark 3.1. Provided that $\theta \mapsto f_\theta$ and $\mathcal{D} \ni \alpha \mapsto L(\alpha)$ are both locally Lipschitz—which is the case for all filtrations and loss functions considered in this survey—, Theorem 2.24 along with Rademacher’s theorem ensure that $\mathcal{L}$ is locally Lipschitz hence differentiable almost everywhere, meaning that the gradient $\nabla_\theta \mathcal{L}$ exists generically. However, the (lack of linear) structure of $\mathcal{D}$ (see Remark 2.22) a priori prevents from a straightforward computation of $\nabla_\theta \mathcal{L}$ using the chain rule as differentials of maps from M to $\mathcal{D}$ and from $\mathcal{D}$ to $\mathbb{R}$ have yet to be defined.

3.1 Formulation of differentiability

This subsection summarizes the main results of [80], providing a formal framework to defined differentials (in particular, gradients) of composite maps $\mathcal{L}: M \rightarrow \mathcal{D} \rightarrow \mathbb{R}$ where M and N are two manifolds of class C^∞ and without boundary (in practice, one often considers $M = \mathbb{R}^{d_1}$ and $N = \mathbb{R}^{d_2}$, typically with $d_2 = 1$).

Definition 3.2 ([80, Def. 3.1]). Let $m, n \in \mathbb{N}$. The space of *ordered persistence diagrams* with m ordinary points and n essential points is $\mathbb{R}^{2m} \times \mathbb{R}^n$ equipped with the Euclidean norm. The map $Q_{m,n}: \mathbb{R}^{2m} \times \mathbb{R}^n \rightarrow \mathcal{D}$ quotients the space by the action of the product of the symmetric groups $\mathfrak{S}_m \times \mathfrak{S}_n$ through permutations of points. That is, for any ordered persistence diagram $\tilde{\alpha} = ((b_1, d_1, \dots, b_m, d_m), (v_1, \dots, v_n)) \in \mathbb{R}^{2m} \times \mathbb{R}^n$,

$$Q_{m,n}(\tilde{\alpha}) := \{(b_i, d_i), 1 \leq i \leq m\} \cup \{(v_j, +\infty), 1 \leq j \leq n\} \in \mathcal{D},$$

where points are counted with multiplicity.

Definition 3.3 ([80, Def. 3.3]). Let M be a manifold and $r \in \mathbb{N} \cup \{\infty\}$. A persistence diagram-valued map $\text{PH}: M \rightarrow \mathcal{D}$ is said to be *r -differentiable* at $\theta \in M$ if there exist an open neighborhood U of θ , $m, n \in \mathbb{N}$, and a map $\tilde{B}: U \rightarrow \mathbb{R}^{2m} \times \mathbb{R}^n$ of class C^r such that $\text{PH} = Q_{m,n} \circ \tilde{B}$. One calls $\tilde{B}$ a *local lift* of PH .

Remark 3.4. This definition of r -differentiability, based on *diffeology* [70], implies in particular that $\text{PH}(\theta')$ must have exactly m ordinary points and n essential points for any θ' in the neighborhood U of θ . In particular, since the quotient map $Q_{m,n}$ is (Lipschitz) continuous when $\mathcal{D}$ is equipped with the distance FG_q for $q \in [1, +\infty]$ ([80, Prop. 3.2]), 0-differentiability of a map PH implies its continuity with respect to FG_q . The converse is however false: a map PH could create or destroy a point locally (which would make it not 0-differentiable) while still being continuous for the distance FG_q . For instance, let $M = \mathbb{R}$ and $\text{PH}(\theta) = \{(1 - \theta^2, 1 + \theta^2)\} \in \mathcal{D}$ if $\theta \neq 0$, and $\emptyset$ (the empty diagram) if $\theta = 0$. This map is not 0-differentiable at $\theta = 0$, but it is continuous as $\text{FG}_q(\text{PH}(\theta), \emptyset) = \sqrt{2}\theta^2 \rightarrow 0$ as $\theta \rightarrow 0$.

Enforcing the conservation of the number of points m in Definition 3.3 might seem to be a strong restriction. However, when PH is given by $\text{PH}(\theta) := \text{Dgm}(f_\theta)$ for some (parametrized) filtration $f_\theta \in \text{Filt}_K$ (on some finite simplicial complex K)—a very standard case in practice—, generic configurations (i.e., configurations where f_θ takes distinct values on different simplices) always induce, locally, the same pre-order on K and thus the same number of points in the resulting persistence diagram. It means that, in applications, considering local perturbations of the parameter θ (which is the case when computing gradients) must, generically, let m and n unchanged.

Definition 3.5 ([80, Def. 3.7]). Let M be a smooth manifold and $r \in \mathbb{N} \cup \{\infty\}$. Moreover, let $\text{PH}: M \rightarrow \mathcal{D}$ be a PD-valued map and $\theta \in M$. A C^r *local coordinate system* for PH at θ is a collection of maps $(b_i, d_i: U \rightarrow \mathbb{R})_{i \in I}$ and $v_j: U \rightarrow \mathbb{R})_{j \in J}$ for finite sets I, J defined on an open neighborhood U of θ such that:

- (1) The maps b_i, d_i, v_j are all of class C^r ;
- (2) For any $\theta' \in U$, one has the multi-set equality:

$$\text{PH}(\theta') = \{(b_i(\theta'), d_i(\theta'))\}_{i \in I} \cup \{(v_j(\theta), +\infty)\}_{j \in J}.$$

One simply writes $(U, (b_i, d_i)_{i \in I})$ for a local coordinate system.

Lemma 3.6 ([80, Prop. 3.8]). Let M be a manifold and $\text{PH}: M \rightarrow \mathcal{D}$ be a PD-valued map. Then PH is r -differentiable at $\theta \in M$ if and only if it admits a C^r local coordinate system at θ .

We now discuss the differentiability of maps *defined* on PDs and valued in a manifold N . In most applications, N is typically a Banach, Hilbert or Euclidean space (quite often, simply $\mathbb{R}$ or $\mathbb{R}^d$), and such maps are then referred to as *topological losses*, or *vectorizations* of persistence diagrams, depending on the applications.

Definition 3.7 ([80, Def. 3.10]). Let N be a smooth manifold and $r \in \mathbb{N} \cup \{\infty\}$. A map $L: \mathcal{D} \rightarrow N$ is said to be r -differentiable at $\alpha \in \mathcal{D}$ if for any $m, n \in \mathbb{N}$ and any vector $\tilde{\alpha} \in \mathbb{R}^{2m} \times \mathbb{R}^n$ satisfying $Q_{m,n}(\tilde{\alpha}) = \alpha$, the map $L \circ Q_{m,n}: \mathbb{R}^{2m} \times \mathbb{R}^n \rightarrow N$ is C^r on an open neighborhood of $\tilde{\alpha}$.

The differentials of PD-valued maps and maps defined on PDs can then be defined using lifts.

Definition 3.8 ([80, Def. 3.6 and 3.13]). One has the following differentials:

1. Let $\text{PH}: M \rightarrow \mathcal{D}$ be 1-differentiable at $\theta \in M$ and let $\tilde{B}: U \rightarrow \mathbb{R}^{2m} \times \mathbb{R}^n$ be a C^1 lift of PH defined on an open neighborhood U of θ . The differential $d_{\theta, \tilde{B}}\text{PH}$ of PH at θ with respect to the lift $\tilde{B}$ is defined as the differential of $\tilde{B}$ at θ :

$$d_{\theta, \tilde{B}}\text{PH}: T_\theta M \xrightarrow{d_\theta \tilde{B}} \mathbb{R}^{2m}.$$

2. Let $L: \mathcal{D} \rightarrow N$ be 1-differentiable at $\alpha \in \mathcal{D}$ and $\tilde{\alpha} \in \mathbb{R}^{2m} \times \mathbb{R}^n$ be a pre-image of α under $Q_{m,n}$. The differential of L at α with respect to $\tilde{\alpha}$ is defined as the differential of $L \circ Q_{m,n}$ at $\tilde{\alpha}$:

$$d_{\alpha, \tilde{\alpha}}L: \mathbb{R}^{2m} \times \mathbb{R}^n \xrightarrow{d_{\tilde{\alpha}}(L \circ Q_{m,n})} T_{L(\alpha)}N.$$

While the differentials of PH and L defined above depend respectively on the choice of the lift $\tilde{B}$ and the ordered diagram $\tilde{\alpha}$, the following result established in [80] states that the chain rule used for composing these maps is oblivious to $\tilde{B}$ and $\tilde{\alpha}$.

Proposition 3.9 (Chain rule [80, Prop. 3.14]). Let $\text{PH}: M \rightarrow \mathcal{D}$ be r -differentiable at $\theta \in M$ and $L: \mathcal{D} \rightarrow N$ be r -differentiable at $\text{PH}(\theta)$. Then, one has:

1. $L \circ \text{PH}: M \rightarrow N$ is C^r at θ as a map between smooth manifolds;
2. If $r \geq 1$, for any local C^1 lift $\tilde{B}: U \rightarrow \mathbb{R}^{2m} \times \mathbb{R}^n$ of PH at θ , one has

$$d_\theta(L \circ \text{PH}) = d_{\text{PH}(\theta), \tilde{B}(\theta)} L \circ d_{\theta, \tilde{B}} \text{PH}. \quad (3.1)$$

From the practitioner viewpoint, this result is crucial: it practically allows to compute the gradients of real-valued composite maps going through $\mathcal{D}$ in a *canonical way*. Formally, it means that, in applications, when dealing with a composite map $\mathcal{L}: M \xrightarrow{\text{PH}} \mathcal{D} \xrightarrow{L} N = \mathbb{R}$ where both PH and L are 1-differentiable, it is legitimate to treat the intermediate diagram as an element of the Euclidean space $\mathbb{R}^{2m} \times \mathbb{R}^n$, deriving a standard gradient in $\mathbb{R}^{2m} \times \mathbb{R}^n$ as well, and eventually backpropagate it to obtain a gradient with respect to θ .

Just as in Remark 3.4, we emphasize that r -differentiability (as per Definitions 3.5 and 3.8) of the maps PH and L also implies that of $\mathcal{L} = L \circ \text{PH}$ in the usual sense. However, it is possible to find composite maps $\mathcal{L}$ that would be of class C^r , $r \in \mathbb{N} \cup \{\infty\}$, without PH and/or L being r -differentiable. Nonetheless, it turns out that typical maps PH (induced by filtrations) and L involved in topological optimization are r -differentiable, allowing one to use Equation (3.1) faithfully. The next two sections provide such examples.

3.2 Examples of topological losses

In most applications of topological optimization based on PDs (see Section 5 for examples), the functions to minimize, or (*topological*) *losses*, are instances of r -differentiable maps $L: \mathcal{D} \rightarrow N$ with $N = \mathbb{R}$. These losses are typically used to model some qualitative objective (such as, e.g., increasing or decreasing the sizes of topological features in order to regularize a model with topological priors). In the following, we provide few examples of standard topological losses; following Remark 2.18, these losses only depends on the ordinary part of the persistence diagram. All the results of Subsection 3.1 can be adapted seamlessly simply by dropping the terms depending on n ; we will write Q_m (instead of $Q_{m,0}$) the corresponding quotient map.

Example 3.10 (Total persistence and some variations). The *total persistence* $\text{Pers}(\alpha)$ of a persistence diagram $\alpha \in \mathcal{D}^o$ is the sum of the distances to the diagonal of the (ordinary) points (b, d) of α :

$$\text{Pers}(\alpha) = \frac{1}{2} \sum_{(b,d) \in \alpha} (d - b)^2.$$

For $\alpha \in \mathcal{D}^o$ and an ordered persistence diagram $\tilde{\alpha} \in \mathbb{R}^{2m}$ satisfying $Q_m(\tilde{\alpha}) = \alpha$, we have

$$\text{Pers} \circ Q_m := \mathbb{R}^{2m} \ni (b_1, d_1, \dots, b_m, d_m) \mapsto \frac{1}{2} \sum_{i=1}^m (d_i - b_i)^2 \in \mathbb{R}.$$

Hence, the function $\text{Pers}: \mathcal{D} \rightarrow \mathbb{R}$ is ∞ -differentiable everywhere on $\mathcal{D}$, and its gradient can be identified with that of its lift $(b_1, d_1, \dots, b_m, d_m) \mapsto (b_i - d_i, d_i - b_i)_{i=1}^m$.

The total persistence bears some similarities with the total variation: intuitively, it quantifies the *topological complexity* of input data, by measuring and adding up the sizes of all the topological features. Minimizing this loss can be understood as reducing this complexity, resulting in smoother,

less complex datasets or models. As such, it has been used, e.g., for reducing overfitting in complex predictive models, as it can capture geometric patterns that traditional L^1 or L^2 penalties are oblivious to. See for instance [38, 61] and Subsection 5.2. Practically speaking, minimizing the total persistence attempts at pushing all the persistence diagram points toward the diagonal $\partial\Omega$; its global minimum is 0 and reached if and only if the diagram α is empty.

Naturally, this loss admits several variation, such as (non-exhaustive list): only minimizing the second coordinates, that is, the death times of the topological features (this only modifies the filtration values of the death simplices), using another exponent $p > 1$ on the terms: $|d_i - b_i|^p$, etc.

Instead of minimizing the total persistence, one may consider maximizing it (or minimizing its opposite), in which case one is trying to enhance the topological features appearing in the filtration by pushing persistence diagram points away from the diagonal. This is what happens in Figure 1.1.

A slight yet useful variation of the total persistence loss is the *topological simplification loss*, where one only penalizes points in the diagram that are already close to the diagonal, i.e., $\sum_{i=1}^m (d_i - b_i)1_{|d_i - b_i| < \eta}$ for some fixed threshold $\eta > 0$. At the level of the persistence diagram, this loss will push points that are already close to the diagonal (often considered as “topological noise”) closer to it, while preserving points away from the diagonal (corresponding to “topological signal” or “macroscopic topological features”).

Example 3.11 (Distance to a target diagram). Given a diagram α_0 in $\mathcal{D}^o$, the (squared) *distance to α_0* is a function on $\mathcal{D}^o$ defined as:

$$\text{Dist}_\beta: \mathcal{D}^o \rightarrow \mathbb{R}; \alpha \mapsto \frac{1}{2} \text{FG}_2(\alpha, \beta)^2, \quad (3.2)$$

where FG_2 denotes the 2-th diagram distance as in Definition 2.20. By minimizing this loss, one can enforce α to be equal, or at least close, to a fixed, target persistence diagram β , by moving the persistence diagram points toward the ones of the target β . This objective typically occurs in generative models when the dataset to model can be efficiently characterized by its geometric patterns, as is the case for the distribution of dark matter in large scale simulations of the universe [13, 98, 134], or for the spatial arrangements of single cells in biopsies [7, 8, 129]. Another standard use case is dimension reduction, where one forces the latent spaces learned by neural network architectures to have persistence diagrams as close as possible to the one of the input, high-dimensional data, in order to preserve its geometric patterns and topological features. See [89, 131, 127] and Subsection 5.2.2.

If we consider $\pi^* \in \Gamma(\alpha, \beta)$ the (generically unique⁸) optimal partial matching between two diagrams $\alpha, \beta \in \mathcal{D}^o$, and let $\tilde{\alpha} = (x_1, \dots, x_m) \in \mathbb{R}^{2m}$ denote a lift of α (where $x_i = (b_i, d_i) \in \Omega$), one has $\text{FG}_2(\alpha, \beta)^2 = \sum_{x \in \alpha \cup \partial\Omega} \|x - \pi^*(x)\|^2$. It follows from the envelope theorem that the gradient of Dist_β can be identified with

$$(x_i - \pi^*(x_i))_{i=1}^m, \quad (3.3)$$

Unsurprisingly, following the opposite of this gradient will push $x_i \in \alpha$ toward its target $\pi^*(x_i) \in \beta \cup \partial\Omega$.

Stability of the gradient. Even though the optimal partial matching π^* between two diagrams $\alpha, \beta \in \mathcal{D}^o$ is generically unique, it is an unstable quantity: slightly perturbing the target β (or the current diagram α) may yield a significantly different optimal matching π^* . It means that while Dist is differentiable almost everywhere on $\mathcal{D}^o$, the gradient (3.3) is not a stable quantity, which can be an issue in practical applications (e.g., instability in gradient descents). One way of mitigating this is to add an *entropic regularization term* in (3.2) following the computational optimal transport literature [49, 102]. This approach has been adapted to PDs in [76, 75] and used in the context of topological optimization in [62].

⁸as the solution of a linear programming problem on a convex polytope.

Example 3.12 (Singleton loss). Given a target point $q_0 \in \mathbb{R}^2$, a persistence diagram $\alpha \in \mathcal{D}$, and a point $p_0 \in \alpha$, the *singleton loss* is defined as:

$$\text{Singleton}_{q_0}(\alpha, p_0) = \|p_0 - q_0\|.$$

Intuitively, minimizing this loss amounts to pushing a specific persistence diagram point p_0 in the direction of a specific target point q_0 . This loss can be understood as the simplest version of the distance to a target persistence diagram α_0 , where the target persistence diagram is comprised of only one point $\alpha_0 = \{q_0\}$, and where the matching between α and α_0 is imposed (as one forces $p_0 \in \alpha$ to move towards q_0). This loss is particularly relevant in the context of *big-step* gradient descent, as it allows to drastically speed up topological optimization, see Section 4.3.

Aside from these topological losses directly mapping a diagram in $\mathcal{D}$ to a value in $\mathbb{R}$, another family of topological losses can be obtained by composing two maps $L = \ell \circ \Phi$: a first *vectorization* map $\Phi : \mathcal{D} \rightarrow N$ where N is a Hilbert space or simply the Euclidean space $\mathbb{R}^d$, and a second map $\ell : N \rightarrow \mathbb{R}$ from that linear space to $\mathbb{R}$ which comes from the problem at hand: mean squared error for regression tasks, cross-entropy in classification tasks, etc. The gradient of ℓ is defined in the usual sense, and the gradient of L can then be obtained by composing it with a lift of the differential of the vectorization Φ , defined as in Subsection 3.1.

The vectorization step is a convenient step as it turns the PDs (which live in a non-flat space [124]) into vectors on which standard machine learning pipelines can be used. The most popular vectorizations of PDs are the *linear* ones: they include, e.g., persistence surfaces and their variations [2, 39, 74, 108] and persistence landscapes [20, 37].

Example 3.13 (Linear vectorizations of persistence diagrams). Given a map $\phi : \Omega \rightarrow N$ (N being a Hilbert or Euclidean space), one can define a vectorization $\Phi : \mathcal{D}^\circ \rightarrow N$ as $\Phi : \alpha \mapsto \sum_{x \in \alpha} \phi(x)$. Such vectorizations are called *linear* as, if one encodes a diagram α as a measure $\mu := \sum_x \delta_x$ where δ_x denote the Dirac mass at x in the open half-plane Ω , this map is the linear map $\mu \mapsto \int \phi d\mu$ —see [54, §5.1] for details.

For such vectorizations, assuming that the map ϕ is C^1 , it follows that the differential of Φ at $\alpha \in \mathcal{D}^\circ$ can be identified with

$$\sum_{i=1}^m d\phi((b_i, d_i)),$$

for some lift $(b_i, d_i)_{i=1}^m \in \mathbb{R}^{2m}$ of α .

Note that linear vectorizations were initially introduced as fixed maps that depend on user-defined parameters, that is, each $\phi : \Omega \rightarrow N$ is a parametrized map $\phi = \phi_\eta$, with parameters η controlling the weights to be assigned to each PD point (depending on, e.g., its distance to the diagonal $\partial\Omega$). As tuning η is usually done with cross-validation, which can be expensive in running time, a subsequent body of work has recently emerged about *learning* η : either with deep learning using specialized neural network architectures [67, 66, 30, 138, 72], with codebooks obtained by, e.g., running k -means on the training PDs [111], or with pre-defined template functions fitted on the training set [101].

Remark 3.14 (Kernel methods to vectorize persistence diagrams). Another standard way of deriving vectorizations is with *kernel methods*: in that case, the map Φ is defined implicitly through a *kernel* k representing the scalar product of some (implicit) Hilbert space $\mathcal{H}$: $k(\alpha, \beta) := \langle \Phi(\alpha), \Phi(\beta) \rangle_{\mathcal{H}}$ for any $\alpha, \beta \in \mathcal{D}$. The most popular kernel are the Gaussian-like ones: $k(\alpha, \beta) := \exp(-d(\alpha, \beta)/\sigma)$, $\sigma > 0$, where d is a distance between PDs. Note that not all distances induce valid Gaussian kernels for PDs (i.e., such that there exists an implicit corresponding Hilbert space), only the so-called *conditionally negative semi-definite*⁹ (CNSD) do. While the diagram distances FG_q are unfortunately not CNSD, interpreting PDs as discrete measures as in Example 3.13 allows to borrow tools from other domains

⁹I.e., those distances that satisfy $\sum_i c_i = 0 \Rightarrow \sum_i \sum_j c_i c_j d(\alpha_i, \alpha_j) \leq 0$, for any PDs $\{\alpha_i\}_i$ and coefficients $\{c_i\}_i$.

(such as optimal transport and information geometry) in order to design CNSD metrics: examples include the sliced Wasserstein distance [31] and the Fisher information metric [78]. Moreover, similar to the explicit linear representations, parametrized kernels $k = k_\eta$ can also be learned at training time, using, e.g., metric learning [137].

Remark 3.15 (Automatic differentiation). Since persistence diagrams can be identified with their lifts when it comes to compute gradients, one can in practice resorts on automatic differentiation to compute gradients of maps defined from $\mathcal{D}$ to $\mathbb{R}$, simply by actually defining them as maps from $\mathbb{R}^{2m}$ to $\mathbb{R}$ (note that m should be allows to vary in general) in a framework compatible with automatic differentiation (e.g., PyTorch [100], TensorFlow [1], JAX [18], etc.).

3.3 Persistence diagrams of parametrized families of filtrations

In addition to topological losses, most topological optimization problems involve some r -differentiable PD-valued maps PH (for a given homology dimension) defined on a parameter manifold M (see beginning of Section 3). As θ often parametrizes filtration values assigned to simplicial complexes, the goal of this section is to present in more details the smoothness associated to such filtration maps.

Definition 3.16. Let K be a finite simplicial complex and M be a smooth manifold. A map $F: M \rightarrow \mathbb{R}^{|K|}$ is said to be a *parametrized family of filtrations* if for any $\theta \in M$ and $\sigma, \sigma' \in K$ with $\sigma \subseteq \sigma'$, one has $[F(\theta)]_\sigma \leq [F(\theta)]_{\sigma'}$, where $[v]_\sigma$ denotes the entry of a vector $v \in \mathbb{R}^{|K|}$ at the position of σ (upon using a fixed, arbitrary ordering of the simplices of K). In other words, a parametrized family of filtrations is a map $F: M \rightarrow \text{Filt}_K$ (after identifying $\mathbb{R}^K$ and $\mathbb{R}^{|K|}$).

Note that a parametrized family of filtrations is called just a parametrization in [80]. Now, let M be a smooth manifold, and $F: M \rightarrow \text{Filt}_K$ be a parametrized family of filtrations of class C^r . In what follows, we consider a PD-valued map of the form:

$$\mathcal{P}_p: M \xrightarrow{F} \text{Filt}_K \xrightarrow{\text{PH}_p} \mathcal{D}. \quad (3.4)$$

Note that from now on, the domain of PH is Filt_K (and not M directly, as in the previous sections). We shall show that $\mathcal{P}_p$ is r -differentiable on a generic (open dense) subset of M . First, we state a local differentiability result.

Theorem 3.17 ([80, Thm. 4.7]). Let $\theta \in M$. Suppose $F: M \rightarrow \text{Filt}_K$ is of class C^r on some open neighborhood of θ , and that $F(\theta) \sim F(\theta')$ (as per Definition 2.25) for all $\theta' \in U$. Then $\mathcal{P}_p$ is r -differentiable at θ .

Proof. By Proposition 2.26, the persistence pairs P_p and the unpaired simplices U_p for $F(\theta)$ remain unchanged for all (ordering equivalent) $F(\theta'), \theta' \in U$. Hence, within U , one can write:

$$\mathcal{P}_p(\theta') = \{([F(\theta')]_\sigma, [F(\theta')]_{\sigma'})\}_{(\sigma, \sigma') \in P_p} \cup \{([F(\theta')]_\tau, \infty)\}_{\tau \in U_p}$$

for any $\theta' \in U$. This gives a C^r local coordinate system for $\mathcal{P}_p$ at x . $\square$

Now, we state a global generic differentiability result. Set

$$\tilde{M} := \left\{ \theta \in M \mid \begin{array}{l} \exists \text{ open neighborhood } U_\theta \text{ of } \theta \text{ such that} \\ F(\theta') \sim F(\theta) \text{ for all } \theta' \in U_\theta \end{array} \right\}.$$

Then one can prove that $\tilde{M}$ is generic (open dense) in M (see [80, Lem. 4.10]).

Theorem 3.18 ([80, Thm. 4.9]). Suppose $F: M \rightarrow \text{Filt}_K$ is of class C^r on some open subset U of M . Then $\mathcal{P}_p$ is r -differentiable on $U \cap \tilde{M}$.

Proposition 3.19 ([80, Prop. 4.14]). Let $\theta \in U \cap \tilde{M}$. Let $(\sigma_1, \sigma'_1), \dots, (\sigma_m, \sigma'_m)$ be the persistence pairs and $\tau_1, \dots, \tau_n$ be the unpaired simplices of $F(\theta)$. Then, the map:

$$\tilde{\mathcal{P}}_p: \theta' \mapsto ([F(\theta')]_{\sigma_i}, [F(\theta')]_{\sigma'_i}_{i=1}^m, ([F(\theta')]_{\tau_j})_{j=1}^n)$$

is a local C^r lift of $\mathcal{P}_p$ at θ , and the corresponding differential is:

$$d_{\theta, \tilde{\mathcal{P}}_p} \mathcal{P}_p(\cdot) = ([d_\theta F(\cdot)]_{\sigma_i}, [d_\theta F(\cdot)]_{\sigma'_i}_{i=1}^m, ([d_\theta F(\cdot)]_{\tau_j})_{j=1}^n).$$

Intuitively, the differential of $\mathcal{P}_p$ is simply obtained by computing the differential of the filtration map first, and then picking the entries of the paired and unpaired simplices. An important property of the space of filtrations Filt_K is that it admits a *Whitney stratification*.

Definition 3.20. Let M be a subset of $\mathbb{R}^d$. A *Whitney stratification* $\mathcal{S} = \{M_i\}_{i \in I}$ of M is a locally finite partition by connected (not necessarily closed) smooth submanifolds M_i , called *strata*, satisfying the following conditions:

- (Frontier) For each stratum $M_i \in \mathcal{S}$, the set $\overline{M_i} \setminus M_i$ is a union of strata.
- (Condition B) Let $M', M'' \in \mathcal{S}$ be two strata, and $\theta \in M'$. If two sequences $\{\theta'_k\}_k \subset M'$ and $\{\theta''_k\}_k \subset M''$ both converge to θ , if the line connecting θ'_k and θ''_k converges to some line l , and if $T_{\theta''_k} M''$ converges to some plane T , then T contains l .

A stratum with the maximal dimension is called a *top-dimensional stratum*. Eventually, given $\theta \in M$, a stratum $M_i \in \mathcal{S}$ is said to be *incident* to θ if θ belongs to its closure: $\theta \in \overline{M_i}$.

Let $\Omega(\text{Filt}_K)$ be the set of equivalence classes with respect to the ordering equivalence $\sim$. Then, $\Omega(\text{Filt}_K)$ is a Whitney stratification of Filt_K by semi-algebraic subsets, whose stratum are separated by a family $\mathcal{F}$ of hyperplanes, defined with $\mathcal{F} := \{v \in \mathbb{R}^{|K|} \mid [v]_\sigma = [v]_{\sigma'}\}_{\sigma \neq \sigma' \in K}$. Thanks to this fact, one can extend the local lift of $\mathcal{P}_p$ in Proposition 3.19 to a global lift.

Proposition 3.21 ([80, Prop. 4.23]). Let K be a simplicial complex of dimension d . For $0 \leq p \leq d$, there exist integers m_p, n_p such that $\sum_{p=0}^d (2m_p + n_p) = |K|$ and a map $\text{Perm}: \text{Filt}_K \rightarrow \prod_{p=0}^d \mathbb{R}^{2m_p} \times \mathbb{R}^{n_p} \times \mathbb{R}^{n_p} \cong \mathbb{R}^{|K|}$ satisfying the following:

1. the restriction $\text{Perm}|_S$ to each stratum S of $\Omega(\text{Filt}_K)$ is a permutation map,
2. the following diagram commutes

$$\begin{array}{ccc} \text{Filt}_K & \xrightarrow{\text{Perm}} & \prod_{p=0}^d \mathbb{R}^{2m_p} \times \mathbb{R}^{n_p} \\ & \searrow \text{PH} & \downarrow \prod_{p=0}^d Q_{m_p, n_p} \\ & & \mathcal{D}^{d+1}, \end{array}$$

where $\mathcal{D}^{d+1}$ denotes the $(d+1)$ -th Cartesian power of $\mathcal{D}$.

Consider the parametrized PD-valued map:

$$\mathcal{P}: M \xrightarrow{F} \text{Filt}_K \xrightarrow{\text{PH}} \mathcal{D}^{d+1}; \theta \mapsto F(\theta) \mapsto (\text{Dgm}_p(F(\theta)))_{p=0}^d.$$

Corollary 3.22 ([80, Cor. 4.24]). The map:

$$\tilde{\mathcal{P}}: M \rightarrow \prod_{p=0}^d \mathbb{R}^{2m_p} \times \mathbb{R}^{n_p}; \quad \theta \mapsto \text{Perm}(F(\theta))$$

is a global lift of $\mathcal{P}$, i.e., $Q \circ \tilde{\mathcal{P}} = \mathcal{P}$ on M , where $Q = \prod_{p=0}^d Q_{m_p, n_p}$.

In words, Corollary 3.22 ensures that global lifts for PD-valued maps associated to parametrized families of filtrations can be obtained by computing all the permutations (i.e., the indices of the paired and unpaired simplices) of the different stratum of $\Omega(\text{Filt}_K)$. This lift is global, as, for a given parameter θ , it suffices to identify the strata to which $F(\theta)$ belongs to, and then to apply the corresponding permutation of the entries of $\mathbb{R}^{|K|}$ to obtain the persistence diagram. This observation is key in the implementation of several gradient descent schemes for topological optimization (see for instance “Sampling strata” paragraph in Section 4.2).

3.4 Stratified filtrations and directional differentiability

In the previous section, we have seen that global lifts of PD-valued maps can be computed through a stratification of the filtration space Filt_K . In this section, we now study the case where the domain M of the filtration map $F: M \rightarrow \text{Filt}_K$ itself can be stratified, as this is the case for most common filtrations in TDA.

Definition 3.23. Let M and N be manifolds endowed with stratifications $\mathcal{S}_M$ and $\mathcal{S}_N$, respectively (see Definition 3.20). A map $f: M \rightarrow N$ is said to be *weakly stratified* if for any $N_j \in \mathcal{S}_N$ the preimage $f^{-1}(N_j)$ is a union of strata in $\mathcal{S}_M$.

A function $f: M \rightarrow \mathbb{R}$ is said to be *Whitney stratifiable* if its graph admits a Whitney stratification.

Proposition 3.24 ([80, Prop. 4.16]). Let $F: M \rightarrow \text{Filt}_K$ be a continuous parameterized family of filtrations. Suppose that M is a semi-algebraic (resp. compact subanalytic) set in $\mathbb{R}^d$ and F is a semi-algebraic (resp. subanalytic) map. Then, there is a stratification of M by semi-algebraic (resp. subanalytic) sets such that the restriction of F to each stratum is C^∞ .

Theorem 3.25 ([80, Thm. 4.19]). Let M be a manifold endowed with a Whitney stratification $\mathcal{S}$. Let $F: M \rightarrow \text{Filt}_K$ be a continuous parameterized family of filtrations such that:

1. F is a weakly stratified map with respect to $\mathcal{S}$ and $\Omega(\text{Filt}_K)$,
2. the restriction of F to each stratum in $\mathcal{S}$ is C^r , and
3. for any $\theta \in M$ and any stratum M_i incident to θ , there is an open neighborhood U of θ such that $F|_{M_i \cap U}$ extends to a C^r map $U \rightarrow \mathbb{R}^K$.

Then, at any $\theta \in M$, the PD-valued map $\mathcal{P}_p: M \rightarrow \mathcal{D}$ (defined in Equation (3.4)) is r -differentiable along each stratum that is incident to θ .

Corollary 3.26 ([80, Cor. 4.20]). Under the assumptions of Proposition 3.24, there is a Whitney stratification of M by semi-algebraic (resp. subanalytic) subsets, such that $\mathcal{P}_p$ is ∞ -differentiable on the top-dimensional strata (see Definition 3.20). If furthermore F is C^r , then $\mathcal{P}_p$ is everywhere r -differentiable along incident strata.

Combining everything, one can finally obtain the differentiability properties of the composite map $\mathcal{L} = L \circ \mathcal{P}_p$ in the case of parametrized families of filtrations.

We eventually end this section with an additional notion of regularity on the filtration with respect to the stratification, namely stating the existence of a C^2 extension. It will be used to define the gradient sampling approach of [79] presented in Section 4.

Definition 3.27. A map $\mathcal{L}: M \rightarrow \mathbb{R}$ is said to be *stratifiably smooth* if there exists a Whitney stratification $\mathcal{S}$ of M such that for each top-dimensional stratum $M_i \in \mathcal{S}$, the restriction $\mathcal{L}|_{M_i}$ admits an extension $\mathcal{L}_i$ of class C^2 in a neighborhood of M_i .

Proposition 3.28. Let $L: \mathcal{D} \rightarrow \mathbb{R}$ be a 2-differentiable map, and $F: M \rightarrow \text{Filt}_K$ be a parameterized family of filtrations satisfying the conditions of Theorem 3.25 with $r = 2$. Then, the composite function $\mathcal{L} = L \circ \text{PH}_p \circ F$ is stratifiably smooth.

3.5 Examples of stratified filtrations and their differentials

As in Section 3.2, we now review the differentiability properties of the common filtrations in TDA presented in Section 2.1, and we explicit their differentials.

Example 3.29 (Vietoris–Rips filtration, Example 2.11). Let Δ_n is the complete simplicial complex on n vertices, i.e., that comprises all the faces of the $(n - 1)$ -dimensional simplex. The family of Vietoris–Rips filtrations on (ordered) point clouds of n points $(x_1, \dots, x_n) \in (\mathbb{R}^d)^n =: M$ is the semi-algebraic parametrized family of filtrations:

$$F: M \rightarrow \mathbb{R}^{|\Delta_n|} = \mathbb{R}^{2^n - 1},$$

defined, for any ordered point cloud $X = (x_1, \dots, x_n) \in M$ and any simplex $\sigma \subseteq \{1, \dots, n\}$, by:

$$[F(X)]_\sigma = \max_{i,j \in \sigma} \frac{1}{2} \|x_i - x_j\|.$$

One can easily check that the permutation induced by PH is constant on the strata whose boundaries are the subspaces $S_{i,j,k,l} = \{(x_1, \dots, x_n) \in \mathbb{R}^d \mid \|x_i - x_j\| = \|x_k - x_l\|\}$ over all the 4-tuples (i, j, k, l) such that at least three of the four indices i, j, k, l are distinct. Then, for a specific simplex σ such that $[F(X)]_\sigma = \max_{i,j \in \sigma} \frac{1}{2} \|x_i - x_j\| = \|x_{i^*} - x_{j^*}\|$, one can check that the differential of F is equal to $[d_X F(\cdot)]_\sigma = \langle \nabla_X [F(X)]_\sigma, \cdot \rangle$ with:

$$\nabla_X [F(X)]_\sigma = [\mathbf{0}, \dots, \mathbf{0}, \underbrace{\frac{x_{i^*} - x_{j^*}}{\|x_{i^*} - x_{j^*}\|}}_{\text{index } i^*}, \mathbf{0}, \dots, \mathbf{0}, \underbrace{\frac{x_{j^*} - x_{i^*}}{\|x_{i^*} - x_{j^*}\|}}_{\text{index } j^*}, \mathbf{0}, \dots, \mathbf{0}] \in (\mathbb{R}^d)^n.$$

Finally, within a given stratum, computing the differential of PH simply amounts to permuting the entries of $d_X F(\cdot)$ according to the paired and unpaired simplices of the stratum.

This example naturally extends to general Vietoris–Rips filtrations for metric spaces in the following way. Let $M \subset \mathcal{M}_n(\mathbb{R})$ be the set of $n \times n$ symmetric matrices with non-negative entries and 0 on the diagonal. This is a semi-algebraic subset of the space of n -by- n matrices $\mathcal{M}_n(\mathbb{R}) \simeq \mathbb{R}^{n^2}$, of dimension $m = (n - 1)(n - 2)/2$. The map $F: M \rightarrow \mathbb{R}^{|\Delta_n|} = \mathbb{R}^{2^n - 1}$ defined by $[F(A)]_\sigma = \max_{i,j \in \sigma} a_{i,j}$ for any $A = (a_{i,j})_{1 \leq i,j \leq n} \in M$, is a semi-algebraic family of filtrations. Moreover, the set of strata can be chosen to be the set of matrices with at least two equal entries.

Remark 3.30. The differentiability of the Vietoris–Rips filtrations was first considered in [59], which raised the first problem in the TDA literature that involved differentiating persistence diagrams. The problem is as follows: given an ordered point cloud $X = (x_1, \dots, x_n) \in (\mathbb{R}^d)^n$ and target persistence diagram α_0 , move X continuously in order to make the persistence diagram $\text{Dgm}_{\text{VR}}(X)$ (computed with the Vietoris–Rips filtration, with X regarded as a usual point cloud) closer to α^* . The authors proposed the so-called *continuation method* to solve this problem, which is based on the Newton–Raphson method.

We modify their approach and propose to (try to) build an ordered point cloud X with a prescribed diagram α^* , with the Jacobian $D_X \text{PH}$ of the map $\text{PH}: (\mathbb{R}^d)^n \ni X \mapsto \text{Dgm}_{\text{VR}}(X) \in \mathcal{D}$, in the following way. Denoting by X_0 the initial point cloud, we repeatedly update an ordered point cloud by $X_k + \gamma_k (D_X \text{PH}(X_k))^\dagger v_k$ for some vector v_k in each step k , where γ_k is a step size. Here v_k is defined with an optimal partial matching between $\text{PH}(X_k)$ and α^* with respect to FG_2 . Unlike the original approach in [59], we recompute the partial matching in each step, which enables to adaptively obtain the direction to update the point cloud. The detail of our implementation is presented in Algorithm 2.

Algorithm 2 Continuation of Point Clouds via Persistence Diagrams [59]

- 1: **Input:** Initial ordered point cloud X_0 , target diagram α^* , order of diagram distance q , number of steps N , a sequence of step sizes $\{\gamma_k\}_{k=0}^{N-1}$
- 2: **for** $k = 0$ to $N - 1$ **do**
- 3: Set $\alpha_k \leftarrow \text{PH}(X_k)$; # Recall $\text{PH}(\cdot) = \text{Dgm}_{\text{VR}}(\cdot)$ here
- 4: Take π_k be an optimal partial matching between α_k and α^* with respect to FG_q (see Definition 2.20);
- 5: Take an arbitrary ordering $(y[1], \dots, y[m])$ of the points in α_k and regard it as a vector;
- 6: Define a vector v_k by setting the i -th element as

$$v_k[i] \leftarrow \pi_k(y[i]) - y[i] \in \mathbb{R}^2;$$

- 7: Compute Jacobian $J_k \leftarrow \text{DPH}(X_k)$;
- 8: Update an ordered point cloud using the pseudo-inverse $J_k^\dagger$ by

$$X_{k+1} \leftarrow X_k + \gamma_k J_k^\dagger v_k;$$

- 9: **end for**
 - 10: **return** X_N
-

Example 3.31 (Weighted Rips filtration). Weighted Rips filtrations are a generalization of Vietoris–Rips filtrations where weights are assigned to the vertices of the complete simplicial complex Δ_n . Given a function $f: \mathbb{R}^d \rightarrow \mathbb{R}$, the family of weighted Rips filtrations $F: M = (\mathbb{R}^d)^n \rightarrow \mathbb{R}^{|\Delta_n|} = \mathbb{R}^{2^n - 1}$ associated with f is defined, for any $X = \{x_1, \dots, x_n\} \in M$ and any simplex $\sigma \subseteq \{1, \dots, n\}$, by:

$$[F(X)]_\sigma = \begin{cases} 2f(x_j) & (\sigma = \{j\}); \\ \max\{2f(x_i), 2f(x_j), \|x_i - x_j\| + f(x_i) + f(x_j)\}, & (\sigma = \{i, j\}, i \neq j); \\ \max\{[F(X)]_{\{i, j\}} \mid i, j \in \sigma\} & (|\sigma| \geq 3). \end{cases} \quad (3.5)$$

Since Euclidean distances and max function are semi-algebraic, this family of filtrations is semi-algebraic as soon as the weight function f is semi-algebraic. Moreover, the differential of F can be easily computed in a way that is similar to the Vietoris–Rips filtrations (one just needs to distinguish among the three cases in Equation (3.5), and to add the differentials $d_{x_i}f(\cdot)$, $d_{x_j}f(\cdot)$ of f when needed). The differential of PH then follows by applying the stratum-specific permutation.

This example easily extends to the case where the weight function depends on the point cloud $X = \{x_1, \dots, x_n\}$, i.e., when the weight at vertex y is defined by $f(x, y)$ with $f: (\mathbb{R}^d)^n \times \mathbb{R}^d \rightarrow \mathbb{R}$. A particular example of such a family is given by the so-called DTM filtration [4], where $f(x, y)$ is the average distance from y to its k -nearest neighbors in X . In this case, f is semi-algebraic, and the family of DTM filtrations is semi-algebraic.

Example 3.32 (Sublevel sets filtrations, Examples 2.13 to 2.15). Let K be a simplicial complex with n vertices $v_1, \dots, v_n$. Any real-valued function f defined on the vertices of K can be represented as a vector $[f(v_1), \dots, f(v_n)]$ in $\mathbb{R}^n$. The family of sublevel sets filtrations $F: M = \mathbb{R}^n \rightarrow \text{Filt}_K$ of functions on the vertices of K is defined by $[F(f)]_\sigma = \max_{i \in \sigma} f_i$ for any $f = [f_1, \dots, f_n] \in M$ and any simplex $\sigma \subseteq \{1, \dots, n\}$. This filtration is also known as the *lower-star filtration* of f , and is a very general way of designing filtrations: it includes, for instance, the height filtration (Example 2.13), filtrations on graphs (using graph nodes as vertices, Example 2.14) and filtrations on images (using pixel corners as vertices, Example 2.15). The function F is obviously semi-algebraic, and the set of strata S can be chosen as $S = \bigcup_{1 \leq i < j \leq n} \{[f_1, \dots, f_n] \in M \mid f_i = f_j\}$. Moreover, for a specific simplex σ such that $[F(f)]_\sigma = \max_{i \in \sigma} f_i = f_{i^*}$, the differential of F is simply obtained as $[d_f F(\cdot)]_\sigma = [\cdot]_{i^*}$, i.e., picking

the i^* -th entry of the vector. The differential of PH then follows by applying the stratum-specific permutation.

Note however that in many cases, the function f itself depends on some parameters $f = f(\theta)$, and the differential of f with respect to its own parameters has to be incorporated in the chain rule. This happens, e.g., when graph filtrations are learned by a graph neural network, as in [68], or when image filtrations are learned by a convolutional neural network, as in [10].

4 Optimizing persistence-based objective functions

In Section 3, we presented the differentiability properties of the persistence maps PH and filtration maps F . Now, in this section, our aim is to present the different optimization schemes that have been proposed in the literature, as well as their theoretical guarantees, for minimizing composite topological losses $\mathcal{L}$ of the form $\mathcal{L} := L \circ \text{PH} \circ F: M \rightarrow \mathbb{R}$, defined on a parameter manifold M . As one typically performs topological optimization while targeting downstream machine learning tasks, and due to the prominence of gradient-based optimization through *automatic differentiation* in deep learning, it is not surprising that most if not all techniques developed in the TDA literature consist on performing schemes akin to gradient descent.

Below, we detail the most standard stochastic gradient descent scheme [29, 51] in Section 4.1, that we call *vanilla* gradient descent, as well as its convergence properties. Then, we explain two other schemes that both aim at improving it: the *stratified* gradient descent [79] with its improved convergence properties in Section 4.2, and the *big-step* gradient descent [91] with its faster empirical convergence in Section 4.3. Finally, we describe two extensions that can both be applied to any of the three previous schemes: *downsampling* [119, 131], and *diffeomorphic interpolations* [34], in Section 4.4. See Figure 4.1 for a schematic overview of these methods.

4.1 Vanilla gradient descent

In this section, we present the most natural way to minimize a topological loss based on PDs. Let K be a simplicial complex, M be an open subset of $\mathbb{R}^d$, and $F: M \rightarrow \text{Filt}_K$ be a parametrized family of filtrations of class C^∞ . Let $L: \mathcal{D} \rightarrow \mathbb{R}$ be a 2-differentiable map, and define a loss function $\mathcal{L}$ with $\mathcal{L} := L \circ \text{PH} \circ F: M \rightarrow \mathbb{R}$. As explained in Section 3 (see, e.g., Remark 3.1 and Proposition 3.28), the loss $\mathcal{L}$ is often smooth *almost everywhere* on M , so one can still define its *Clarke subgradient*:

Definition 4.1. The *Clarke subgradient* of $\mathcal{L}$ at $\theta \in M$ is:

$$\partial\mathcal{L}(\theta) := \text{Conv} \left\{ \lim_{\theta_i \rightarrow \theta} \nabla\mathcal{L}(\theta_i) \mid \mathcal{L} \text{ is differentiable at } \theta_i \right\}.$$

In the following, we will refer to any element $g_\theta \in \partial\mathcal{L}(\theta)$ as a *vanilla gradient* for $\mathcal{L}$ at θ (as opposed to the other gradient alternatives presented in Sections 4.2 and 4.3). Stochastic (sub)gradient descent is then performed by progressively updating an estimation θ_k of a minimizer θ^* of $\mathcal{L}$ on U with:

$$\theta_{k+1} = \theta_k - \gamma_k(g_{\theta_k} + \zeta_k), \quad g_{\theta_k} \in \partial\mathcal{L}(\theta_k), \quad (4.1)$$

where the sequence $\{\gamma_k\}_k$ is the *learning rate* of the process, and $\{\zeta_k\}_k$ is a sequence of random variables used for robustifying the estimations; e.g. $\zeta_k \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, I)$.

Implementation. In practice, it can be computed simply by composing the differential of L with the differential of the persistence map—which amounts to finding the permutation associated to the current persistence pairs and unpaired simplices computed with Algorithm 1,¹⁰ as per Proposition 3.19—

¹⁰Most TDA libraries include such functions, see for instance the `persistence_pairs()` method in the Gudhi library (<https://gudhi.inria.fr/python/latest/>).

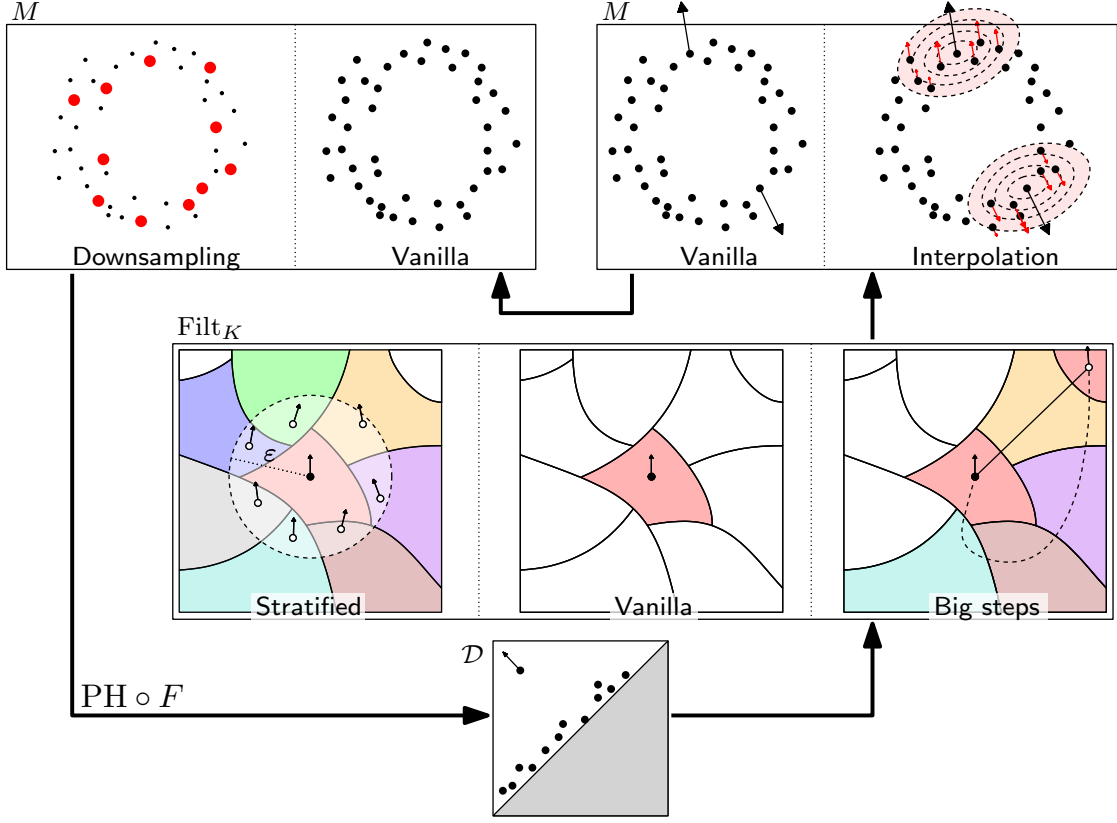


Figure 4.1: Schematic illustration of the different gradient schemes presented in this survey (for point cloud filtrations). The space of filtrations Filt_K is displayed in two dimensions with strata separated with solid lines. While stratified gradient descent aggregates the gradients associated to strata intersecting an open ball around the current estimate, big step gradient descent proposes a way to directly jump to the next iterate without going through other, intermediate strata (as vanilla gradient descent—the dashed line—would). Two possible extensions are also displayed: downsampling, which uses a smaller dataset in order to save computation time for computing persistence diagrams in the forward pass, and diffeomorphic interpolation, which extends the gradients computed in the backward pass to a vector field (defined everywhere in the parameter space) using kernels.

and the differential of the filtration map (see, e.g., Section 3.5 for examples of such differentials). See Algorithm 3.

Usually, the gradients $\nabla_{\theta}[F(\theta)]_{\sigma}$ and the partial derivatives $\frac{\partial L}{\partial [\alpha]_i}$ can be either computed explicitly as they are easy to derive from the usual losses and filtrations in the TDA literature (see Sections 3.2 and 3.5), or they can be automatically computed with backpropagation using standard deep learning libraries such as `PyTorch`, `TensorFlow` or `jax`, as long as the filtration map F and the loss L are implemented with operations from these libraries.

Convergence guarantees. It has been shown in [51] that under mild technical conditions on the sequences $\{\alpha_k\}_k$ and $\{\zeta_k\}_k$, stochastic subgradient descent converges almost surely to a critical point of $\mathcal{L}$ as soon as $\mathcal{L}$ is locally Lipschitz. More precisely, consider the following standard assumptions (see [51, Assumption C]):

- (1) for any k , $\alpha_k \geq 0$, $\sum_{k=1}^{\infty} \alpha_k = +\infty$, and $\sum_{k=1}^{\infty} \alpha_k^2 < +\infty$;

Algorithm 3 VanillaGradient(θ)

Input: Current iterate $\theta \in M$

First step is to identify the critical simplices associated to the persistence map in the current filtration

$((\sigma_1, \sigma_2), \dots, (\sigma_{2m+1}, \sigma_{2m})), (\tau_1, \dots, \tau_n) \leftarrow \text{PersistencePairs}(F(\theta));$

Use these critical simplices as a local lift and compute the differential as per Proposition 3.19

Recall that $[v]_\sigma$ denotes the entry of v at the position of σ in an arbitrary simplex ordering

$\alpha \leftarrow [[F(\theta)]_{\sigma_1}, \dots, [F(\theta)]_{\sigma_{2m}}, [F(\theta)]_{\tau_1}, \dots, [F(\theta)]_{\tau_n}] \in \mathbb{R}^{2m+n};$

$\nabla_\theta[\alpha]_i \leftarrow \nabla_\theta[F(\theta)]_{\sigma_i}, \forall 1 \leq i \leq 2m;$

$\nabla_\theta[\alpha]_{2m+j} \leftarrow \nabla_\theta[F(\theta)]_{\tau_j}, \forall 1 \leq j \leq n;$

Finally, compute the vanilla gradient with the chain rule as per Proposition 3.9

$g_\theta \leftarrow \sum_{i=1}^{2m+n} \frac{\partial L}{\partial [\alpha]_i} ([\alpha]_i) \cdot \nabla_\theta[\alpha]_i;$

return g_θ

(2) $\sup_k \|\theta_k\| < +\infty$, almost surely;

(3) denoting by $\mathcal{F}_k$ the increasing sequence of σ -algebras $\mathcal{F}_k = \sigma(\{\theta_j, g_{\theta_j}, \zeta_j \mid j < k\})$, there exists a function $p: \mathbb{R}^d \rightarrow \mathbb{R}$ which is bounded on bounded sets such that almost surely, for any k ,

$$\mathbb{E}[\zeta_k | \mathcal{F}_k] = 0 \quad \text{and} \quad \mathbb{E}[\|\zeta_k\|^2 | \mathcal{F}_k] < p(\theta_k).$$

Assumption (1) is easily satisfied, for example, by taking $\alpha_k = 1/k$. Assumption (2) is usually easy to check for most of the functions $\mathcal{L}$ encountered in practice (see Section 3.2). Assumption (3) is also a standard condition, which states that, conditioned upon the past, the variables ζ_k have zero mean and controlled moments; e.g., this can be achieved by taking a sequence of independent and centered variables with bounded variance that are also independent of the θ_k 's and g_{θ_k} 's.

Under these assumptions, the following result is an immediate consequence of Corollary 5.9 in [51].

Theorem 4.2. Let K be a simplicial complex, $M \subseteq \mathbb{R}^d$ be an open subset, and $F: M \rightarrow \text{Filt}_K$ be a Whitney stratifiable parametrized family of filtrations of K . Let $L: \mathcal{D} \rightarrow \mathbb{R}$ be a function such that $L \circ Q_{m_p, n_p}$ is Whitney stratifiable (e.g., L is 2-differentiable in the sense of Definition 3.7) and $\mathcal{L} := L \circ \text{PH} \circ F$ is locally Lipschitz. Then, under Assumptions (1), (2) and (3) above, almost surely the limit points of the sequence $\{\theta_k\}_k$ obtained from the iterations of Equation (4.1) are critical points of $\mathcal{L}$ and the sequence $\{\mathcal{L}(\theta_k)\}_k$ converges.

While easy to define and implement, the vanilla gradient also suffers from a few weaknesses. First, on the theoretical side, while Theorem 4.2 ensures that stochastic subgradient descent converges, nothing is said about the quality of the critical point it converges to, nor about the convergence rate, i.e., the descent speed. Second, on the practical side, it is easy to see that updating points in a PD (by minimizing some loss) only influences the persistence pairs and unpaired simplices associated to them; this in turn means that, for a given PD point, only at most two filtration values are updated accordingly, leading to sparse gradients and slow, erratic convergence. In the following sections, we will explore two gradient alternatives and two gradient extensions that were designed to deal with these issues.

4.2 Stratified gradient descent

A recent alternative to vanilla gradients was proposed in [79]. The main idea is to make use of the stratifications of Filt_K induced by the persistence and filtration maps in order to design a more efficient gradient using a procedure similar to the well-known *gradient sampling* method. Recall that gradient sampling at a point x involves sampling points in an ε -neighborhood of x , and taking the vector with smallest norm in the convex hull of the corresponding gradients as a new, smoother gradient (after

renormalization). The key idea of this section is to incorporate the information encoded in the strata of the persistence and filtration maps within this procedure. This in turn allows to guarantee that the limit points of the sequence of iterates $\{\theta_k\}_k$ is close to an approximate critical point, as characterized with *Goldstein subgradients*. Let $\mathcal{L}: M \rightarrow \mathbb{R}$ be a stratifiably smooth function (see Definition 3.27), with $M \subset \mathbb{R}^d$. In this section, we also consider the following assumptions on $\mathcal{L}$:

- (A1) $\mathcal{L}: M \rightarrow \mathbb{R}$ has bounded sublevel sets.
- (A2) For any $\theta \in M$, we have an oracle checking whether $\mathcal{L}$ is differentiable at θ .
- (A3) For each $\theta \in M$ and ε -close top-dimensional stratum M_i , we have an oracle that returns an element $\theta_i \in M_i$ with $\|\theta - \theta_i\| \leq \varepsilon$.

Definition 4.3. Let $\varepsilon > 0$. The *Goldstein subgradient* of $\mathcal{L}$ is

$$\partial_\varepsilon \mathcal{L}(\theta) := \text{Conv} \left\{ \lim_{\theta_i \rightarrow \theta'} \nabla \mathcal{L}(\theta_i) \mid \|\theta - \theta'\| \leq \varepsilon, \mathcal{L} \text{ is differentiable at } \theta_i \right\}.$$

Here, Conv denote the convex hull of a set. A point θ is said to be ε -stationary if $0 \in \partial_\varepsilon \mathcal{L}(\theta)$ and it is (ε, η) -stationary, for some $\eta \geq 0$, if $d(0, \partial_\varepsilon \mathcal{L}(\theta)) \leq \eta$.

The set of Goldstein subgradients at θ can be thought of as “average” (convex combinations) of nearby gradients, and is well defined even if $\mathcal{L}$ is not differentiable at θ . Note that local minimum (as long as local maximum and usual saddle points) satisfy $0 \in \partial_{\varepsilon=0} \mathcal{L}(\theta)$.

The whole point of stratified gradient descent is to use an approximation $\tilde{\partial}_\varepsilon \mathcal{L}(\theta)$ (computed with assumption (A3) above) of $\partial_\varepsilon \mathcal{L}(\theta)$, provided that $\mathcal{L}$ is stratifiably smooth, to update the iterates.

Definition 4.4. Let $\varepsilon > 0$. The ε -stratified subgradient of $\mathcal{L}$ is

$$\tilde{\partial}_\varepsilon \mathcal{L}(\theta) := \text{Conv} \{g_{\theta_i} \mid M_i \cap B(\theta, \varepsilon) \neq \emptyset\},$$

where θ_i is an arbitrary point of M_i such that $\|\theta - \theta_i\| \leq \varepsilon$ (obtained using assumption (A3) above), and g_{θ_i} is the vanilla gradient of $\mathcal{L}$ at θ_i . Moreover, the vector:

$$g_\theta^{\text{SGS}} := \underset{g \in \tilde{\partial}_\varepsilon \mathcal{L}(\theta)}{\text{argmin}} \|g\|$$

is called the *descent direction* associated to $\tilde{\partial}_\varepsilon \mathcal{L}(\theta)$.

A key result of [79] is that using stratified subgradients allows to guarantee a loss decrease.

Proposition 4.5 ([79, Proposition 4]). Assume $\mathcal{L}$ is stratifiably smooth, with Lipschitz constant $C > 0$. Let θ be non-stationary, and g_θ^{SGS} be the descent direction associated to $\tilde{\partial}_\varepsilon \mathcal{L}(\theta)$. Finally, let $\beta > 0$. Then:

- (i) for small enough ε , one has $\varepsilon \leq \frac{1-\beta}{2C} \|g_\theta^{\text{SGS}}\|$, and
- (ii) for such ε , and any $\alpha \leq \frac{\varepsilon}{\|g_\theta^{\text{SGS}}\|}$, one has:

$$\mathcal{L}(\theta - \alpha g_\theta^{\text{SGS}}) \leq \mathcal{L}(\theta) - \beta \cdot \alpha \cdot \|g_\theta^{\text{SGS}}\|^2.$$

Implementation. We now detail through a series of implementation how to compute faithful approximations of ε -stratified subgradients. The most basic step is Algorithm 4, that is basically an instance of gradient sampling, except that the neighborhood sample points are not chosen randomly within $B(\theta, \varepsilon)$ but will instead be picked precisely in ε -close top-dimensional strata (if any), leveraging the knowledge we have about the structure of stratification of the persistence map PH. See Figure 4.1.

Algorithm 4 StratifiedGradient(θ, S)

Input: Current iterate $\theta \in M$, set of samples $S = \{\theta_1, \dots, \theta_m\}$
 $g_\theta \leftarrow \text{VanillaGradient}(\theta)$;
 $G \leftarrow \{g_\theta\}$; # Initialize with the vanilla gradient at current iterate
for $1 \leq i \leq m$ **do**
 $g_{\theta_i} \leftarrow \text{VanillaGradient}(\theta_i)$;
 $G \leftarrow G \cup \{g_{\theta_i}\}$; # Add the vanilla gradients of samples in neighboring strata
end for
Solve quadratic minimization problem $\tilde{g}_\theta^{\text{SGS}} := \operatorname{argmin}\{\|g\|^2 \mid g \in \operatorname{Conv}(G)\}$;
return $\tilde{g}_\theta^{\text{SGS}}$

Algorithm 5 ControlledStratifiedGradient($\theta, \varepsilon, m, \gamma, \beta, C, \eta$)

Input: Current iterate θ , neighborhood size ε , sampling size m , neighborhood decrease rate $0 < \gamma < 1$, loss decrease constant β , (upper bound on the) Lipschitz constant C of $\mathcal{L}$, norm threshold η
 $\tilde{\varepsilon} \leftarrow \varepsilon$;
 $S \leftarrow \text{Sample}(\theta, \tilde{\varepsilon}, m)$;
 $g_\theta^{\text{SGS}} \leftarrow \text{StratifiedGradient}(\theta, S)$;
while $\tilde{\varepsilon} > \frac{1-\beta}{2C} \|g_\theta^{\text{SGS}}\|$ **do**
 # Ensures the neighborhood size is small enough so that the loss decrease can be controlled
 as per Proposition 4.5
 if $\|g_\theta^{\text{SGS}}\| \leq \eta$ **then**
 return $\alpha = 0$ and g_θ^{SGS}
 else
 $\tilde{\varepsilon} \leftarrow \gamma \tilde{\varepsilon}$;
 $S \leftarrow \text{Sample}(\theta, \tilde{\varepsilon}, m)$;
 $g_\theta^{\text{SGS}} \leftarrow \text{StratifiedGradient}(\theta, S)$;
 end if
end while
return g_θ^{SGS} and $\alpha := \tilde{\varepsilon} / \|g_\theta^{\text{SGS}}\|$

Then, the final stratified gradient g_θ^{SGS} is obtained by progressively refining the gradient $\tilde{g}_\theta^{\text{SGS}}$ of Algorithm 4 over smaller and smaller neighborhoods with Algorithm 5, until the neighborhood size satisfies item (i) in Proposition 4.5.

Finally, the corresponding stratified gradient descent is given by Algorithm 6.

Remark 4.6. The stratified gradient descent scheme presented in Algorithm 6 is a simplified version of the original one [79, Algorithm 2], which does not require to know the Lipschitz constant C of $\mathcal{L}$ in advance (see also Remark 5 and Algorithm 6 in [79]). However, such (upper bounds on) Lipschitz constants can be derived for most standard TDA losses (see Section 3.2).

Remark 4.7. Strictly speaking, nothing guarantees that the sequence of iterates $\{\theta_k\}_k$ belongs to the interiors of some strata (on which $\mathcal{L}$ is differentiable), which is required to ensure that the convergence properties of stratified gradient descent hold in [79]. As such, the authors of [79] use small perturbations of the iterates to make sure that $\mathcal{L}$ is differentiable on them (using assumption (A2) above). However, since obtaining iterates that end up exactly at the boundary between several strata happens with probability zero in most applications, practical implementations are usually oblivious to this fact, so we leave this issue aside in this article and refer to [79] for more details.

Remark 4.8. Instead of progressively decreasing $\tilde{\varepsilon}$ as in Algorithm 5, one can reduce it in constant time with Algorithm 7, which is a slight variation of Algorithm 5. In words, the idea is simply to fix a

Algorithm 6 StratifiedGradientDescent($\theta_0, \varepsilon, m, \gamma, \beta, C, \eta$)

Input: Initial iterate θ_0 , neighborhood size ε , sampling size m , neighborhood decrease rate $0 < \gamma < 1$, loss decrease constant β , (upper bound on the) Lipschitz constant C of $\mathcal{L}$, norm threshold η

$k \leftarrow 0$;
 $g_{\theta_{-1}}^{\text{SGS}} \leftarrow +\infty \cdot \mathbf{1}$;
while $\|g_{\theta_{k-1}}^{\text{SGS}}\| > \eta$ **do**
 $(g_{\theta_k}^{\text{SGS}}, \alpha_k) \leftarrow \text{ControlledStratifiedGradient}(\theta_k, \varepsilon, m, \gamma, \beta, C, \eta)$;
 Apply Equation (4.1) with $\alpha_k, g_{\theta_k}^{\text{SGS}}$ and $\zeta_k = 0$;
 $k \leftarrow k + 1$;
end while
return θ_k

set S_0 of samples, and to use subsets of this set instead of sampling new points when reducing $\tilde{\varepsilon}$. The reason that the condition $\tilde{\varepsilon} \leq \frac{1-\beta}{2C} \|g_{\theta}^{\text{SGS}}\|$ can be satisfied after only one iteration is that it suffices to decrease $\tilde{\varepsilon}$ to $\frac{1-\beta}{2C} \|g_{\theta}^{\text{SGS}}\|$ directly (if the condition was not satisfied initially); indeed, as the new set of samples S satisfies $S \subseteq S_0$, it follows that the corresponding gradient sets (in Algorithm 4) also satisfy $G \subseteq G_0$, and thus $\text{Conv}(G) \subseteq \text{Conv}(G_0)$ and $\min\{\|g\|^2 \mid g \in \text{Conv}(G)\} \geq \min\{\|g\|^2 \mid g \in \text{Conv}(G_0)\}$. The norm of the stratified gradient can thus only increase when using subsets of the initial sample set, ensuring that the condition becomes satisfied. However, as g_{θ}^{SGS} is computed with a smaller number of samples (as $S \subseteq S_0$) by Algorithm 4, it becomes a rougher estimation of $\text{argmin}\{\|g\|^2 \mid g \in \partial_{\varepsilon}\mathcal{L}(\theta)\}$ and it thus becomes less likely for Proposition 4.5 to apply.

Algorithm 7 ConstantTimeControlledStratifiedGradient($\theta, \varepsilon, m, \gamma, \beta, C, \eta$)

Input: Current iterate θ , neighborhood size ε , sampling size m , loss decrease constant β , (upper bound on the) Lipschitz constant C of $\mathcal{L}$, norm threshold η

$\tilde{\varepsilon} \leftarrow \varepsilon$;
 $S_0 \leftarrow \text{Sample}(\theta, \tilde{\varepsilon}, m)$;
 $g_{\theta}^{\text{SGS}} \leftarrow \text{StratifiedGradient}(\theta, S_0)$;
if $\tilde{\varepsilon} > \frac{1-\beta}{2C} \|g_{\theta}^{\text{SGS}}\|$ **then**
 if $\|g_{\theta}^{\text{SGS}}\| \leq \eta$ **then**
 return $\alpha = 0$ and g_{θ}^{SGS}
 else
 $\tilde{\varepsilon} \leftarrow \frac{1-\beta}{2C} \|g_{\theta}^{\text{SGS}}\|$;
 $S \leftarrow \{\theta \in S_0 \mid \|\tilde{\theta} - \theta\| \leq \tilde{\varepsilon}\}$;
 $g_{\theta}^{\text{SGS}} \leftarrow \text{StratifiedGradient}(\theta, S)$;
 end if
end if
return g_{θ}^{SGS} and $\alpha := \tilde{\varepsilon} / \|g_{\theta}^{\text{SGS}}\|$

Sampling strata. There are several ways to implement the sampling algorithm **Sample** required by Algorithm 5 and Algorithm 7. Recall that the main objective for **Sample** is the ability to sample points in ε -close strata. In the case of the persistence map PH, the strata are completely characterized by permutations of the filtration values as per Proposition 3.21. Hence, a practical way of exploring neighboring strata is by looking at shortest paths on the permutahedron: first identify the permutation associated to the current iterate θ , retrieve the corresponding node V_{θ} on the permutahedron, and then explore other permutations / strata by looking for the permutahedron nodes that are the closest to V_{θ} with, e.g., Dijkstra's algorithm or diffusion. This will allow to explore the strata that can be obtained from the current one by a minimal number of transpositions first. Moreover, it is also possible to store

the persistence pairs and unpaired simplices corresponding to the permutations that have already been visited, so that the vanilla gradients need not necessarily be recomputed from scratch at every iteration of Algorithm 6. See also [79, Section 4.2]. Recall however that when the filtration map F is also stratified, the strata of PH and those of F have to be combined in the sampling algorithm `Sample`.

Convergence guarantees. In short, it can be shown that stratified gradient descent converges to an (ε, η) -stationary point in a finite number of iterations, which is a substantial improvement over the guarantees of vanilla gradient descent from a theoretical viewpoint.

Theorem 4.9 ([79, Thm. 7]). If $\eta > 0$, then Algorithm 6 produces an (ε, η) -stationary point using at most $O\left(\frac{1}{\eta \min(\varepsilon, \eta)}\right)$ iterations.

The above theorem provides explicit conditions ensuring the convergence of stochastic subgradient descent for functions of persistence. The main criterion to be checked is the local Lipschitz condition for $\mathcal{L}$, which is guaranteed as soon as F and L are Lipschitz.

4.3 Big-step gradient descent

In this section, we detail another gradient, that we call the *big-step* gradient g_θ^{BS} . It was introduced in [91] for minimizing singleton losses (see Example 3.12), in a much faster way than vanilla gradient descent. The key idea of the big-step gradient is to move, for a single update on PD points, a much bigger set of simplices (and their filtration values) than the associated persistence pairs and unpaired simplices only. In terms of stratified spaces, this provides a way to skip a lot of strata with one “jump” in a single gradient descent iteration—see Figure 4.1. This also allows to make the singleton loss—associated to a persistence pair $q_0 = (\sigma, \tau)$ —more “global”: indeed, since big-step gradient descent always keeps σ and τ paired together by design, it makes sense to use a loss that is defined with respect to the PD point q_0 , which always exists—on the other hand, if gradient iterates were allowed to go through different strata that possibly do not pair σ and τ together, the singleton loss would be ill-defined.

Minimizing singleton loss. Let K be a simplicial complex, and $f: K \rightarrow \mathbb{R}$ be a filtration of K . In this paragraph, we focus on minimizing a singleton loss, that is, pushing one PD point p_0 towards a target point $q_0 = (q_x, q_y)$. Let (σ, τ) be the persistence pair corresponding to p_0 . The goal is to update the value of $f(\sigma)$ towards q_x , and similarly for $f(\tau)$ and q_y , while ensuring that they stay matched together so that the singleton loss stays well-defined.

In other words, given a p -simplex τ and a target value t , we want to change the value of $f(\tau)$ towards t without modifying the simplex σ that is paired with τ . To this end, we want to find the p -simplices that belong to the set X_σ , defined as:

$$X_\sigma := \{\tau' \in \text{Sk}_p(K) \mid \sigma \text{ becomes paired with } \tau' \text{ after swapping } \tau \text{ and } \tau' \text{ in the filtration}\}, \quad (4.2)$$

where $\text{Sk}_p(K)$ is the p -skeleton of K (see Definition 2.1).

Indeed, if one sets $f(\tau')$ to t directly for all $\tau' \in X_\sigma$, one can then safely move $f(\tau)$ to t directly without modifying the simplex σ that is paired with τ . This allows to modify much more simplices at the same time than the vanilla gradient, leading to much faster convergence in the iterations of Equation (4.1). Hence, one defines a new gradient as follows.

Definition 4.10. The *big-step* gradient g_θ^{BS} associated to the singleton loss L is obtained as the usual vanilla gradient computed with a modification of the differential of the map L , obtained by replacing the partial derivatives associated to every simplex $\tau' \in X_\sigma$ and every simplex $\sigma' \in X_\tau$ with the partial derivatives of the simplices τ and σ respectively, where (σ, τ) is the persistence pair associated to L .

Implementation. The big-step gradient can be computed with Algorithm 8,¹¹ which is a variation of Algorithm 3. Gradient descent can then be performed with Equation (4.1).

Algorithm 8 BigStepGradient(θ)

Input: Current iterate $\theta \in M$, critical simplices (σ, τ) associated to singleton loss L
 $((\sigma_1, \sigma_2), \dots, (\sigma_{2m+1}, \sigma_{2m})), (\tau_1, \dots, \tau_n) \leftarrow \text{PersistencePairs}(F(\theta));$
 $\alpha \leftarrow [[F(\theta)]_{\sigma_1}, \dots, [F(\theta)]_{\sigma_{2m}}, [F(\theta)]_{\tau_1}, \dots, [F(\theta)]_{\tau_n}] \in \mathbb{R}^{2m+n};$
 $\nabla_\theta[\alpha]_i \leftarrow \nabla_\theta[F(\theta)]_{\sigma_i}, \forall 1 \leq i \leq 2m;$
 $\nabla_\theta[\alpha]_{2m+j} \leftarrow \nabla_\theta[F(\theta)]_{\tau_j}, \forall 1 \leq j \leq n;$
 $t_1 \leftarrow \frac{\partial L}{\partial[\alpha]_\sigma}, t_2 \leftarrow \frac{\partial L}{\partial[\alpha]_\tau};$
 # Compute simplices that one needs to move jointly with σ and τ to preserve persistence pairs
 (see Equation (4.2) and Algorithm 9)
 $X_\sigma \leftarrow \text{MovingSet}(\sigma, \tau, t_1, F(\theta));$
 $X_\tau \leftarrow \text{MovingSet}(\tau, \sigma, t_2, F(\theta));$
 # Update partial derivatives of simplices to match match them with partial derivatives of σ, τ
for $\tau' \in X_\sigma$ **do**
 $\frac{\partial L}{\partial[\alpha]_{\tau'}} \leftarrow t_1;$
end for
for $\sigma' \in X_\tau$ **do**
 $\frac{\partial L}{\partial[\alpha]_{\sigma'}} \leftarrow t_2;$
end for
 $g_\theta^{\text{BS}} \leftarrow \sum_{i=1}^{2m+n} \frac{\partial L}{\partial[\alpha]_i} ([\alpha]_i) \cdot \nabla_\theta[\alpha]_i;$
return g_θ^{BS}

We now present how to compute the set X_σ with the function `MovingSet`. To ease notation, let $f := F(\theta)$ be the current filtration. Suppose there are $|\text{Sk}_p(K)| = m_p$ simplices $\tau_1 (= \tau), \dots, \tau_{m_p}$ (sorted in ascending order according to the proximity of their values under f to $f(\tau)$) between $f(\sigma)$ and t . We define the set X_σ^k ($k = 1, \dots, m_p$) as the set of p -simplices as follows:

- For $k = 1$, $X_\sigma^1 := \{\tau_1\}$.
- For $k = 2, \dots, m_p$:

$$X_\sigma^k := \begin{cases} X_\sigma^{k-1} & \text{if } \tau \text{ is paired with } \sigma \text{ for the filtration } f_k \\ X_\sigma^{k-1} \cup \{\tau_k\} & \text{otherwise} \end{cases}$$

where the order induced by f_k is obtained from that of f with the following modifications:

1. For any $\tau' \in X_\sigma^{k-1}$, the order of τ_k and τ' is swapped.
2. For any $\sigma_1, \sigma_2 \in K$ with $\sigma_1, \sigma_2 \notin X_\sigma^{k-1} \cup \{\tau_k\}$, the order of σ_1 and σ_2 is preserved.

Now, the set $X_\sigma := X_\sigma^{m_p}$ is the set of p -simplices defined in Equation (4.2).

One way for obtaining $X_\sigma^{m_p}$ is to iteratively compute X_σ^k based on the definition for $k = 1, \dots, m_p$, as described in Algorithm 9. To examine whether the persistence pair (σ, τ) changes due to swapping simplex τ_k with the simplices in the set X_σ^{k-1} , one can use the algorithm proposed in [46]. This algorithm computes the persistence pair changes in $O(|K|)$ time, for a single swap of adjacent simplices (in the order induced by the filtration). In the k -th for-loop, we need to swap simplices $2(k-1)$ times, and thus the overall number of swaps is $O(m_p^2)$ in the worst case. Therefore, the overall computational complexity is $O(|K| \cdot m_p^2)$.

¹¹Note that if the parameter space M is Filt_K itself, the updates of the partial derivatives prescribed by Algorithm 8 need also be applied on the faces (if the simplex value is decreased) or cofaces (if the simplex value is increased) of the simplices in X_σ and X_τ in order to ensure that $F(\theta)$ remains a filtration.

Algorithm 9 MovingSet(τ, σ, t, f)

Input: Target simplex $\tau \in \text{Sk}_p(K)$, the simplex σ paired with τ , target value t , filtration f
 $X_\sigma^1 \leftarrow \{\tau\};$
for each $\tau_k \in \text{Sk}_p(K)$ with $f(\tau_k)$ between $f(\tau)$ and t **do**
 transpose τ_k with each simplex in X_σ^{k-1} , updating the pairing using the algorithm in [46]; — (†)
 if τ_k becomes paired with σ **then**
 $X_\sigma^k \leftarrow X_\sigma^{k-1} \cup \{\tau_k\};$
 undo the transpositions in (†) so that τ_k is at the opposite end of X_σ^k ;
 else
 $X_\sigma^k \leftarrow X_\sigma^{k-1};$
 end if
end for
return $X_\sigma^{m_p}$

To develop a faster algorithm for finding $X_\sigma^{m_p}$, the authors in [91] provide a method to compute $X_\sigma^{m_p}$ in $O(m_p)$ time. The key observation is that persistence pairing is computed by reducing the boundary matrix, which can be interpreted as finding decompositions $R_p = D_p V_p$, where matrices R_p are reduced, meaning the lowest non-zeros in their columns appear in unique rows, and V_p are upper-triangular invertible matrices (see [56, Section VII.1]). Sticking with the notation of [91], we let $U_p = V_p^{-1}$. Building on this idea, they prove the following theorem, which shows that we can compute $X_\sigma^{m_p}$ explicitly using such matrices U_p, V_p .

Theorem 4.11. Let X_σ be the set of p -simplices computed by Algorithm 9. It holds

$$X_\sigma = \begin{cases} \{\tau' \mid t < f(\tau') < f(\tau), V_p[\tau', \tau] \neq 0\} & \text{if } \tau \text{ is a death simplex and } t < f(\tau), \\ \{\tau' \mid f(\tau) < f(\tau') < t, U_p[\tau, \tau'] \neq 0\} & \text{if } \tau \text{ is a death simplex and } f(\tau) < t, \\ \{\tau' \mid t < f(\tau') < f(\tau), U_p^\perp[\tau, \tau'] \neq 0\} & \text{if } \tau \text{ is a birth simplex and } t < f(\tau), \\ \{\tau' \mid f(\tau) < f(\tau') < t, V_p^\perp[\tau, \tau'] \neq 0\} & \text{if } \tau \text{ is a birth simplex and } f(\tau) < t. \end{cases}$$

Minimizing combined loss. Given a more general loss $L = \sum_{(p,q) \in \gamma} (p - q)^2$, where $\gamma \subseteq \alpha \times \alpha_0$ is a (fixed) partial matching between the PD of the current estimate θ and a fixed, target PD α_0 , one difficulty is that the different terms in the sum can induce several different target values for every simplex. Hence, one has to reduce these multiple target values to a single one for each simplex. In order to achieve this, the authors in [91] propose the following heuristic. Fix a simplex $\sigma \in K$. Let $a := f(\sigma)$ be the initial value, and $a_1, a_2, \dots$ be the different target values induced by the singleton losses appearing in the combined loss for σ . Then, they propose to choose the target value a' for σ (and the corresponding singleton loss) with:

$$a' = a_{i^*}, \quad \text{where } i^* = \operatorname{argmax}_i |a - a_i|.$$

Intuitively, this advocates for using the largest push on every simplex—see [91, Section 3.7] for a motivation of this choice.

4.4 Gradient extensions

In this section, we assume that some procedure for computing a gradient $(\theta, P, K) \mapsto G_{\theta, P, K}$ for $\mathcal{L}$ at θ on the simplicial complex K has been fixed (with P being additional parameters depending on the procedure), whether it is the vanilla ($P = \emptyset$), stratified ($P = \{\varepsilon, m, \gamma, \beta, C, \eta\}$) or big-step method ($P = \{\sigma, \tau\}$), and we present two extensions of these gradients, that is, two additional procedures that produce new gradients-like objects with additional benefits, such as being less sparse, more robust or more computationally efficient.

4.4.1 Smoothing gradient with downsampling

A simple idea to reduce the computational cost and sparsity of topological gradients is to compute them on several smaller simplicial complexes of fixed and controlled sizes, and average the results. This is the approach called *downsampling simplicial complexes*, advocated in [131] and [119]. See Figure 4.1. In these works, two different ways are proposed for downsampling simplicial complexes.

Downsampling with subcomplexes. The first approach involves averaging gradients computed from subcomplexes of the initial simplicial complex.

Definition 4.12. Let K be a simplicial complex and $\mathcal{P}(K)$ be a family of subcomplexes of K . Let $G_{\theta,P,K}$ be a gradient computation procedure. The *downsampled, or distributed gradient* is defined as:

$$\tilde{G}_{\theta,P,K} := \frac{1}{|\mathcal{P}(K)|} \sum_{K' \in \mathcal{P}(K)} G_{\theta,P,K'},$$

where the gradients $G_{\theta,P,K'}$ were computed using the original filtration restricted to K' .

A common example of family of subcomplexes is $\mathcal{P}_n(K) := \{K' \subseteq K \mid |\text{Sk}_0(K')| = n \text{ and } \forall \sigma \in K, \text{Sk}_0(\sigma) \subseteq \text{Sk}_0(K') \implies \sigma \in K'\}$, comprised of those maximal subcomplexes with exactly n vertices. Such a family was used in [131] for instance, in which the authors focused on the family of Vietoris–Rips complexes computed from subsamples (of fixed size) of an initial point cloud. All such complexes can be seen as subcomplexes of the Vietoris–Rips complex of the full point cloud.

Downsampling with nerves. The second approach involves averaging gradients computed from *nerve complexes*.

Definition 4.13. Let K be a simplicial complex and $\mathcal{U}$ be an open cover of K , that is, a family of open sets such that $K = \bigcup_{U \in \mathcal{U}} U$. The *nerve* of $\mathcal{U}$ is the simplicial complex $\mathcal{N}(\mathcal{U})$ with vertices $\text{Sk}_0(\mathcal{N}(\mathcal{U})) = \mathcal{U}$ and simplices:

$$\sigma = \{U_{i_0}, \dots, U_{i_p}\} \in \mathcal{N}(\mathcal{U}) \iff \bigcap_{j=0}^p U_{i_j} \neq \emptyset.$$

Definition 4.14. Let K be a simplicial complex and $\mathcal{C}$ be a family of open covers of K . Let $G_{\theta,P,K}$ be a gradient computation procedure. Then, the *nerve gradient* is defined as:

$$\tilde{G}_{\theta,P,K} := \frac{1}{|\mathcal{C}|} \sum_{\mathcal{U} \in \mathcal{C}} G_{\theta,P,\mathcal{N}(\mathcal{U})},$$

where the gradients $G_{\theta,P,\mathcal{N}(\mathcal{U})}$ were computed using a filtration of $\mathcal{N}(\mathcal{U})$ derived from the original one.

In [119], the authors suggest several way for designing a filtration on $\mathcal{N}(\mathcal{U})$. One possibility is, e.g., to assign to every vertex U_i of $\mathcal{N}(\mathcal{U})$ the average of the original filtration values of the vertices of K that belong to U_i , or, more generally, any weighted average of such values such that the weights sum to one, and to extend to the whole nerve with lower-star (see Example 3.32). Gradient smoothing can be made even stronger by computing $G_{\theta,P,\mathcal{N}(\mathcal{U})}$ as the integral over all such weighted averages.

Application to topological gradient descent. There are several benefits for using either downsampled or nerve gradients in Equation (4.1). Indeed, they both tend to produce denser gradients (as they incorporate several gradients computed on different complexes) in a much faster way than the original gradient procedures (as the size of complexes they are applied to are controlled).

4.4.2 Extending gradient with diffeomorphic interpolation

This approach proposed in this section, introduced in [34], proposes an alternative way to extend a gradient associated to the persistence diagram of a *geometric* simplicial complex K , that is, a simplicial complex embedded in $\mathbb{R}^d$ with a filtration that is entirely parametrized by the location of the vertices of K , i.e., any parameter θ is of the form $\theta = \{x_1, \dots, x_n\} \in (\mathbb{R}^d)^n$. Examples of such filtrations include the Vietoris–Rips, the Čech, or the height filtrations (see Section 2.1). As explained before, any given topological loss $\mathcal{L}$ yields a vanilla gradient $G_{\theta, \emptyset, K} \in \partial\mathcal{L}(\theta) \in \mathbb{R}^{n \times d}$ that is typically sparse, as its rows represent the x_i ’s, and are thus 0 for any non critical vertex in the filtration.

The main idea conveyed by this approach is to build a vector field $\tilde{G}_{\theta, P, K}: \mathbb{R}^d \rightarrow \mathbb{R}^d$ that *interpolates* any topological gradient $G_{\theta, P, K}$ on its *non-zero* entries. As they are many possible candidates, it is natural to seek the *smoothest* way to interpolate the gradient, which is done by finding a vector field of minimal norm in a given Reproducing Kernel Hilbert Space (RKHS), hence the name of *diffeomorphic interpolation*.

Diffeomorphic interpolation. Let $\theta = \{x_1, \dots, x_n\} \in \mathbb{R}^{n \times d}$, let $I \subseteq \{1, \dots, n\}$ and consider a set of vectors $a_i \in \mathbb{R}^d$ for $i \in I$. Let $k: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}^{d \times d}$ be a matrix-valued kernel operator whose outputs are symmetric and positive definite, i.e., $\forall x, y, \alpha \in \mathbb{R}^d$, $\alpha^T k(x, y) \alpha \geq 0$. This kernel induces an RKHS¹² $\mathcal{H} \subset (\mathbb{R}^d)^{\mathbb{R}^d}$ whose elements are vector fields $V: \mathbb{R}^d \rightarrow \mathbb{R}^d$. Observe also that, as for any $\alpha, x \in \mathbb{R}^d$, $V \in \mathcal{H} \mapsto \alpha^T V(x) \in \mathbb{R}$ is a (continuous) linear form, Riesz’s representation theorem gives the existence of $k_x^\alpha \in \mathcal{H}$ such that $\langle k_x^\alpha, V \rangle_{\mathcal{H}} = \langle \alpha, V(x) \rangle$. The goal is thus to find $\tilde{G}_{\theta, P, K} \in \mathcal{H}$ such that, for all $i \in I$, $\tilde{G}_{\theta, P, K}(x_i) = a_i$ (with a_i being the non-zero entries of $G_{\theta, P, K}$) that would be as smooth as possible, i.e., of minimal norm, yielding the minimization problem:

$$\text{minimize } \|V\|_{\mathcal{H}}, \text{ s.t. } V(x_i) = a_i, \forall i \in I.$$

The solution $\tilde{G}_{\theta, P, K}$ of this problem is the projection of 0 onto the affine set $\{V \in \mathcal{H} \mid V(x_i) = a_i, \forall i \in I\}$ and thus belongs to $\{V \in \mathcal{H} \mid V(x_i) = 0, \forall i \in I\}^\perp$, and thus to $\{V \in \mathcal{H} \mid \langle k_{x_i}^{\alpha_i}, V \rangle_{\mathcal{H}} = 0, \forall i \in I, \alpha_i \in \mathbb{R}^d\}^\perp$. Eventually $\tilde{G}_{\theta, P, K} \in \text{span}(\{k_{x_i}^{\alpha_i} \mid i \in I\})$. This justifies to search for $\tilde{G}_{\theta, P, K}$ in the form of $\tilde{G}_{\theta, P, K}(x) = \sum_{i \in I} k(x, x_i) \alpha_i$, and the interpolation that it must satisfy yields the following definition (see also [135, Theorem 8.8]).

Definition 4.15. Let K be a geometric simplicial complex associated to $\theta = \{x_1, \dots, x_n\} \in (\mathbb{R}^d)^n$, k be a kernel on $\mathbb{R}^d$, and $G_{\theta, P, K}$ be a gradient computation procedure. Let $\{a_i\}_{i \in I}$, $I \subseteq \{1, \dots, n\}$ be the collection of non-zero entries of $G_{\theta, P, K}$. Then, the *diffeomorphic gradient* is defined as:

$$\tilde{G}_{\theta, P, K}(x) = \sum_{i \in I} k(x, x_i) (\mathbb{K}^{-1} a)_i,$$

where $\mathbb{K}$ is the block matrix $(k(x_i, x_j))_{i, j \in I}$ and $a = (a_i)_{i \in I}$.

In particular, $\tilde{G}_{\theta, P, K}$ inherits from the regularity of k and will typically be a diffeomorphism. Note that $\tilde{G}_{\theta, P, K}$ can be understood as the convolution of $G_{\theta, P, K}$ with the kernel k , but involving a correction $\mathbb{K}^{-1}$ guaranteeing that after the convolution, the interpolation constraint is satisfied. See Figure 4.1.

Example 4.16. A natural choice of a kernel operator is to let k be the Gaussian kernel defined by $k(x, y) := \exp\left(-\frac{\|x-y\|^2}{2\sigma^2}\right) I_d$ for some bandwidth $\sigma > 0$. In that setting, the expression of $\tilde{V}$ reduces to

$$\tilde{G}_{\theta, P, K}(x) = \sum_{i \in I} \rho_\sigma(\|x - x_i\|) \alpha_i,$$

where $\rho_\sigma(u) := e^{-\frac{u^2}{2\sigma^2}}$, and $\alpha_i := (\mathbb{K}^{-1} a)_i$ with $\mathbb{K} = (\rho_\sigma(\|x_i - x_j\|) I_d)_{i, j \in I}$.

¹²In many applications, RKHS are restricted to spaces of functions valued in $\mathbb{R}$, but the theory adapts faithfully to the more general setting of vector-valued maps.

Application to topological gradient descent. Applying Equation (4.1) with $\tilde{G}_{\theta,P,K}$ (instead on $G_{\theta,P,K}$) on the point cloud $\theta = \{x_1, \dots, x_n\}$ leads to several benefits:

- From a computational perspective, the most expensive operation is to invert a $|I| \times |I|$ matrix, where I denotes the non-zero entries of $G_{\theta,P,K}$ and is typically fairly small due to its sparsity (especially for the vanilla gradient $G_{\theta,\emptyset,K}$).
- The diffeomorphism $\tilde{G}_{\theta,P,K}$ is defined on the *whole space* $\mathbb{R}^d$, not only on the set of vertices $\{x_1, \dots, x_n\}$. It enables *extrapolation* of the gradient to unseen points. This observation induces several useful consequences:
 1. Diffeomorphic gradients combine very well with distributed gradients (see Section 4.4.1): if $\{x_1, \dots, x_n\}$ is actually a subsample of a much larger point cloud X (for which computing persistence diagrams is computationally prohibitive), then one can compute $\tilde{G}_{\theta,P,K}$ on $\{x_1, \dots, x_n\}$ only, and then use it in order to update the whole point cloud X .
 2. Diffeomorphic gradients can be *reused*: assume one has performed topological optimization on some dataset, and a new data point is given. In order to optimize it as well with vanilla gradient descent, one would have to redo topological optimization from scratch using the original dataset complemented with the new point: indeed, the vanilla gradients recorded in the previous optimization are not well-defined on the new data point. On the other hand, the sequence of diffeomorphic gradients from previous optimization can be reapplied at no additional cost as they are vector fields.
- In the regime $\alpha_k \rightarrow 0$, the update in Equation (4.1) yields the same decrease in the loss $\mathcal{L}$ than an update with the vanilla gradient $G_{\theta,\emptyset,K}$.

Remark 4.17. When performing several iterations of topological optimization, one can collect the sequence of diffeomorphic gradients, and produce a *discretized gradient flow* by composing these vector fields. Note however that producing a flow from a sequence of vanilla gradients can also be done in different ways (i.e., without necessarily using diffeomorphic interpolations with kernels), the only required ingredient is to build a parametrized time varying vector field $v_t(\cdot; \theta): \mathbb{R}^d \rightarrow \mathbb{R}^d$ that interpolates, or at least approximate, the vanilla gradient whenever that one is non-zero. An appealing alternative is thus to rely on *Neural Ordinary Differential Equations* (NODEs), which are ODEs that are parametrized by neural network architectures, i.e., $\dot{x}(t) = v_t(x, \theta)$ where v_t is a neural network taking $x \in \mathbb{R}^d$ as input and θ denotes its parameters. These neural nets are trained so that they induce ODEs which produce time-varying datasets that are as close as possible to some ground-truth datasets computed explicitly on a finite number of time points. If one measures these proximities between datasets with PDs (using, e.g., Vietoris–Rips filtrations), learning a NODE will produce a flow which tries to match the ground-truth PDs (on the provided time points). Namely, the typical loss to be optimized would be $\theta \mapsto \text{FG}(\text{Dgm}(x_\theta(t=1)), \alpha_{\text{ref}})$, where $x_\theta(t=1)$ denotes the solution of the ODE at time $t=1$ (with parameters θ), $\text{Dgm}(x(t=1))$ is its Vietoris–Rips persistence diagram, and α_{ref} is some target reference persistence diagram.

4.5 Summary table

Table 4.1 synthesizes the guarantees and complexities associated to the gradient schemes and extensions presented in this section. Recall that the baseline is the vanilla gradient, whose worst-case complexity is cubic with respect to the number of simplices considered N . In Table 4.1, we used the following notations:

1. $\Omega_\varepsilon^{\text{Sample}}$ is the complexity of sampling gradients on strata at distance ε using the oracle associated to Assumption (A3) in Section 4.2,

2. $\Omega_{S_\varepsilon}^{\text{QP}}$ is the complexity (that depends on the numerical solver) of solving the quadratic programming problem $\text{argmin}\|g\|^2$ subject to S_ε constraints (where S_ε is the number of strata returned by the oracle),
3. $\Omega_{d,|I|}^{\text{Inv}}$ is the complexity of inverting a matrix of size $d \cdot |I|$ (where d is the data dimension and $|I|$ is the number of non-zero entries of the gradient), and
4. n is the number of downsampled complexes with N' simplices, and m_p is the number of p -simplices, where p is the dimension of the simplex associated to the singleton loss.

Note that both gradient schemes and both gradient extensions make the vanilla gradient denser, ensuring a smoother descent. In terms of limitations, recall that interpolation requires geometric simplicial complexes, and that big-step gradients only work for singleton (or combined) losses.

	Complexity	Guarantees (in addition to smoother descent)
Vanilla	$O(N^3)$	-
Stratified	$O(N^3) + O(\Omega_\varepsilon^{\text{Sample}}) + O(\Omega_{S_\varepsilon}^{\text{QP}})$	Non-asymptotic convergence
Big-step	$O(N^3) + O(N \cdot m_p^2)$	Empirically faster descent
Distributed	$O(n \cdot (N')^3)$	Scalable + Faster iterations
Diffeomorphic	$O(N^3) + O(\Omega_{d, I }^{\text{Inv}})$	Scalable + Reusable gradients

Table 4.1: Comparisons of the gradient schemes and extensions presented in Section 4 for a fixed simplicial complex with N simplices.

5 An overview of some applications

We now provide a tour of practical applications involving topological optimization based on persistent homology. The aim is not to be exhaustive, but rather to give an overview of typical problems for which topological optimization has proved to be useful. We separate these problems into two families: *filtration learning for producing better descriptors*, in which filtrations used for PD computation are automatically learned instead of being chosen a priori (see Section 5.1), and *topological regularization for constraining models*, in which PDs are used as features for penalties in statistical inference and machine learning models in order to reduce complexity and/or use topological priors (see Section 5.2).

5.1 Filtration learning

The aim of filtration learning is to optimize filtrations so that the resulting PDs (and their subsequent vectorizations if applicable) are tailored for solving a given data science task. This provides a way for designing topological descriptors that does not rely on a user-specific filtration choice. It is often used in a supervised context, where one is given some simplicial complexes $\{K_i\}_{1 \leq i \leq n}$, together with some labels $\{y_i\}_{1 \leq i \leq n}$ from a label space $\mathcal{Y}$, that a model $\phi: \mathcal{D} \rightarrow \mathcal{Y}$ aims at predicting using the information captured by PDs. Typically, each data point induces a filtration map $F_i: \theta \mapsto f_\theta \in \text{Filt}_{K_i}$, usually parametrized with, e.g., a neural network, and a loss L_i of the form $L_i: \alpha \mapsto \ell(y_i, \phi(\alpha))$, where ℓ depends on the label type (e.g., mean squared error when $y_i \in \mathbb{R}^d$, or cross-entropy when y_i is a categorical variable). The final loss is then:

$$\mathcal{L}: \theta \mapsto \sum_{i=1}^n L_i(\text{PH} \circ F_i(\theta)). \quad (5.1)$$

Remark 5.1. It is common that the model ϕ depends itself on some parameters θ' , i.e., it is of the form $\phi_{\theta'}(\alpha)$. These parameters are also optimized at training time, leading to a final loss of the form

$\mathcal{L}(\theta, \theta')$. This typically happens when, e.g., one wants to complement the topological descriptors with some standard models and/or descriptors, which come with their own parameters. For the sake of simplicity, we leave this point aside in this section.

5.1.1 Filtration learning for images

Reference article: Barbarani et al. [10]

In this work, authors propose to rely on topological optimization to propose a methodology to detect *salient points* on images, or image *keypoints*. The main idea is to learn a filtration F_θ of which the local maxima (recorded in a PD in the sub) are interpreted as keypoints. In this case, the filtration map $F_\theta: \mathbb{R}^{d_1 \times d_2} \rightarrow \mathbb{R}^{d'_1 \times d'_2}$ is encoded as a convolutional neural network (CNN), d_1, d_2 (resp. d'_1, d'_2) being the shape of the input (resp. output) image. Given an input image $x \in \mathbb{R}^{d_1 \times d_2}$, $F_\theta(x)$ can be understood as a height map amenable to persistence computation (see Example 2.15). See Figure 5.1 for a schematic illustration of the pipeline described in [10]. The topological loss L_i is defined on each image I_i as

$$L_i = \sum_{j \neq i} L_{(I_i, I_j)}(\text{PH} \circ F_\theta(I_i), I_j),$$

where $L_{(I_i, I_j)}(\alpha_i, I_j)$ is the sum over the PD points in α_i of the products between: 1. the opposite of the lifetime of the PD point (hence strongly penalizing points close to the diagonal, thus favoring peaked height maps), and 2. a similarity term measuring the differences between (a) the height map values of the critical pixels in I_i associated to the persistence pair of the PD point, with (b) the height map values of their corresponding pixels in I_j using a ground-truth correspondence U between the pixels of I_i and I_j (hence penalizing keypoints that are not reproducible across images). In a nutshell, L_i is small if $F_\theta(I_i)$ has its prominent local maxima on pixels that also correspond to local maxima in the other images $\{I_j\}$ under U . The benefits on using persistent homology in this work is that it removes the need of using any prior and/or hyperparameters on the keypoint densities, scales and frequencies across images, in contrast to related works in the literature.

5.1.2 Filtration learning for graphs

Reference article: Horn et al. [68]

Given that graphs can be interpreted as 1-dimensional simplicial complexes, it is natural to incorporate topological features in graph classification and regression. In the context of the work [68], this is done in the following way. Graph nodes (a.k.a. vertices) $v \in V$ come with attributes $x^{(v)} \in \mathbb{R}^d$. A first network $\Phi: \mathbb{R}^d \rightarrow \mathbb{R}^k$ (e.g., a single hidden layer network in [68]) assigns to each node $x^{(v)}$ a set of k values $a_1^{(v)}, \dots, a_k^{(v)}$, yielding k different *views* of the graph G : each view represents the same graph G but with different attributes $a_i^{(v)} \in \mathbb{R}$, $i \in \{1, \dots, k\}$, $v \in V$ on the nodes. Given these k views, one can consider the filtrations $(G_{i,t})_{t \geq 0}$ for $i = 1, \dots, k$ where the set of vertices of $G_{i,t}$ is $\{v \in V \mid \Phi(x^{(v)})_i < t\}$, and the set of edges is $\{(v, v') \mid \max(\Phi(x^{(v)})_i, \Phi(x^{(v')})_i) < t\}$. These k filtrations yield k persistence diagrams (accounting for homology dimension 0 and 1). These k diagrams are themselves turned into vectors (see Example 3.13) and aggregated by another network $\Psi: \mathcal{D}^k \rightarrow \mathbb{R}^d$, eventually processed to obtained a new representation $\tilde{x}^{(v)}$ of the node v which accounts for the topological structure of the initial graph G with nodes attributes $x^{(v)}$. Here, the maps Φ and Ψ are optimized during training, involving an intermediate topological optimization step, yielding a loss of the form of Equation (5.1). See Figure 5.2 for a schematic illustration of the pipeline developed in [68]. We refer the interested reader to [103] for another example of graph filtration learning, and graph classification problems with topological descriptors.

5.1.3 Filtration learning for geometric complexes

Reference article: Nishikawa et al. [92]

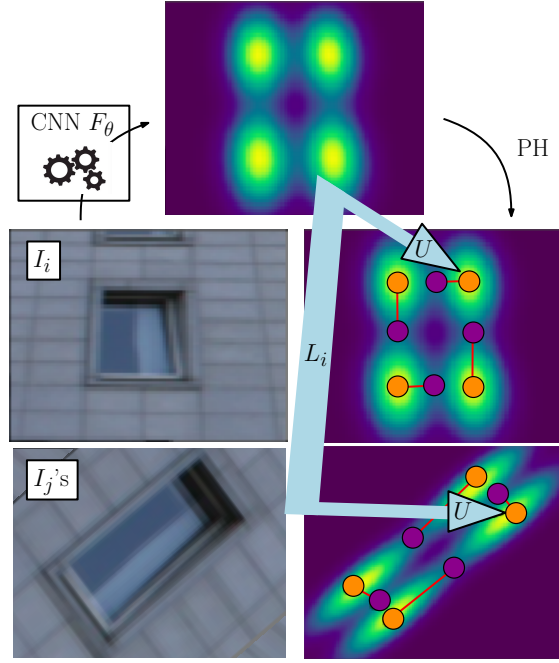


Figure 5.1: Filtration learning for images. Once orange/magenta pixels corresponding to prominent maxima of the height map (i.e., yielding points with large persistence in the PD) have been obtained for two given images, they are compared using a ground truth correspondence U . The goal is to optimize the CNN weights θ so that F_θ produces useful height maps, in the sense that corresponding PDs enable the identification of keypoints as local maxima of the height map. Inspired from [10, Figure 2].

Learning filtrations can also be achieved for geometric complexes induced by point clouds. Such a method has recently been proposed for weighted Rips filtrations (see Example 3.31). More precisely, for a given point cloud X , the weight function $f = F(\theta) = f_\theta : X \rightarrow \mathbb{R}$ can be parametrized with a combination of DeepSet [136] and fully-connected neural network architectures trained on the pairwise distance matrices of the point clouds. See Figure 5.3.

More formally, given a point cloud $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ and a point $x \in \mathbb{R}^d$, the proposed network outputs a weight value $f_\theta(X, x)$ for the point x . To make the resulting PD isometry invariant, it is desirable that the weight function also enjoys such invariance. To achieve this, the network architecture computes features based on distance matrices, that can be decomposed in the following three parts:

Pointwise feature. The feature of point $x \in \mathbb{R}^d$ can be obtained through aggregation of the distances $\|x - x_1\|, \dots, \|x - x_n\|$ to the points in X as follows:

$$g_1(x) := \phi^{(2)}(\text{op}(\{\phi^{(1)}(\|x - x_j\|)\}_{j=1}^n)),$$

where $\phi^{(1)}$ and $\phi^{(2)}$ are fully-connected neural networks.

Global feature. The global feature of the entire point cloud X is inspired by the DeepSet architecture [136] and can be obtained as follows. First, pointwise feature vectors are computed with the same architecture as g_1 , i.e.,

$$g_2(x_i) := \phi^{(4)}(\text{op}(\{\phi^{(3)}(d(x_i, x_j))\}_{j=1}^n)),$$

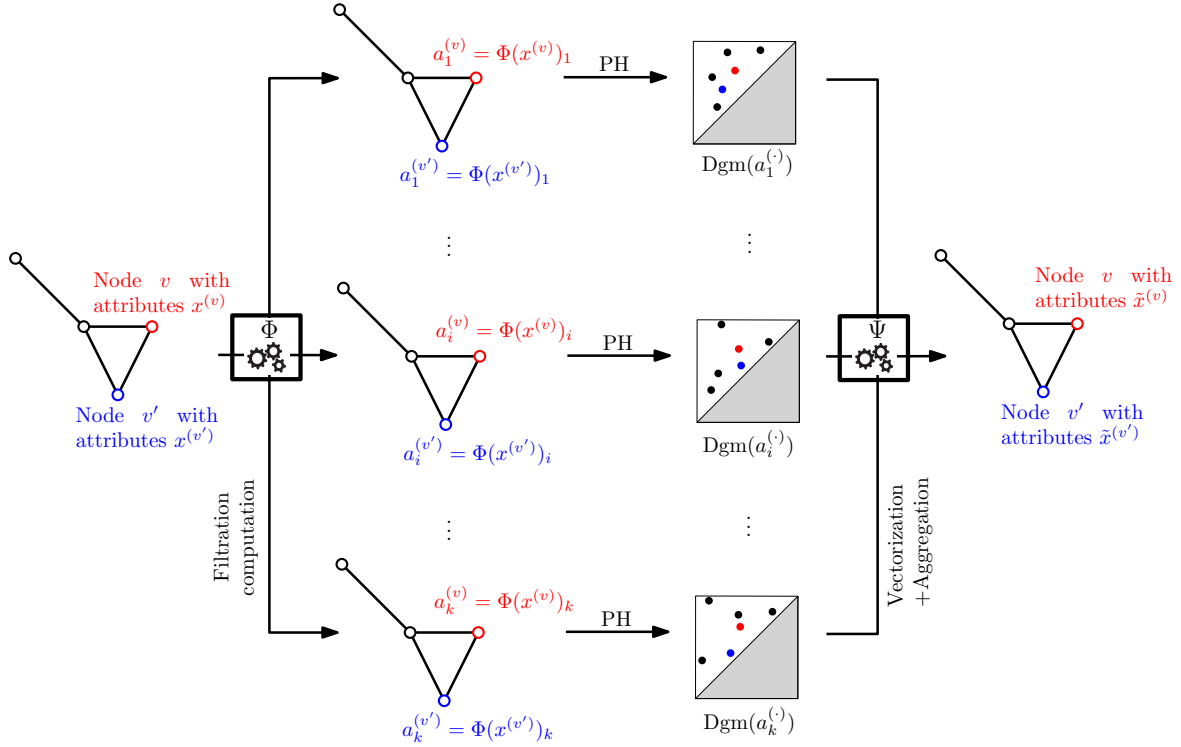


Figure 5.2: Filtration learning for graphs. Inspired from [68, Figure 2].

where $\phi^{(3)}$ and $\phi^{(4)}$ are fully-connected neural networks. Then, a feature vector $h(X)$ for the entire point cloud X is obtained by applying a DeepSet architecture:

$$h(X) := \phi^{(5)}(\text{op}(\{g_2(x_i)\}_{i=1}^n)),$$

where $\phi^{(5)}$ is a fully-connected neural network.

Combining features. The local and global features can be gathered in the concatenated vector $[h(X), g_1(x)]$. The weight value of point x can finally be obtained through a fully-connected neural network $\phi^{(6)}$:

$$f_\theta(X, x) := \phi^{(6)}([h(X), g_1(x)]^\top),$$

where $\theta := (\theta_k)_{k=1}^6$ is the set of parameters that appear in network $\phi^{(k)}$ ($k = 1, \dots, 6$). Once the filtration f_θ is computed, the corresponding PDs can be obtained and vectorized in a differentiable way with PersLay [30], and fed to any standard deep learning model ϕ in order to classify point clouds, by minimizing the cross-entropy loss ℓ in Equation (5.1). See the corresponding article [92] for applications on protein and 3D mesh datasets.

5.2 Topological regularization

Topology has also been proved to be a good method for *constraining* models, either for (i) limiting their *complexities*, or for (ii) imposing *topological priors*.

- (i) The more complicated a model is, the more likely it is that PDs computed out of it contain complex information. Hence, there are various contexts in statistical inference and machine learning in which models are improved by penalizing those whose PDs are too rich, which is called *topological regularization*.

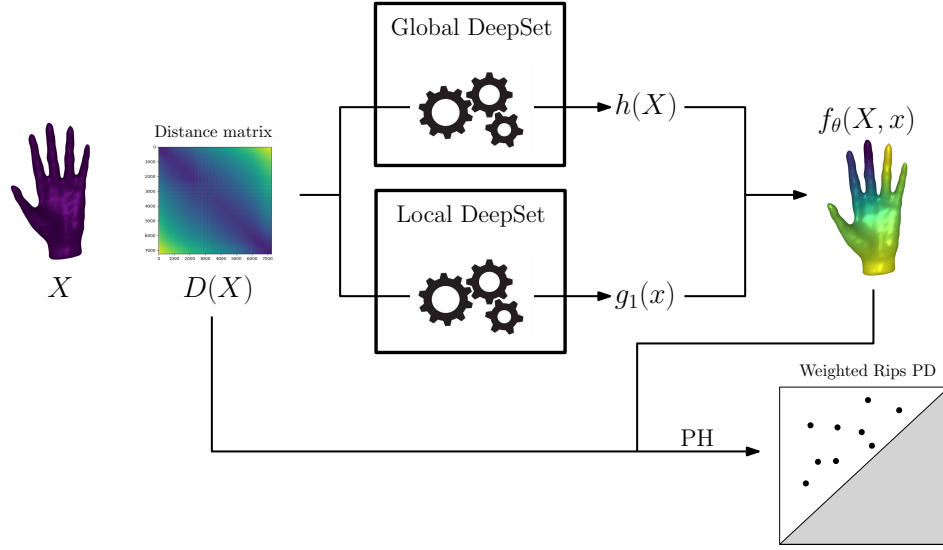


Figure 5.3: Filtration learning for point clouds.

- (ii) In the context of geometric complexes and point clouds, topological regularization can often be interpreted as *topological priors*, i.e., imposing that datasets contain specific geometric features, which is particularly relevant in *dimensionality reduction* (see Section 5.2.2), or in generative models (e.g., generate images that contains topological priors).

Given a parametrized model $\phi(\theta)$, this is usually achieved by optimizing losses of the form:

$$\mathcal{L} : \theta \mapsto L_{\text{data}}(\phi(\theta)) + \lambda_{\text{topo}} L_{\text{topo}}(\text{PH} \circ F(\phi(\theta))), \quad (5.2)$$

where L_{data} is a standard machine learning loss. The coefficient $\lambda_{\text{topo}} > 0$ controls the strength of the topological penalty or prior, and is often picked with cross-validation.

5.2.1 Topological penalization of model complexity

Reference article: Chen et al. [38]

In this work, the authors consider a binary classification problem, i.e., find a model $\phi_\theta : \mathbb{R}^d \rightarrow [-1, 1]$, where $\text{sign}(\phi_\theta(x))$ eventually states the class assigned to a point $x \in \mathbb{R}^d$. Given a finite (training) sample $x_1, \dots, x_n$ and corresponding labels $\{y_1, \dots, y_n\} \subset \{-1, 1\}$, one seek to optimize the model parameters θ so that $\text{sign}(\phi_\theta(x_i)) \simeq y_i$, $i = 1, \dots, n$. For expressive classes of models (e.g., large neural networks), many values of θ may yield near-to-perfect accuracy on the training set; but in order to mitigate overfitting, one may balance between the model training accuracy and the model regularity. Of particular interest is the decision boundary $\{x \in \mathbb{R}^d \mid \phi_\theta(x) = 0\}$: intuitively, models with simpler decision boundaries (that still achieve a decent training accuracy) should be preferred.

In [38], the authors address the regularity of the decision boundary in terms of *topological complexity*: ideally, the decision boundary should have as few topological features (connected components, number of holes...) as possible. Precisely, given ϕ_θ , one can use the model predictions as a filtration for the whole (k -NN graph of the) data space X , i.e., one can use $F : \phi_\theta \mapsto \{\phi_\theta(x)\}_{x \in X}$ in Equation (5.2). As the classification boundary of the model is exactly $\phi_\theta^{-1}(\{0\})$, one can thus set $L_{\text{topo}} = \text{PersTot}$ (see Example 3.10), or the following loss:

$$L_{\text{topo}} : \alpha \mapsto \sum_{(b,d) \in \alpha} \min\{|b|, |d|\},$$

and then apply these losses to $\text{Dgm}(F_\theta)$, where $F_\theta := F(\phi_\theta)$, restricted to the PD points whose birth times are negative and death times are positive. As for the term L_{data} , it is usually defined as any of the standard classification scores from the data science literature. See Figure 5.4. This work showcases in particular an important application of topological regularization on graphs. It relies on the capacity of graphs (such as, e.g., k -nearest neighbor (k -NN) graphs) to model the data space. Namely, when given a function f defined on a finite set of data points, one can extend the domain of the function to a k -NN graph built on top of the data points in order to be able to compute $\text{Dgm}(f)$ (which would not be well-defined otherwise). This in turn allows to use $\text{Dgm}(f)$ to penalize and reduce the complexity of the function f . See also [61] for a similar technique applied to the regression of functions defined on manifolds (and approximated by k -NN graphs computed on finite samples).

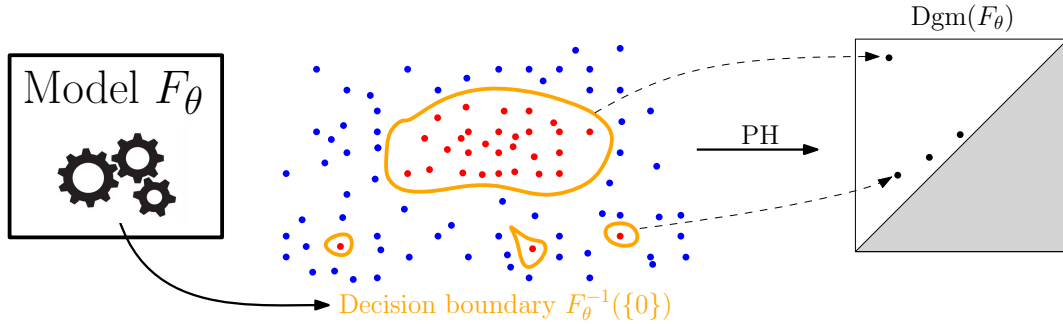


Figure 5.4: A schematic illustration of the use of topological regularization for binary classifiers defined. On the left, a dataset made of two classes (red and blue points, better in color) and the classification boundary of the model (orange). On the right, the PD of that boundary. Intuitively, the small circles used to catch isolated red points are likely to yield overfitting. They are accounted as points close to the diagonal in the persistent diagram. Penalizing the apparition of topological features in the classification boundary that are only used to catch few points is likely to mitigate overfitting. Inspired from [38, Figure 2].

5.2.2 Penalization to favor topological priors

Topology-constrained image generation. Reference article: Wang et al. [133]

Constraining models with topological penalties has found some success on generative models for images. Indeed, considering, e.g., generative adversarial models (GANs), one can complement the standard GAN losses (the discriminator and generator losses) with a topological penalty that forbids the generator to produce images with incorrect topologies. Incorrect topologies are criteria that are difficult to measure and quantify with standard image descriptors, but that can be easily obtained on binary images using, e.g., the distance to the closest black pixel as filtration values (referred to as the distance transform (DT) in the article). See Figure 5.5. More specifically, the topological GAN loss can be written as:

$$L_{\text{topo}}(\text{PH} \circ F(\phi(\theta))) = W(\{\alpha_i\}_i, \{\alpha_j(\phi(\theta))\}_j),$$

where W is an optimal transport distance between two sets of PDs (using the 1st diagram distance FG_1 as ground metric, see Definition 2.20), and i (resp. j) ranges over the index set of the real (resp. fake, or synthetic) images. As the synthetic PDs $\{\alpha_j\}_j$ are computed on the synthetic images, which are themselves produced by the generator ϕ , they depend on the generator parameters θ . Overall, this helps to produce images whose geometric and topological features match better those of the images in the training set. Works sharing a similar goal have been proposed in mechanical sciences [12, 120, 69, 71], where persistence-based losses are included to favor the apparition of certain topological features in material design, or topological diversity in the population of material designed.

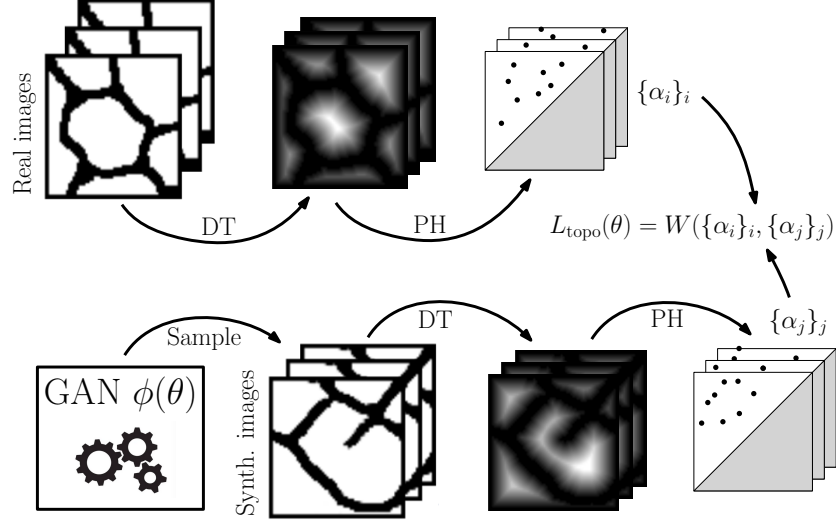


Figure 5.5: Topological regularization for images. Inspired from Wang et al. [133].

Dimensionality reduction. Reference article: Moor et al. [89]

The main goal of dimensionality reduction is to visualize datasets in high dimensions. Given a dataset $X \in \mathbb{R}^{n \times d}$, representing n points with $d \gg 1$ dimensions, the goal is thus to find a *reduced dataset* $\tilde{X} \in \mathbb{R}^{n \times d'}$, with $d' \ll d$, typically $d' = 2$ or 3 . Additionally, this reduced dataset should be faithful, that is, close to X with respect to a given notion of proximity between X and $\tilde{X}$. For instance, multidimensional scaling (MDS) aims at preserving the pairwise distances of X , i.e., the pairwise distances between embedding points in $\tilde{X}$ should be as close as possible to the corresponding distances in the original dataset X . In addition to such losses, it is also natural to ask that the topology, understood through the lens of persistent homology, of $\tilde{X}$ and X should coincide to some extent. This can be achieved with the loss provided in Equation (5.2), applied to point clouds:

$$\mathcal{L}(\theta) := L_{\text{data}}(\tilde{X}(\theta)) + \lambda_{\text{topo}} L_{\text{topo}}(\text{PH} \circ F(\tilde{X}(\theta))), \quad (5.3)$$

where F is a filtration map on the $\tilde{X}(\theta)$, $L_{\text{topo}}: \mathcal{D} \rightarrow \mathbb{R}$ is a loss that controls the PD of the reduced dataset, and L_{data} is a standard geometric loss, such as, e.g., the reconstruction loss of autoencoder neural networks. For instance, in [89], authors consider an encoder $E_\theta: \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$ producing low-dimensional embeddings and set $\tilde{X}(\theta) := E_\theta(X)$, a decoder $D_{\theta'}: \mathbb{R}^{d'} \rightarrow \mathbb{R}^d$, and thus $L_{\text{data}}(\tilde{X}(\theta)) := |D_{\theta'} \circ E_\theta(X) - X|^2$. On the other hand, in order to favor encoders that preserves the topology (namely, the Vietoris–Rips PD) of the input point cloud X , one may take $L_{\text{topo}} = \text{FG}_q(\cdot, \text{Dgm}_{\text{VR}}(X))$ (see Example 3.11)¹³. See Figure 5.6, as well as [55, 127, 131, 34] for other instances of topological losses for dimensionality reduction. Note also that in [34], the use of the diffeomorphic gradient (see Section 4.4.2) allows to decouple Equation (5.3): one can compute the topological regularization on the output of a black-box, pre-trained model ϕ_θ *after it has been trained*, and use the corresponding sequence G of vector fields to produce a *new* model $\tilde{\phi}_\theta := G \circ \phi_\theta$.

Topology-aware segmentation. Reference article: Liu et al. [83]

Consider a dataset consisting of a collection of point clouds in $\mathbb{R}^d$, each containing N labeled points. Let $\mathcal{C} := \{1, \dots, C\}$ be the set of labels and $\mathcal{X}$ be the space of point clouds. The goal of segmentation tasks is to learn a model $\phi_\theta: \mathcal{X} \times \mathbb{R}^d \rightarrow \mathcal{C}$ to estimate the label $\phi_\theta(X, x_k)$ of the point x_k in the point

¹³Strictly speaking, the topological loss used in [89] is slightly different and involves minimizing the differences between the distances associated to the critical pairs in both PDs—we leave this subtlety aside in this survey.

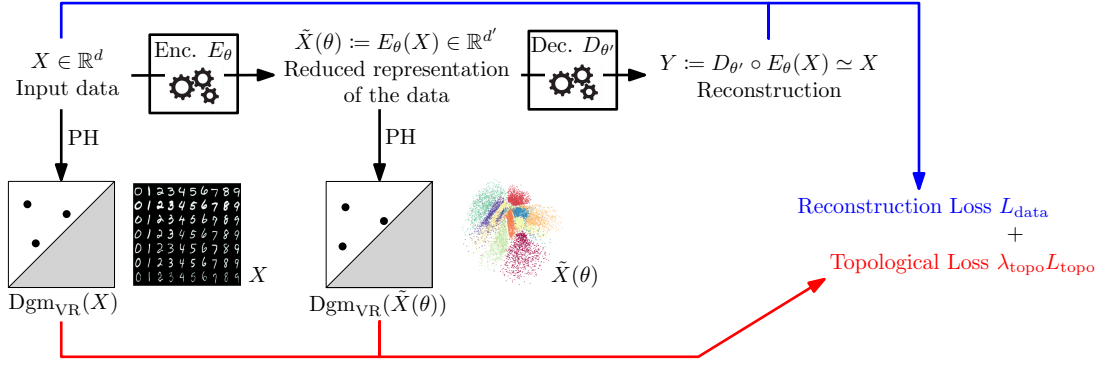


Figure 5.6: Topological regularization for point clouds. Inspired from [89, Figure 2].

cloud X . As usual, θ represents the trainable parameters. Typically, one can use neural networks such as PointNet++ [107] and DGCNN [104] for ϕ_θ .

Let $\{(x_k^i, y_k^i)\}_{k=1}^N\}_{i=1}^M$ be a training dataset, where x_k^i is the k -th point in the i -th point cloud, and $y_k^i \in \mathcal{C}$ denotes its label. Treating this problem as a classification task, the model ϕ_θ can be optimized using a standard cross-entropy loss, denoted by L_{CE} hereafter. However, while neural networks showcase remarkable ability to learn local feature of point clouds, practitioners observe that they often fail to capture global features, especially topological features. To deal with this issue, the authors of [83] propose to incorporate a topological term in their loss function to make the segmented point cloud have close topological structures to the ground truth. Namely, given $t > 0$, one constructs two filtrations $F_\theta^{(c)}(X_i), G^{(c)}(X_i): R[X]_t \rightarrow \mathbb{R}$ on the Vietoris-Rips complex $R[X]_t$ using the output of model ϕ_θ and the ground truth in the following way:

$$F_\theta^{(c)}(X_i)(\sigma) = 1 - \min_{x_{ij} \in \sigma} \phi_\theta^{(c)}(X_i, x_{ij}), \quad G^{(c)}(X_i)(\sigma) = 1 - \min_{x_{ij} \in \sigma} g_{ij}^{(c)}.$$

They define their topological loss L_{topo} as (a variant of) the FG distance between the persistence diagrams of F_θ and G :

$$L_{\text{topo}}(f_\theta, g) := \sum_{k=1}^M \sum_{i=1}^N \sum_{c=1}^C \text{FG}_2(D(F_\theta^{(c)}(X_i)), D(G^{(c)}(X_i))),$$

Minimizing the following combined loss of the form of Equation (5.2)

$$L_{\text{CE}}(\phi_\theta, g) + \lambda_{\text{topo}} L_{\text{topo}}(\phi_\theta, g)$$

favors a segmentation with topological structures close to the ground truth. Mixing these two terms eventually produces a segmentation generally closer to that given by the ground truth than using L_{CE} alone [83, Table 1]. A similar approach has been explored in different works, see for instance [52, 43, 19].

6 Numerical illustrations

In this section, we showcase the empirical behaviors of the different topological gradients and their extensions presented in Section 4. We start with a simple proof-of-concept experiment, and then consider a *topology-preserving* dimensionality reduction problem, a classical application of topology-based optimization with gradient descent (see also Subsection 5.2.2). We stress that the goal of this section is mostly pedagogical: it does not aim at establishing state-of-the-art results but to provide

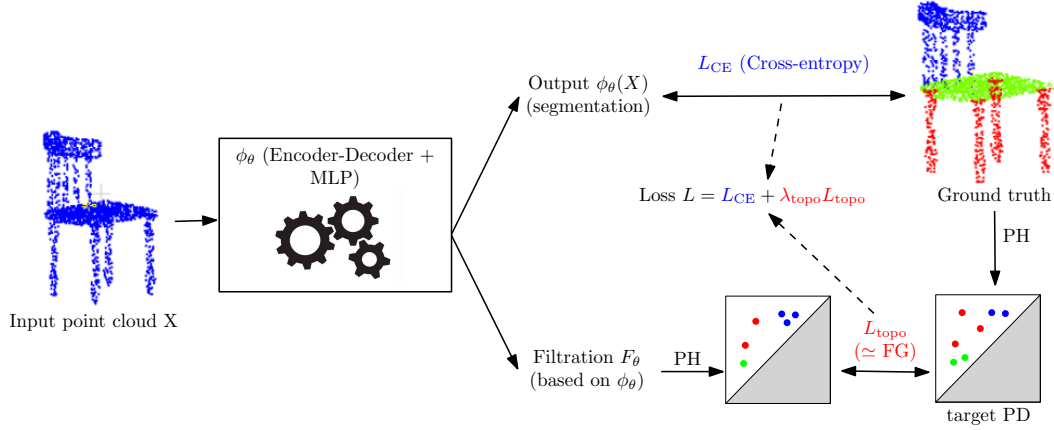


Figure 5.7: Schematic overview of topological segmentation as proposed by [83]. The model ϕ_θ is trained in order to both produce a segmentation close to that given by the ground truth, both in terms of cross-entropy and in terms of similarity between the computed persistence diagrams. Inspired from [83, Figure 2]

an all-in-one-place illustration of the different persistence-based topological optimization variants discussed in this survey. Our code¹⁴ is available at https://github.com/git-westriver/benchmark_ph_optimization.

6.1 Illustration of the different gradient descent methods

A first proof-of-concept. In this experiment, we optimize the coordinates of a point cloud X with respect to a loss that is based on the Vietoris–Rips persistence diagram $\text{Dgm}_{\text{VR}}(X)$. The point cloud X is initialized in $\mathbb{R}^2$ as 100 points close to a unit circle, along with one outlier near the origin. We write $X = (x_0, \dots, x_{100}) \in \mathbb{R}^{2 \times 100}$ and optimize X with the loss function

$$\mathcal{L}(X) := -\text{FG}_2(\text{Dgm}_{\text{VR}}^{(1)}(X), \emptyset) + \text{Reg}(X),$$

where $\text{Dgm}_{\text{VR}}^{(1)}(X)$ is the subset of $\text{Dgm}_{\text{VR}}(X)$ corresponding to homology dimension 1, and $\text{Reg}(X) = \sum_{(x,y) \in X} (|x| - 2)_+^2 + (|y| - 2)_+^2$ is a regularization term that penalizes points escaping the compact set $[-2, 2]^2$. Intuitively, minimizing this loss amounts to maximizing the total 1-persistence in X , i.e., amounts to making the initial loop (that is notably perturbed by the outlier) as salient as possible. We use the different optimization methods described in Section 4. Figure 6.1 shows the snapshots with the different gradient displayed, along with the loss evolution of gradient steps for each optimization method; Table 6.1 displays the execution times over 20 steps.

For each method, we have two hyper-parameters: (a) the learning rate η used to minimize the loss, and chosen from $\eta \in \{0.064, 0.128, 0.256\}$, and (b) the decay rate γ used to decrease η with $\eta \cdot \gamma^{t-1}$ at the t -th step, and chosen from $\gamma \in \{1.0, 0.9, 0.8, 0.7\}$. For each method, we selected the values of η, γ that eventually yield the smallest loss value after 20 gradient steps.

The main observation that one can make from Figure 6.1 is that, as expected, standard gradients (first row) of $\mathcal{L}$ are typically sparse, moving only few points at each step. A similar comment holds for the stratified gradient descent approach and the continuation one. In contrast, the diffeomorphic (fourth row) and big-step (fifth row) gradients provide a gradient-like update that is non-zero on much more points of X , yielding overall more efficient updates.

¹⁴Note that, in its current implementation, our code can only be applied to point clouds.

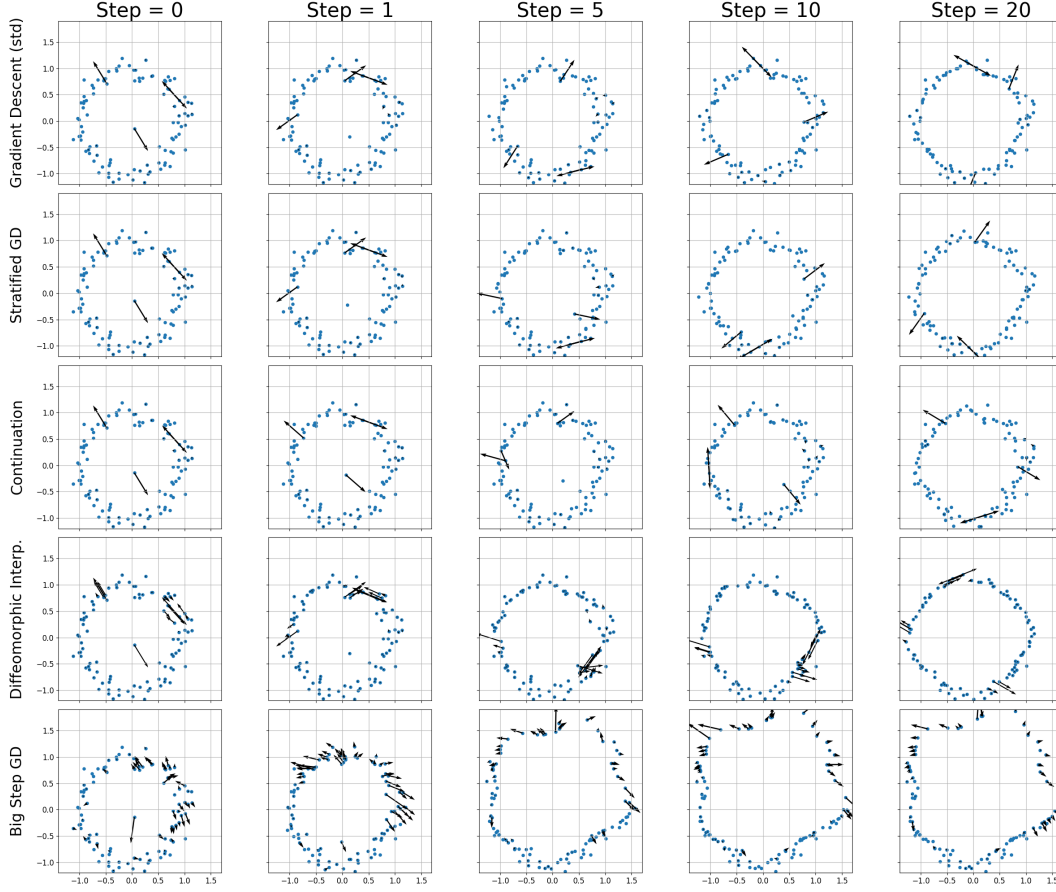


Figure 6.1: Snapshot of X over timesteps 0, 1, 5, 10, 20 for the different methods.

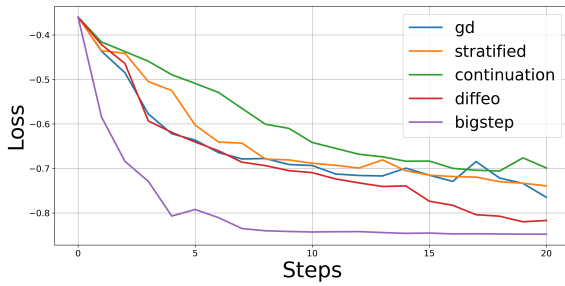


Figure 6.2: Evolutions of the losses over steps.

Table 6.1: Running time over 20 steps.

Method	Time (s)
Vanilla	0.55 ± 0.025
Stratified	4.55 ± 0.326
Big-step	62.1 ± 0.064
Continuation	0.60 ± 0.006
Diffeomorphic	0.64 ± 0.004

Overall, the big-step gradient yields, by a significant margin, the most efficient updates in terms of loss decrease, reaching a near-global configuration in less than 10 iterations. This comes at a computational price: big-step gradients are significantly longer to compute than vanilla ones and all other alternatives showcased here¹⁵.

Scaling topological optimization via subsampling. All the methods considered in this proof-of-concept experiment require to repeatedly compute the Vietoris–Rips persistence diagram of X (in homology dimension 1) in order to get the corresponding gradient. This is known to be prohibitive for large point clouds (with around a few thousands of points) and invite the practitioner to rely on subsampling methods. Point clouds that are close in the Gromov–Hausdorff distance yield close VR persistence diagrams thanks to Theorem 2.24. However, this is not true when it comes to computing gradients: a critical pair in a subsample has little probability to also be critical in the original point cloud. Furthermore, sparsity of standard gradients is an even bigger issue when the input object is large, as only a tiny fraction of points will be updated at each step.

We investigate qualitatively two approaches that help mitigating this phenomenon: the diffeomorphic and distributed gradients (see Subsection 4.4.1). We recall that diffeomorphic interpolation takes a (standard) gradient, possibly computed on a subsample X' of a large point cloud X , provides a vector field defined on the whole space (hence in particular on the support of X , not only X'). The distributed gradient consists, substantially, in averaging the gradients of several subsamples of X , also yielding denser gradients.

For illustrative purpose, we first consider a point cloud $X \subset \mathbb{R}^2$ with $n = 2,000$ points, close to a unit circle, and consider the loss $\mathcal{L}(X) = \text{FG}_2^2(\text{Dgm}_{\text{VR}}^{(1)}(X), \emptyset)$, i.e., the goal is to minimize the persistence of the underlying loop, by both collapsing it (i.e., by reducing the death time) and tearing it (i.e., by increasing the birth time). We fix the subsampling size at $s = 50$, and showcase on Figure 6.3 the vanilla gradient (computed from a given subsample of size s), the diffeomorphic gradient and the distributed gradient averaged over 100 repetitions. Both diffeomorphic and distributed gradients achieve their goal of providing an alternative to the vanilla gradient that is non-zero on much more points in X . The former acts at a local scale, while the latter is more global, a possibly appealing property which comes at the price of requiring to compute more (small) persistence diagrams to get a single gradient-like object.

In order to compare these different methods, we reproduce an experiment proposed in [34]. We let X be the **Stanford bunny** [122], a 3D point cloud made of $n = 35,947$ points, and consider the loss $\mathcal{L}(X) = -\text{FG}_2^2(\text{Dgm}_{\text{VR}}^{(2)}(X), \emptyset) + \text{Reg}(X)$, where (as in the previous experiment) Reg is a confinement term that penalizes points that would go out of the compact set $[-1, 1]^3$. This loss aims at increasing the persistence of the bunny’s cavity (in homology dimension 2). The size of X prevents from a direct computation of $\text{Dgm}_{\text{VR}}^{(2)}(X)$ and gradients of the loss $\mathcal{L}$ must be replaced by estimates obtained from subsamples. We therefore fix a subsampling size of $s = 100$, and compare the vanilla gradient descent with the diffeomorphic gradient descent (with $\sigma = 0.05$, the value used in [34]) and the distributed gradient descent (summing 10 vanilla gradients)¹⁶. Eventually, we also consider the descent scheme using the diffeomorphic interpolation of a distributed gradient (also with 10 repetitions) instead of a vanilla gradient. Results are displayed in Figure 6.4. As in [34], the vanilla gradient descent—which updates the position of only very few points (around 4 among the $\sim 36\text{k}$ points of X) at each iteration—does not produce any perceptible modifications after 200 epochs. A similar comment holds for the distributed gradient approach: as we average only 10 vanilla gradients at each iteration, one may expect only ~ 40 points to be moved at each iteration, which remains too low to produce noticeable changes. As in [34], the diffeomorphic interpolation approach produces satisfactory results on this type of task. Combining the diffeomorphic and distributed gradients yields the best loss decrease *per step*,

¹⁵Note that stratified gradients are also longer to compute due to the need of exploring nearby strata, while barely presenting other benefits in this simple setting. Recall that the purpose of this approach is mostly to provide convergence guarantees, which are not necessary in this experiment.

¹⁶These values make the running time per step of diffeomorphic and distributed gradients similar.

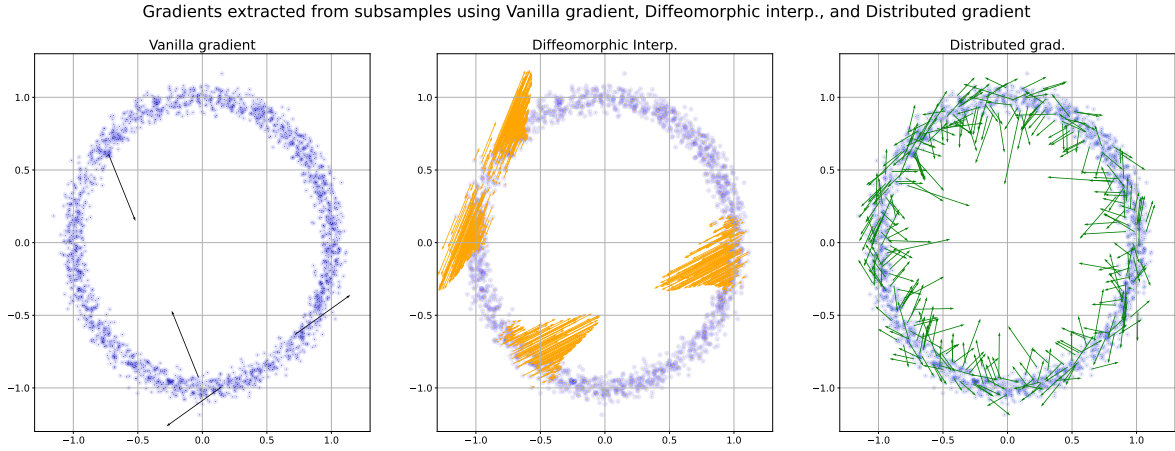


Figure 6.3: (Left) Vanilla gradient computed on a subsample of the original point cloud. Two pairs of points were critical in that subsample (one corresponding to a reduction of the death time, one to an increase of the birth time), yielding only four non-zero components in the gradient. (Middle) The diffeomorphic gradient extends the vanilla one in a smooth way on the input point cloud X , and thus follows a similar pattern while moving substantially more points than the vanilla gradient. (Right) The distributed gradient also exhibits much more non-zero components than the vanilla one.

suggesting that taking combinations of these two methods can be efficient in some situations. Note that in terms of raw running times, distributed gradients (with 10 repetitions) require to compute 10 persistence diagrams and 10 corresponding vanilla gradients at each update step (hence a running time per step about 10 times longer than using diffeomorphic gradients alone), which overall makes diffeomorphic gradients faster to reach a near global optimum on this task. Nonetheless, it suggest that distributed and diffeomorphic gradients can interplay positively.

Remark 6.1. Similarly, the interaction between big-step and diffeomorphic gradients was initiated in [34], but no clear benefit was identified. More generally, investigating the possible interactions (suggested by Figure 4.1) between topological gradients further is an interesting future research track. Moreover, combining variants of the standard gradient descent algorithm (e.g., using momentum¹⁷ as explored in [91, §4.1]) with the topological gradients presented in this article is also another important research avenue.

6.2 Topological Autoencoder

In this section, we reproduce the experimental setting of topological autoencoders described in [29] (see also Subsection 5.2.2 and [89]). More precisely, we generated a point cloud in $\mathbb{R}^3$ comprised of two nested circles (see Figure 6.5) and embedded it non-linearly in $\mathbb{R}^9$ by converting each point $p = (x, y, z)$ into the exponential of the 3×3 anti-symmetric matrix whose coefficients are x, y , and z . The converted point cloud is denoted by $X \subset \mathbb{R}^9$. The Vietoris–Rips persistence diagram of X in homology dimension 1 exhibits two loops (as a smooth transformation of the nested circles depicted in Figure 6.5). The goal of this experiment is to propose an embedding of X in $\mathbb{R}^2$ using an autoencoder that would preserve topology, i.e., that would contain (exactly) two underlying loops.

We use an autoencoder whose encoder $f_\theta: \mathbb{R}^9 \rightarrow \mathbb{R}^2$ and decoder $g_\theta: \mathbb{R}^2 \rightarrow \mathbb{R}^9$ are both made of three fully-connected layers with width 32 and ReLU activations as well as batch normalization. We

¹⁷Indeed, momentum (which accumulates gradients over iterations) provides a natural way to tackle vanilla gradient sparsity.

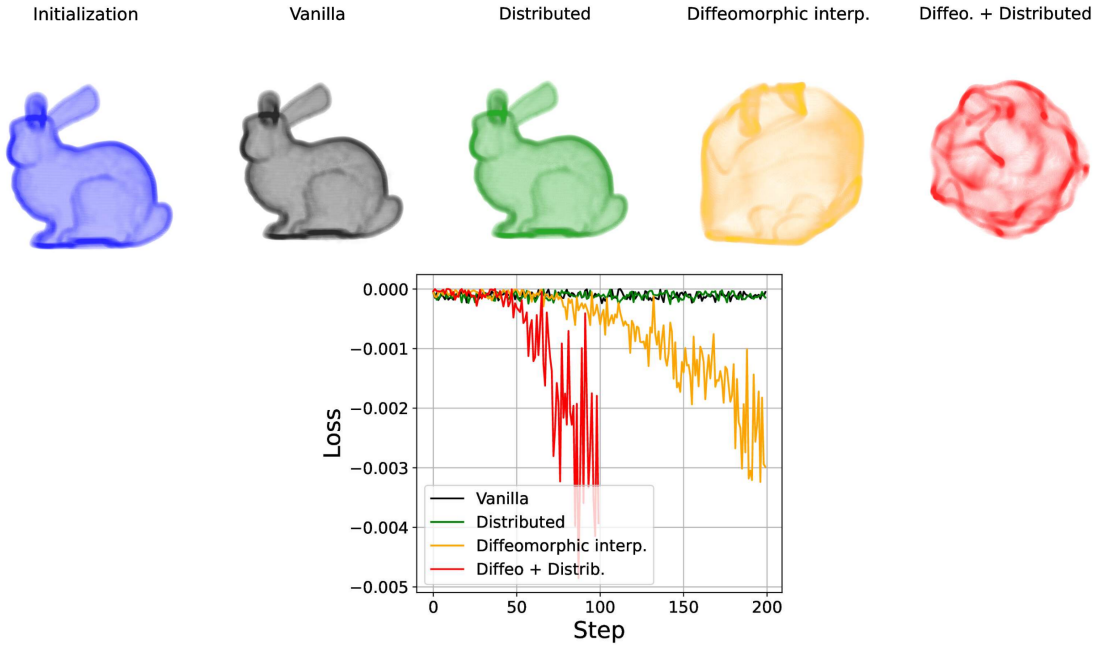


Figure 6.4: (Top row, from left to right) The initial point cloud X , the output of vanilla gradient descent with subsampling after 200 epochs (barely any changes), the output of distributed gradient descent (averaged over 10 samples) after 200 epochs (barely any changes), the output of diffeomorphic gradient descent (using vanilla gradient) after 200 epochs, and the output of diffeomorphic gradient descent (using distributed gradients averaged over 10 samples) after 200 epochs. (Bottom row) Evolution of the losses across iterations.

initialized the autoencoder by pre-training with the usual reconstruction loss (900 epochs). See the leftmost part of Figure 6.6. Then, we trained the autoencoder using the sum of the reconstruction loss and the topological loss computed as

$$\mathcal{L}(\theta) = \|X - (g_\theta \circ f_\theta)(X)\|_2 + \lambda \cdot \text{FG}_2 \left(\text{Dgm}_{\text{VR}}^{(1)}(X), \text{Dgm}_{\text{VR}}^{(1)}(f_\theta(X)) \right),$$

i.e., the second diagram distance between the Vietoris-Rips persistence diagrams (in homology dimension 1) of the original space ($\mathbb{R}^3$) and the latent space ($\mathbb{R}^2$). For the optimization of the loss function with the topological loss, we used the different topological gradients described in Section 4. See Figure 6.6 for the resulting point clouds in $\mathbb{R}^2$ and Table 6.2 for the execution times over 100 epochs for each gradient.

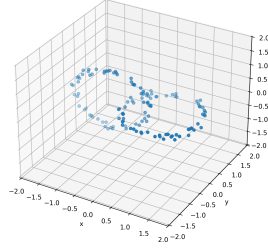


Figure 6.5: The original point cloud in $\mathbb{R}^3$

Table 6.2: Running time over 100 steps.

Method	Time (s)
Vanilla	3.39 ± 0.06
Stratified	4.54 ± 0.6
Big-step	776.5 ± 2.99
Continuation	1.25 ± 0.06
Diffeomorphic	1.89 ± 0.07

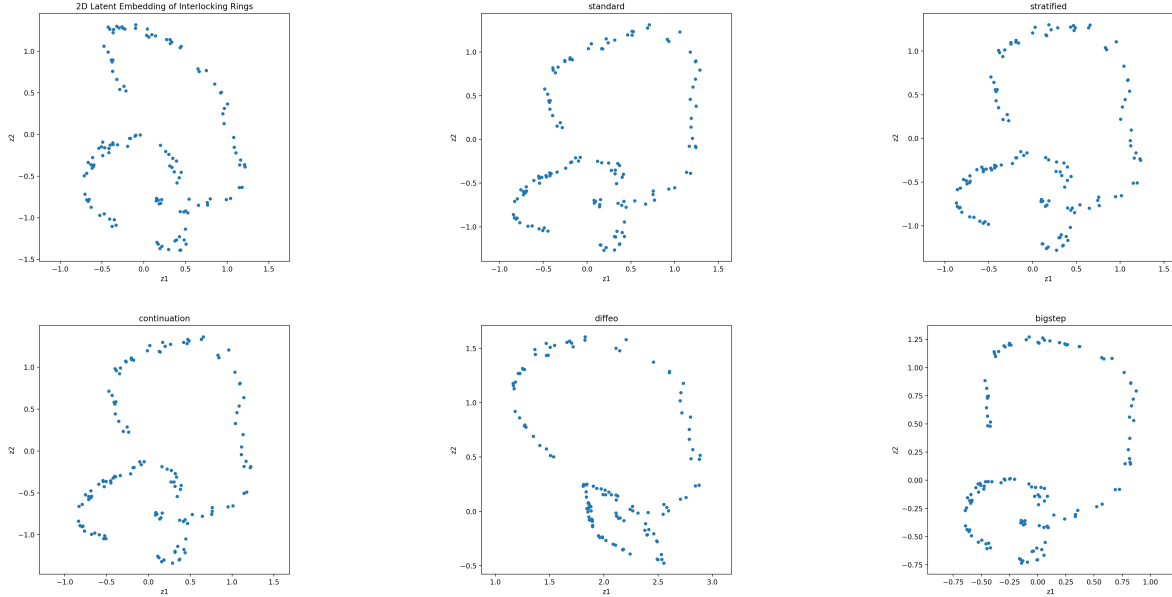


Figure 6.6: Embedding of the point cloud X in $\mathbb{R}^2$ for the different topological gradient descents considered. (Top row, from left to right) No topological regularization, standard and stratified gradient descents. (Bottom row, from left to right) Continuation, diffeomorphic, and big-step gradient descents.

First, one may observe that even without adding an explicit topological penalty (top left of Figure 6.6), the latent space embedding already exhibits a fairly satisfying topology, which is a consequence of the implicit regularity of the encoder. The vanilla gradient descent, the stratified gradient descent and the continuation approach barely improve on this initial try. As in the previous experiment, the

diffeomorphic gradient descent (computed with vanilla gradients) and big-step gradient descent yield the best output (i.e., the lowest topological loss), the latter at the price of a higher computational time. This experiment once again suggests that mitigating sparsity of gradients in topological optimization tends to produce better results.

7 Conclusion

Persistence-based topological optimization is one of the most promising research directions in single-parameter persistent homology. The ability of learning topological descriptors, enforcing topological priors in scientific fields (computer vision, material science, computational biology, etc.) and to use topological quantities to regularize the training of machine learning models are exciting tracks worth of development.

It remains important to stress that this tool comes with important limitations if used naively: the poor computational efficiency inherent to the computation of persistence diagrams on large-scale problems (that typically requires resorting to subsampling), but also the sparsity of vanilla gradients described at the end of Section 4.1 yielding to slow optimization schemes. This second aspect, specific to topological optimization, has attracted interest in the recent years and independent workarounds have been proposed, each with their own merits, though a more uniform and canonical formalism has yet to come.

We hope that this survey along with the library we provide will give the necessary tools for mathematicians and computer scientists interested in working on this topic to get started easily.

Open questions. In addition to the refinement of the tools presented in this survey, we identified some research directions that would contribute to develop the field:

1. **Creating topology.** Destroying topology in a given object (e.g., a point cloud) is a fairly easy task: one just has to take a loss function that penalizes points away from the diagonal, such as the total persistence (see Example 3.10), and update the filtration values accordingly (typically reducing the death time, and possibly increasing the birth time as well). In contrast, creating topology if none is initially present in the filtration (i.e., the corresponding PD is empty) requires, at the level of PDs, to create a brand new point on the diagonal and then push it away from it. Unfortunately, the number of topological features (or the absence of) is generically a locally stable property of the filtration. That is, at the level of the filtration, no topology would be created when performing a gradient descent. This may explain why topological optimization has seen much more success when used as a regularization tool (i.e., limiting/destroying topological features) rather than a way to create topology (e.g., in generative models).
2. **Exploring non-gradient-based optimization.** While gradients are obviously the most important tool in optimization in general, the difficulties they pose in persistence-based optimization (non-smoothness, non-convexity, sparsity, etc) suggest that it may be worth seeking for other optimization routines in TDA. One appealing idea would be to rely on *genetic algorithms*, where the filtration values would evolve randomly and be iteratively selected based on their quality (measured with respect to the considered loss function). This type of algorithm may in particular be a first attempt to tackle the question of creating topology presented above.
3. **Extensions to multiparameter persistence.** This survey focuses on single-parameter persistent homology, that is we restricted to filtrations valued in $\mathbb{R}$. In contrast, multiparameter persistence consider functions valued in $\mathbb{R}^d$. While being more general and appealing, this setting comes with huge theoretical and computational challenges, as no canonical counterparts of PDs can be defined [26, 11]. Several descriptors have been defined in the literature (see for instance [17, 28, 40, 48]). The question of involving these descriptors in the context of topological optimization has recently been considered in [114, 32, 84] and is a natural track to follow.

References

- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. {TensorFlow}: a system for {Large-Scale} machine learning. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*, pages 265–283, 2016.
- [2] Henry Adams, Tegan Emerson, Michael Kirby, Rachel Neville, Chris Peterson, Patrick Shipman, Sofya Chepushtanova, Eric Hanson, Francis Motta, and Lori Ziegelmeier. Persistence Images: A Stable Vector Representation of Persistent Homology. *Journal of Machine Learning Research (JMLR)*, 18(8):1–35, 2017.
- [3] Naoki Akai, Takatsugu Hirayama, and Hiroshi Murase. Experimental stability analysis of neural networks in classification problems with confidence sets for persistence diagrams. *Neural Networks*, 143:42–51, 2021.
- [4] Hirokazu Anai, Frédéric Chazal, Marc Glisse, Yuichi Ike, Hiroya Inakoshi, Raphaël Tinarrage, and Yuhei Umeda. DTM-Based Filtrations. In Gill Barequet and Yusu Wang, editors, *35th International Symposium on Computational Geometry (SoCG 2019)*, volume 129 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 58:1–58:15, Dagstuhl, Germany, 2019. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [5] Rayna Andreeva, Benjamin Dupuis, Rik Sarkar, Tolga Birdal, and Umut Şimşekli. Topological Generalization Bounds for Discrete-Time Stochastic Optimization Algorithms. In Amir Globerson, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, volume 37, pages 4765–4818. Curran Associates, Inc., 2024.
- [6] Daniel Archambault, Tamara Munzner, and David Auber. Topolayout: Multilevel Graph Layout by Topological Features. *IEEE Transactions on Visualization and Computer Graphics*, 13(2):305–317, 2007.
- [7] Nieves Atienza, María José Jiménez, and Manuel Soriano-Trigueros. Stable Topological Summaries for Analyzing the Organization of Cells in a Packed Tissue. *Mathematics*, 9(15):1723, 2021.
- [8] Andrew Aukerman, Mathieu Carrière, Chao Chen, Kevin Gardner, Raúl Rabadán, and Rami Vanguri. Persistent Homology Based Characterization of the Breast Cancer Immune Microenvironment: A Feasibility Study. *Journal of Computational Geometry (JoCG)*, 12(2):183–206, 2021.
- [9] Sivaraman Balakrishnan, Alesandro Rinaldo, Don Sheehy, Aarti Singh, and Larry Wasserman. Minimax Rates for Homology Inference. In *Artificial Intelligence and Statistics*, pages 64–72. PMLR, 2012.
- [10] Giovanni Barbarani, Francesco Vaccarino, Gabriele Trivigno, Marco Guerra, Gabriele Berton, and Carlo Masone. Scale-Free Image Keypoints Using Differentiable Persistent Homology. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, volume 235, pages 2990–3002. PMLR, 2024.
- [11] Ulrich Bauer and Luis Scoccola. Multi-Parameter Persistence Modules are Generically Indecomposable. *International Mathematics Research Notices*, 2025(5):rnaf034, 2025.
- [12] Mohammad Mahdi Behzadi and Horea T Ilieş. Gantl: Toward practical and real-time topology optimization with conditional generative adversarial networks and transfer learning. *Journal of Mechanical Design*, 144(2):021711, 2022.

- [13] Matteo Biagetti, Juan Calles, Lina Castiblanco, Alex Cole, and Jorge Noreña. Fisher Forecasts for Primordial Non-Gaussianity from Persistent Homology. *Journal of Cosmology and Astroparticle Physics (JCAP)*, 2022(10):002, 2022.
- [14] Tolga Birdal, Aaron Lou, Leonidas J. Guibas, and Umut Şimşekli. Intrinsic Dimension, Persistent Homology and Generalization in Neural Networks. In Marc'Aurelio Ranzato, Alina Beygelzimer, Yann Dauphin, Percy S. Liang, and Jenn Wortman Vaughan, editors, *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, volume 34, pages 6776–6789. Curran Associates, Inc., 2021.
- [15] Andrew J Blumberg, Itamar Gal, Michael A Mandell, and Matthew Pancia. Robust statistics, hypothesis testing, and confidence intervals for persistent homology on metric measure spaces. *Foundations of Computational Mathematics*, 14(4):745–789, 2014.
- [16] Jean-Daniel Boissonnat, Frédéric Chazal, and Mariette Yvinec. *Geometric and Topological Inference*, volume 57. Cambridge University Press, 2018.
- [17] Magnus Bakke Botnan, Steffen Oppermann, and Steve Oudot. Signed Barcodes for Multi-Parameter Persistence via Rank Decompositions and Rank-Exact Resolutions. *Foundations of Computational Mathematics (FoCM)*, 2024.
- [18] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.
- [19] Daniel Brito-Pacheco, Panos Giannopoulos, and Constantino Carlos Reyes-Aldasoro. Persistent homology in medical image processing: A literature review. *medRxiv*, pages 2025–02, 2025.
- [20] Peter Bubenik. Statistical Topological Data Analysis using Persistence Landscapes. *Journal of Machine Learning Research (JMLR)*, 16(3):77–102, 2015.
- [21] Peter Bubenik and Tane Vergili. Topological spaces of persistence modules and their properties. *Journal of Applied and Computational Topology*, 2(3):233–269, 2018.
- [22] Peter Bubenik and Alexander Wagner. Embeddings of Persistence Diagrams into Hilbert Spaces. *Journal of Applied and Computational Topology (JACT)*, 4(3):339–351, 2020.
- [23] Anuraag Bukkuri, Noemi Andor, and Isabel K Darcy. Applications of topological data analysis in oncology. *Frontiers in artificial intelligence*, 4:659037, 2021.
- [24] Yueqi Cao, Prudence Leung, and Anthea Monod. K-Means Clustering for Persistent Homology. *Advances in Data Analysis and Classification*, 19(1):95–119, 2025.
- [25] Yueqi Cao and Anthea Monod. A geometric condition for uniqueness of frechet means of persistence diagrams. *Computational Geometry*, 128:102162, 2025.
- [26] Gunnar Carlsson and Afra Zomorodian. The Theory of Multidimensional Persistence. *Discrete & Computational Geometry (DCG)*, 42(1):71–93, 2009.
- [27] Mathieu Carrière and Ulrich Bauer. On the Metric Distortion of Embedding Persistence Diagrams into Separable Hilbert Spaces. In Gill Barequet and Yusu Wang, editors, *35th International Symposium on Computational Geometry (SoCG 2019)*, volume 129 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 21:1–21:15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019.

- [28] Mathieu Carrière and Andrew J. Blumberg. Multiparameter Persistence Image for Topological Machine Learning. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, volume 33, pages 22432–22444. Curran Associates, Inc., 2020.
- [29] Mathieu Carrière, Frédéric Chazal, Marc Glisse, Yuichi Ike, Hariprasad Kannan, and Yuhei Umeda. Optimizing Persistent Homology Based Functions. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning (ICML 2021)*, volume 139, pages 1294–1303. PMLR, 2021.
- [30] Mathieu Carrière, Frédéric Chazal, Yuichi Ike, Théo Lacombe, Martin Royer, and Yuhei Umeda. PersLay: A Neural Network Layer for Persistence Diagrams and New Graph Topological Signatures. In Silvia Chiappa and Roberto Calandra, editors, *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS 2020)*, volume 108, pages 2786–2796. PMLR, 2020.
- [31] Mathieu Carrière, Marco Cuturi, and Steve Oudot. Sliced Wasserstein Kernel for Persistence Diagrams. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning (ICML 2017)*, volume 70, pages 664–673. PMLR, 2017.
- [32] Mathieu Carrière, Seunghyun Kim, and Woojin Kim. Sparsification of the Generalized Persistence Diagrams for Scalability Through Gradient Descent. In Oswin Aichholzer and Haitao Wang, editors, *41st International Symposium on Computational Geometry (SoCG 2025)*, volume 332 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 29:1–29:17, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [33] Mathieu Carrière, Steve Oudot, and Maks Ovsjanikov. Stable Topological Signatures for Points on 3D Shapes. *Computer Graphics Forum*, 34(5):1–12, 2015.
- [34] Mathieu Carrière, Marc Theveneau, and Théo Lacombe. Diffeomorphic Interpolation for Efficient Persistence-Based Topological Optimization. In Amir Globerson, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, volume 37, pages 27274–27294. Curran Associates, Inc., 2024.
- [35] F. Chazal, D. Cohen-Steiner, and A. Lieutier. A sampling theory for compact sets in Euclidean space. *Discrete and Computational Geometry*, 41(3):461–479, 2009.
- [36] Frédéric Chazal, Vin De Silva, Marc Glisse, and Steve Oudot. *The Structure and Stability of Persistence Modules*. SpringerBriefs in Mathematics. Springer International Publishing, 2016.
- [37] Frédéric Chazal, Brittany Terese Fasy, Fabrizio Lecci, Alessandro Rinaldo, and Larry Wasserman. Stochastic Convergence of Persistence Landscapes and Silhouettes. In *Proceedings of the 30th Annual Symposium on Computational Geometry (SoCG 2014)*, pages 474–483. Association for Computing Machinery, 2014.
- [38] Chao Chen, Xiuyan Ni, Qinxun Bai, and Yusu Wang. A Topological Regularizer for Classifiers via Persistent Homology. In Kamalika Chaudhuri and Masashi Sugiyama, editors, *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics (AISTATS 2019)*, volume 89, pages 2573–2582. PMLR, 2019.
- [39] Yen-Chi Chen, Daren Wang, Alessandro Rinaldo, and Larry Wasserman. Statistical Analysis of Persistence Intensity Functions. *arXiv:1510.02502*, 2015.
- [40] Yuzhou Chen, Ignacio Segovia-Dominguez, Cuneyt Gurcan Akcora, Zhiwei Zhen, Murat Kantarcioglu, Yulia Gel, and Baris Coskunuzer. EMP: Effective Multidimensional Persistence for Graph Representation Learning. In *Learning on Graphs Conference*, pages 24–1. PMLR, 2024.

- [41] Samir Chowdhury. Geodesics in Persistence Diagram Space. *arXiv:1905.10820*, 2019.
- [42] Yu-Min Chung, Chuan-Shen Hu, Emily Sun, and Henry C. Tseng. Morphological Multiparameter Filtration and Persistent Homology in Mitochondrial Image Analysis. *PLoS One*, 19(9):e0310157, 2024.
- [43] James Clough, Nicholas Byrne, Ilkay Oksuz, Veronika A Zimmer, Julia A Schnabel, and Andrew King. A topological loss function for deep-learning based image segmentation using persistent homology. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.
- [44] David Cohen-Steiner, Herbert Edelsbrunner, and John L. Harer. Stability of Persistence Diagrams. *Discrete & Computational Geometry (DCG)*, 37(1):103–120, 2007.
- [45] David Cohen-Steiner, Herbert Edelsbrunner, John L. Harer, and Yuriy Mileyko. Lipschitz Functions Have L_p -Stable Persistence. *Foundations of Computational Mathematics (FoCM)*, 10(2):127–139, 2010.
- [46] David Cohen-Steiner, Herbert Edelsbrunner, and Dmitriy Morozov. Vines and Vineyards by Updating Persistence in Linear Time. In *Proceedings of the 22nd Annual Symposium on Computational Geometry (SoCG 2006)*, pages 119–126. Association for Computing Machinery, 2006.
- [47] David Cohen-Steiner, André Lieutier, and Julien Vuillamy. Lexicographic optimal homological chains and applications to point cloud triangulations. *Discrete & computational geometry*, 68(4):1155–1174, 2022.
- [48] René Corbet, Ulderico Fugacci, Michael Kerber, Claudia Landi, and Bei Wang. A Kernel for Multi-Parameter Persistent Homology. *Computers & Graphics: X*, 2:100005, 2019.
- [49] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. *Advances in neural information processing systems*, 26, 2013.
- [50] Thomas Davies, Jack Aspinall, Bryan Wilder, and Tran-Thanh Long. Fuzzy C-Means Clustering in Persistence Diagram Space for Deep Learning Model Selection. In Sophia Sanborn, Christian Shewmake, Simone Azeglio, Arianna di Bernardo, and Nina Miolane, editors, *Proceedings of the 1st NeurIPS Workshop on Symmetry and Geometry in Neural Representations*, volume 197, pages 137–157. PMLR, 2023.
- [51] Damek Davis, Dmitriy Drusvyatskiy, Sham Kakade, and Jason D. Lee. Stochastic Subgradient Method Converges on Tame Functions. *Foundations of Computational Mathematics (FoCM)*, 20(1):119–154, 2020.
- [52] Andac Demir, Elie Massaad, and Bulent Kiziltan. Topology-aware focal loss for 3d image segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 580–589, 2023.
- [53] Vincent Divol. Minimax Adaptive Estimation in Manifold Inference. *Electronic Journal of Statistics*, 15(2):5888–5932, 2021.
- [54] Vincent Divol and Théo Lacombe. Understanding the Topology and the Geometry of the Space of Persistence Diagrams via Optimal Partial Transport. *Journal of Applied and Computational Topology (JACT)*, 5(1):1–53, 2021.
- [55] Harish Doraiswamy, Julien Tierny, Paulo J. S. Silva, Luis Gustavo Nonato, and Claudio Silva. TopoMap: A 0-Dimensional Homology Preserving Projection of High-Dimensional Data. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):561–571, 2021.
- [56] Herbert Edelsbrunner and John L. Harer. *Computational Topology: An Introduction*. American Mathematical Society, 2010.

- [57] Emilio Ferrara, Giacomo Fiumara, et al. Topological features of online social networks. *Communications in Applied and Industrial Mathematics*, 2:1–20, 2011.
- [58] Alessio Figalli and Nicola Gigli. A New Transportation Distance Between Non-Negative Measures, with Applications to Gradients Flows with Dirichlet Boundary Conditions. *Journal de Mathématiques Pures et Appliquées*, 94(2):107–130, 2010.
- [59] Marcio Gameiro, Yasuaki Hiraoka, and Ippei Obayashi. Continuation of Point Clouds via Persistence Diagrams. *Physica D: Nonlinear Phenomena*, 334:118–132, 2016.
- [60] Saumya Gupta, Dimitris Samaras, and Chao Chen. TopoDiffusionNet: A Topology-Aware Diffusion Model. In *The 13th International Conference on Learning Representations (ICLR 2025)*. OpenReview.net, 2025.
- [61] Olympio Hacquard, Krishnakumar Balasubramanian, Gilles Blanchard, Clément Levrard, and Wolfgang Polonik. Topologically Penalized Regression on Manifolds. *Journal of Machine Learning Research (JMLR)*, 23(161):1–39, 2022.
- [62] Yasuaki Hiraoka, Yusuke Imoto, Théo Lacombe, Killian Meehan, and Toshiaki Yachimura. Topological node2vec: Enhanced graph embedding via persistent homology. *Journal of Machine Learning Research*, 25(134):1–26, 2024.
- [63] Yasuaki Hiraoka, Takenobu Nakamura, Akihiko Hirata, Emerson G. Escobar, Kaname Matsue, and Yasumasa Nishiura. Hierarchical Structures of Amorphous Solids Characterized by Persistent Homology. *Proceedings of the National Academy of Sciences*, 113(26):7035–7040, 2016.
- [64] Christoph Hofer, Florian Graf, Bastian Rieck, Marc Niethammer, and Roland Kwitt. Graph Filtration Learning. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning (ICML 2020)*, volume 119, pages 4314–4323. PMLR, 2020.
- [65] Christoph Hofer, Roland Kwitt, and Marc Niethammer. Learning Representations of Persistence Barcodes. *Journal of Machine Learning Research (JMLR)*, 20(126):1–45, 2019.
- [66] Christoph Hofer, Roland Kwitt, and Marc Niethammer. Learning Representations of Persistence Barcodes. *Journal of Machine Learning Research (JMLR)*, 20(126):1–45, 2019.
- [67] Christoph Hofer, Roland Kwitt, Marc Niethammer, and Andreas Uhl. Deep Learning with Topological Signatures. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna Wallach, Rob Fergus, S.V.N. Vishwanathan, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, volume 30. Curran Associates, Inc., 2017.
- [68] Max Horn, Edward de Brouwer, Michael Moor, Bastian Rieck, and Karsten Borgwardt. Topological Graph Neural Networks. In *The 10th International Conference on Learning Representations (ICLR 2022)*. OpenReviews.net, 2022.
- [69] Jiangbei Hu, Ying He, Baixin Xu, Shengfa Wang, Na Lei, and Zhongxuan Luo. If-tonir: iteration-free topology optimization based on implicit neural representations. *Computer-Aided Design*, 167:103639, 2024.
- [70] Patrick Iglesias-Zemmour. *Diffeology*, volume 185. American Mathematical Society, 2013.
- [71] Taisei Kii, Kentaro Yaji, Hiroshi Teramoto, and Kikuo Fujita. Data-driven topology design with persistent homology for enhancing population diversity. *International Journal of Mechanical Sciences*, page 110493, 2025.

- [72] Kwangho Kim, Jisu Kim, Manzil Zaheer, Joon Sik Kim, Frédéric Chazal, and Larry Wasserman. PLLay: Efficient Topological Layer based on Persistent Landscapes. In Hugo Larochelle, Marc’Aurelio Ranzato, Raia Hadsell, Maria Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, volume 33, pages 15965–15977. Curran Associates, Inc., 2020.
- [73] Genki Kusano, Kenji Fukumizu, and Yasuaki Hiraoka. Kernel Method for Persistence Diagrams via Kernel Embedding and Weight Factor. *Journal of Machine Learning Research (JMLR)*, 18(189):1–41, 2018.
- [74] Genki Kusano, Yasuaki Hiraoka, and Kenji Fukumizu. Persistence Weighted Gaussian Kernel for Topological Data Analysis. In Maria Florina Balcan and Kilian Weinberger, editors, *Proceedings of the 33rd International Conference on Machine Learning (ICML 2016)*, volume 48, pages 2004–2013. PMLR, 2016.
- [75] Théo Lacombe. An homogeneous unbalanced regularized optimal transport model with applications to optimal transport with boundary. In *International Conference on Artificial Intelligence and Statistics*, pages 7311–7330. PMLR, 2023.
- [76] Théo Lacombe, Marco Cuturi, and Steve Oudot. Large Scale Computation of Means and Clusters for Persistence Diagrams using Optimal Transport. In Samy Bengio, Hanna Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, volume 31. Curran Associates, Inc., 2018.
- [77] Théo Lacombe, Yuichi Ike, Mathieu Carrière, Frédéric Chazal, Marc Glisse, and Yuhei Umeda. Topological Uncertainty: Monitoring Trained Neural Networks through Persistence of Activation Graphs. In Zhi-Hua Zhou, editor, *30th International Joint Conference on Artificial Intelligence (IJCAI 2021)*, pages 2666–2672. International Joint Conferences on Artificial Intelligence Organization, 2021.
- [78] Tam Le and Makoto Yamada. Persistence Fisher Kernel: A Riemannian Manifold Kernel for Persistence Diagrams. In Samy Bengio, Hanna Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, volume 31. Curran Associates, Inc., 2018.
- [79] Jacob Leygonie, Mathieu Carrière, Théo Lacombe, and Steve Oudot. A Gradient Sampling Algorithm for Stratified Maps with Applications to Topological Data Analysis. *Mathematical Programming*, 202(1):199–239, 2023.
- [80] Jacob Leygonie, Steve Oudot, and Ulrike Tillmann. A Framework for Differential Calculus on Persistence Barcodes. *Foundations of Computational Mathematics (FoCM)*, 22(4):1069–1131, 2022.
- [81] Geng Li, Murat Semerci, Bülent Yener, and Mohammed J. Zaki. Effective Graph Classification Based on Topological and Label Attributes. *Statistical Analysis and Data Mining: The ASA Data Science Journal*, 5(4):265–283, 2012.
- [82] Jen-Yu Liu, Shyh-Kang Jeng, and Yi-Hsuan Yang. Applying Topological Persistence in Convolutional Neural Network for Music Audio Signals. *arXiv:1608.07373*, 2016.
- [83] Wei-quan Liu, Hanyun Guo, Weini Zhang, Yu Zang, Cheng Wang, and Jonathan Li. TopoSeg: Topology-Aware Segmentation for Point Clouds. In Luc de Raedt, editor, *31st International Joint Conference on Artificial Intelligence (IJCAI 2022)*, pages 1201–1208. International Joint Conferences on Artificial Intelligence Organization, 2022.

- [84] David Loiseaux and Hannah Schreiber. Multipers: Multiparameter Persistence for Machine Learning. *Journal of Open Source Software (JOSS)*, 9(103):6773, 2024.
- [85] Andrew Marchese, Vasileios Maroulas, and Josh Mike. K-Means Clustering on the Space of Persistence Diagrams. In *Wavelets and Sparsity XVII*, volume 10394, pages 218–227. SPIE, 2017.
- [86] Yuriy Mileyko, Sayan Mukherjee, and John L. Harer. Probability Measures on the Space of Persistence Diagrams. *Inverse Problems*, 27(12):124007, 2011.
- [87] Atish Mitra and Žiga Virk. The Space of Persistence Diagrams on n Points Coarsely Embeds into Hilbert Space. *Proceedings of the American Mathematical Society*, 149(6):2693–2703, 2021.
- [88] Atish Mitra and Žiga Virk. Geometric Embeddings of Spaces of Persistence Diagrams with Explicit Distortions. *arXiv:2401.05298*, 2024.
- [89] Michael Moor, Max Horn, Bastian Rieck, and Karsten Borgwardt. Topological Autoencoders. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning (ICML 2020)*, volume 119, pages 7045–7054. PMLR, 2020.
- [90] Elizabeth Munch, Katharine Turner, Paul Bendich, Sayan Mukherjee, Jonathan Mattingly, and John Harer. Probabilistic Fréchet Means for Time Varying Persistence Diagrams. *Electronic Journal of Statistics*, 9(1):1173–1204, 2015.
- [91] Arnur Nigmatov and Dmitriy Morozov. Topological Optimization with Big Steps. *Discrete & Computational Geometry (DCG)*, 72(1):310–344, 2024.
- [92] Naoki Nishikawa, Yuichi Ike, and Kenji Yamanishi. Adaptive Topological Feature via Persistent Homology: Filtration Learning for Point Clouds. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, volume 36, pages 9131–9143. Curran Associates, Inc., 2023.
- [93] Partha Niyogi, Stephen Smale, and Shmuel Weinberger. Finding the Homology of Submanifolds with High Confidence from Random Samples. *Discrete & Computational Geometry (DCG)*, 39(1):419–441, 2008.
- [94] Ippei Obayashi and Michio Yoshiwaki. Field choice problem in persistent homology. *Discrete & Computational Geometry*, 70(3):645–670, 2023.
- [95] Małgorzata Olejniczak and Julien Tierny. Topological Data Analysis of Vortices in the Magnetically-Induced Current Density in LiH Molecule. *Physical Chemistry Chemical Physics*, 25(8):5942–5947, 2023.
- [96] Steve Oudot. *Persistence Theory: From Quiver Representations to Data Analysis*, volume 209 of *Mathematical Surveys and Monographs*. American Mathematical Society, 2015.
- [97] Steve Oudot, Leonidas J. Guibas, Jie Gao, and Yue Wang. Geodesic Delaunay Triangulations in Bounded Planar Domains. *ACM Transactions on Algorithms (TALG)*, 6(4):1–47, 2010.
- [98] Aaron Ouellette, Gilbert Holder, and Ely Kerman. Topological Data Analysis Reveals Differences between Simulated Galaxies and Dark Matter Haloes. *Monthly Notices of the Royal Astronomical Society*, 523(4):5738–5747, 2023.
- [99] Valerio Pascucci, Xavier Tricoche, Hans Hagen, and Julien Tierny. *Topological Methods in Data Analysis and Visualization: Theory, Algorithms, and Applications*. Springer Science & Business Media, 2010.

- [100] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. In *NIPS 2017 Autodiff Workshop*, 2017.
- [101] Jose A. Perea, Elizabeth Munch, and Firas A. Khasawneh. Approximating Continuous Functions on Persistence Diagrams using Template Functions. *Foundations of Computational Mathematics (FoCM)*, 23(4):1215–1272, 2023.
- [102] Gabriel Peyré and Marco Cuturi. Computational optimal transport: With applications to data science. *Foundations and Trends® in Machine Learning*, 11(5-6):355–607, 2019.
- [103] Phu Pham, Quang-Thinh Bui, Ngoc Thanh Nguyen, Robert Kozma, Philip S. Yu, and Bay Vo. Topological Data Analysis in Graph Neural Networks: Surveys and Perspectives. *IEEE Transactions on Neural Networks and Learning Systems*, pages 1–19, 2025.
- [104] Anh Viet Phan, Minh Le Nguyen, Yen Lam Hoang Nguyen, and Lam Thu Bui. DGCNN: A Convolutional Neural Network Over Large-Scale Labeled Graphs. *Neural Networks*, 108:533–543, 2018.
- [105] Adrien Poulenard, Primož Skraba, and Maks Ovsjanikov. Topological Function Optimization for Continuous Shape Matching. *Computer Graphics Forum*, 37(5):13–25, 2018.
- [106] Neil Pritchard and Thomas Weighill. Coarse Embeddability of Wasserstein Space and the Space of Persistence Diagrams. *Discrete & Computational Geometry (DCG)*, 2024.
- [107] Charles R. Qi, Li Yi, Hao Su, and Leonidas J. Guibas. PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna Wallach, Rob Fergus, S.V.N. Vishwanathan, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, volume 30. Curran Associates, Inc., 2017.
- [108] Jan Reininghaus, Stefan Huber, Ulrich Bauer, and Roland Kwitt. A Stable Multi-Scale Kernel for Topological Machine Learning. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2015)*, pages 4741–4748. IEEE, 2015.
- [109] Bastian Rieck, Christian Bock, and Karsten Borgwardt. A Persistent Weisfeiler-Lehman Procedure for Graph Classification. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning (ICML 2019)*, volume 97, pages 5448–5458. PMLR, 2019.
- [110] Bastian Rieck, Matteo Togninalli, Christian Bock, Michael Moor, Max Horn, Thomas Gumbach, and Karsten M. Borgwardt. Neural Persistence: A Complexity Measure for Deep Neural Networks using Algebraic Topology. In *The 7th International Conference on Learning Representations (ICLR 2019)*. OpenReview.net, 2019.
- [111] Martin Royer, Frédéric Chazal, Clément Levrard, Yuhei Umeda, and Yuichi Ike. ATOL: Measure Vectorization for Automatic Topologically-Oriented Learning. In Arindam Banerjee and Kenji Fukumizu, editors, *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics (AISTATS 2021)*, volume 130, pages 1000–1008. PMLR, 2021.
- [112] Mohammad Saadatfar, Hiroshi Takeuchi, Vanessa Robins, Nicolas Francois, and Yasuaki Hirakawa. Pore Configuration Landscape of Granular Crystallization. *Nature Communications*, 8(1):15082, 2017.
- [113] Filippo Santambrogio. *Optimal Transport for Applied Mathematicians: Calculus of Variations, PDEs, and Modeling*, volume 87 of *Progress in Nonlinear Differential Equations and Their Applications*. Springer International Publishing, Cham, 2015.

- [114] Luis Scoccola, Siddharth Setlur, David Loiseaux, Mathieu Carrière, and Steve Oudot. Differentiability and Optimization of Multiparameter Persistent Homology. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, pages 43986–44011. PMLR, 2024.
- [115] Ole Sigmund and Kurt Maute. Topology optimization approaches: A comparative review. *Structural and multidisciplinary optimization*, 48(6):1031–1055, 2013.
- [116] Yashbir Singh, Colleen M. Farrelly, Quincy A. Hathaway, Tim Leiner, Jaidip Jagtap, Gunnar Carlsson, and Bradley J. Erickson. Topological Data Analysis in Medical Imaging: Current State of the Art. *Insights into Imaging*, 14(1):58, 2023.
- [117] Primož Skraba, Maks Ovsjanikov, Frédéric Chazal, and Leonidas J. Guibas. Persistence-Based Segmentation of Deformable Shapes. In *IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW 2010)*, pages 45–52. IEEE, 2010.
- [118] Primož Skraba and Katharine Turner. Wasserstein Stability for Persistence Diagrams. *arXiv:2006.16824*, 2020.
- [119] Elchanan Solomon, Alexander Wagner, and Paul Bendich. A Fast and Robust Method for Global Topological Functional Optimization. In Arindam Banerjee and Kenji Fukumizu, editors, *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics (AISTATS 2021)*, volume 130, pages 109–117. PMLR, 2021.
- [120] Tomotaka Sugai, Kohei Shintani, and Takayuki Yamada. Data-driven topology optimization using a multitask conditional variational autoencoder with persistent homology. *Structural and Multidisciplinary Optimization*, 67(7):133, 2024.
- [121] Julien Tierny, Guillaume Favelier, Joshua A Levine, Charles Gueunet, and Michael Michaux. The Topology Toolkit. *IEEE transactions on visualization and computer graphics*, 24(1):832–842, 2017.
- [122] Greg Turk and Marc Levoy. Zippered polygon meshes from range images. In *Proceedings of the 21st annual conference on Computer graphics and interactive techniques*, pages 311–318, 1994.
- [123] Katharine Turner. Means and Medians of Sets of Persistence Diagrams. *arXiv:1307.8300*, 2013.
- [124] Katharine Turner, Yuriy Mileyko, Sayan Mukherjee, and John L. Harer. Fréchet Means for Distributions of Persistence Diagrams. *Discrete & Computational Geometry (DCG)*, 52(1):44–70, 2014.
- [125] Katharine Turner, Sayan Mukherjee, and Doug M. Boyer. Persistent Homology Transform for Modeling Shapes and Surfaces. *Information and Inference: A Journal of the IMA*, 3(4):310–344, 2014.
- [126] Yuhei Umeda. Time Series Classification via Topological Data Analysis. *Transactions of the Japanese Society for Artificial Intelligence*, 32(3):D–G72.1–12, 2017.
- [127] Robin Vandaele, Bo Kang, Jefrey Lijffijt, Tijl de Bie, and Yvan Saeys. Topologically Regularized Data Embeddings. In *The 10th International Conference on Learning Representations (ICLR 2022)*. OpenReview.net, 2022.
- [128] Yogesh Verma, Amauri H Souza, and Vikas Garg. Topological neural networks go persistent, equivariant, and continuous. In *Proceedings of the 41st International Conference on Machine Learning*, pages 49388–49407, 2024.

- [129] Oliver Vipond, Joshua A. Bull, Philip S. Macklin, Ulrike Tillmann, Christopher W. Pugh, Helen M. Byrne, and Heather A. Harrington. Multiparameter Persistent Homology Landscapes Identify Immune Cell Spatial Patterns in Tumors. *Proceedings of the National Academy of Sciences*, 118(41):e2102166118, 2021.
- [130] Siddharth Vishwanath, Kenji Fukumizu, Satoshi Kuriki, and Bharath Sriperumbudur. Robust Persistence Diagrams using Reproducing Kernels. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, volume 33, pages 21900–21911. Curran Associates, Inc., 2020.
- [131] Alexander Wagner, Elchanan Solomon, and Paul Bendich. Improving Metric Dimensionality Reduction with Distributed Topology. *arXiv:2106.07613*, 2021.
- [132] Fan Wang, Huidong Liu, Dimitris Samaras, and Chao Chen. TopoGAN: A Topology-Aware Generative Adversarial Network. In *16th European Conference on Computer Vision (ECCV 2020)*, pages 118–136, Berlin, Heidelberg, 2020. Springer-Verlag.
- [133] Fan Wang, Huidong Liu, Dimitris Samaras, and Chao Chen. TopoGAN: A Topology-Aware Generative Adversarial Network. In *16th European Conference on Computer Vision (ECCV 2020)*, pages 118–136, Berlin, Heidelberg, 2020. Springer-Verlag.
- [134] Jacky H.T. Yip, Matteo Biagetti, Alex Cole, Karthik Viswanathan, and Gary Shiu. Cosmology with Persistent Homology: A Fisher Forecast. *Journal of Cosmology and Astroparticle Physics (JCAP)*, 2024(9):034, 2024.
- [135] Laurent Younes. *Shapes and Diffeomorphisms*, volume 171 of *Applied Mathematical Sciences*. Springer International Publishing, 2019.
- [136] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabás Póczos, Ruslan Salakhutdinov, and Alexander J. Smola. Deep Sets. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna Wallach, Rob Fergus, S.V.N. Vishwanathan, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, volume 30. Curran Associates, Inc., 2017.
- [137] Qi Zhao and Yusu Wang. Learning Metrics for Persistence-Based Summaries and Applications for Graph Classification. In Hanna Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, volume 32. Curran Associates, Inc., 2019.
- [138] Qi Zhao, Ze Ye, Chao Chen, and Yusu Wang. Persistence Enhanced Graph Neural Network. In Silvia Chiappa and Roberto Calandra, editors, *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS 2020)*, volume 108, pages 2896–2906. PMLR, 2020.